

CYCLICL - TOWARDS AUTOMATING
REQUIREMENTS TRACEABILITY IN
PRODUCT LINE ENGINEERING

By

THOMAS CHIANG

A Thesis

Submitted to the School of Graduate Studies
in Partial Fulfillment of the Requirements for the Degree

PhD

McMaster University

© Copyright by Thomas Chiang, June, 2025

PHD (June, 2025)

McMaster University

(Software Engineering)

Hamilton, Ontario

TITLE: CyclicL - Towards Automating Requirements Traceability in Product Line Engineering

AUTHOR: Thomas Chiang, B.Eng Computer Engineering, M.A.Sc
Software Engineering (McMaster University)

SUPERVISORS: Richard Paige, Alan Wassyng

NUMBER OF PAGES: [viii](#), [202](#)

Lay Abstract

This thesis investigates a Model-Driven Engineering approach to managing traceability variability between product families and requirements. The approach manages traceability from the concept phase — that is, a product idea — and produces traceable elements as a side-effect of Model-Driven Engineering principles and practices. It shows how to connect goals, use cases, requirements, and feature models to enable end-to-end traceability in product line engineering.

Abstract

Product Line Engineering (PLE) has become a common engineering approach for managing product complexity and reuse across multiple industries. The practice is used to identify reusable code, compose software components, and model product variation. There is, however, a gap in the generation of system traceability when using PLE. Specifically, it is challenging to trace through a feature model from requirements to design elements. Further, traceability itself is both costly and tedious, often completed at the end of development to meet certification standards or for assurance purposes. However, traceability is crucial for supporting change-impact analysis during both the development and maintenance of a product. This thesis proposes a methodology to improve feature identification using [Model-Driven Engineering \(MDE\)](#) techniques, define a hierarchical relationship between features and requirements, and provide a tool-supported approach to generating and maintaining traceability in a PLE context throughout a product life-cycle. The contributions of this thesis address the gap between [Product Line Engineering \(PLE\)](#) and requirement engineering to migrate traceability from a retrospective task to a more active part of development.

Acknowledgments

An expression of thanks to supervisors, industry partners, colleagues, family, or friends.

Contents

Descriptive Note	ii
Lay Abstract	iii
Abstract	iv
Acknowledgments	v
Table of Contents	viii
List of Figures	ix
List of Tables	xii
List of Acronyms	xiii
Declaration of Academic Achievement	xiv
1 Introduction	1
1.1 Motivation	6
1.2 Hypothesis	8
1.3 Organization of Thesis	10
2 Literature Review	12
2.1 Inclusion Criteria	13
2.2 Product Line Engineering	15
2.3 Requirement Engineering and Modelling	17
2.4 Traceability	20
2.5 Discussion	23

3	Methodology	25
3.1	Domain Analysis	31
3.1.1	Goals	33
3.1.2	Use Cases	36
3.2	Features	40
3.3	Goal Refinement and User Stories	45
3.4	Requirement Modelling and Feature Modelling	47
3.5	Feature-Requirement Traceability	55
3.6	Requirement Specification	57
3.7	Requirement Based Feature Identification	58
4	Implementation	60
4.1	Implementation Objectives	61
4.1.1	Goals and Limitations	61
4.1.2	Requirements	64
4.2	Abstract Syntax	66
4.2.1	Feature-Requirement Encapsulation	67
4.2.2	Feature Model Specification	68
4.2.3	Requirement Canvas Specification	70
4.3	Concrete Syntax	75
4.4	Design Decisions	80
4.4.1	Composition Implementations	80
4.5	Summary	81
5	Evaluation	83
5.1	Knowledge Capture & Domain Analysis	85
5.1.1	Goal Diagrams	87
5.1.2	Use Case Diagrams	95
5.2	Product Family Development	105
5.2.1	Composition	107
5.3	Requirement Engineering and Refinement	108
5.4	Tool Supported Development	111
5.5	Iterative and Incremental Change	117
5.6	Discussion	122

6	Future Work	126
7	Conclusion	129
	Appendices	133
A		134
A.1	Full Tool Source Code	134
A.2	Boston Scientific Pacemaker Requirements	134
A.3	What Is a Pacemaker	170
A.4	CyclicL Complete Metamodel	190
A.5	Simple Gherkin-based Requirement Specification Implementation	190

List of Figures

3.1	The four major modelling stages of the CyclicL workflow.	26
3.2	List of steps from Meyer’s Handbook for requirement engineering.	27
3.3	Legend of goal diagram elements.	34
3.4	Example goal diagram outlining the goals of a driver using a vehicle.	35
3.5	A use case diagram outlining what parts of the vehicle a driver will use.	37
3.6	Use Case Diagram (UCD) scoped by the loading cargo goals . .	39
3.7	Initial attempt at a feature model based on use cases identified in UCD	43
3.8	Product variants for the initial vehicle feature model. The model is simplified to only the optional features required for the variant.	44
3.9	CyclicL metamodel. Shows the specification for the feature modelling portion of CyclicL.	50
3.10	CyclicL metamodel. Shows the specification for the first half of the CyclicL requirement canvas.	51
3.11	CyclicL metamodel. Shows the specification for the second half of the requirement canvas portion of CyclicL.	52
3.12	CyclicL metamodel. Shows the specification relationship between the requirement canvas and feature modelling portions of the metamodel. Also shows their relation to the model root. . .	52
3.13	Example of how a feature can be opened up to display encapsulated requirements.	56
3.14	Simplified example of an adaptive cruise control feature model. .	58

4.1	Anticipated goals of users for CyclicL.	62
4.2	Example of a requirement canvas concrete syntax. The example uses a disposable coffee cup with a handle as the diagram’s focus.	66
4.3	Specification of the inheritance relationship between the require- ment canvas and the feature entities in CyclicL.	68
4.4	Definition of the relationship between features and requirements.	69
4.5	Specification for the 4 requirement types in CyclicL.	71
4.6	Specification for design, test, and review in CyclicL.	73
4.7	Specification for the requirement canvas owned by null. This provides a sort of drawing space or backlog for requirements that users have not yet sorted into a feature.	74
4.8	An example of the concrete syntax for the requirement canvas using requirements from the automotive domain.	76
4.9	An example of the concrete syntax for the feature model using features for an adaptive cruise control system from the automo- tive domain.	77
4.10	Gherkin specification for Wheel Heating on/off functional re- quirement. The high-level requirement description is: User shall be able to turn wheel heating on/off.	78
4.11	An example of traceability matrix concrete syntax following the Adaptive Cruise Control features and requirements.	79
5.1	Pacemaker lifecycle as outlined by Boston Scientific requirements.	87
5.2	Doctor goals in the context of the pacemaker system.	91
5.3	Patient goals in the context of the pacemaker system.	92
5.4	Hospital goals in the context of the pacemaker system.	93
5.5	Nurse goals in the context of the pacemaker system.	93
5.6	Patient family goals in the context of the pacemaker system. . .	93
5.7	Technician goals in the context of the pacemaker system.	94
5.8	Combined nurse and doctor goals.	94
5.9	Image of a pacemaker pulse generator on the left. Diagram of a pacemaker device with leads. Extracted from the supplement document ‘What Is A Pacemaker’ found in A.3	96

5.10	Use Case Diagram for the PG device based on the original pacemaker specification.	100
5.11	Intended operation modes for the pacemaker system.	101
5.12	Use Case Diagram for the PG device based on the original pacemaker specification enhanced with information from the goal diagram knowledge capture.	102
5.13	Use Case Diagram for the DCM based on the original pacemaker specification. This diagram combines the hardware and software sub-systems into a single boundary.	103
5.14	Use Case Diagram for the leads based on the original pacemaker specification.	105
5.15	Feature model based on the UCD for the PG_Device.	106
5.16	When right-clicking on a feature, a context menu appears allowing users to create a requirement canvas that is embedded in the selected feature.	111
5.17	Requirements for the ‘Provide programmable pacing’ use case. .	112
5.18	Gherkin specification for the first requirement of the PG_Device.	112
5.19	Gherkin specification for the second requirement of the PG_Device.	113
5.20	Gherkin specification for the third requirement of the PG_Device.	113
5.21	Various stages of approval and testing for the requirements. . .	115
5.22	Showing how traceability is automatically maintained throughout requirement approval and testing.	115
5.23	Feature-requirement traceability during the middle of development for a pacemaker first half.	118
5.24	Feature-requirement traceability during the middle of development for a pacemaker second half.	119
5.25	Feature-feature traceability during the middle of development for a pacemaker.	121
A.1	CyclicL metamodel. Shows the specification for both the requirement canvas and feature modelling portions of CyclicL. . .	190

List of Tables

List of Acronyms

- AQL** Acceleo Querying Language. [69](#)
- DSL** Domain Specific Language. [10](#), [60](#)
- EMF** Eclipse Modeling Framework. [131](#)
- EVL** Eclipse Validation Language. [69](#)
- FDD** Feature-Driven Development. [1](#), [4](#), [6](#), [9](#), [10](#), [29](#), [30](#), [116](#), [130](#), [131](#)
- FODA** Feature-Oriented Domain Analysis. [3](#)
- FORM** Feature-Oriented Reuse Method. [3](#)
- MDE** Model-Driven Engineering. [iv](#), [1](#), [5](#), [6](#), [8–10](#), [12](#), [14](#), [17–19](#), [21](#), [23](#), [25](#), [29](#), [30](#), [59](#), [60](#), [124](#), [127](#), [129](#), [130](#), [132](#)
- PLE** Product Line Engineering. [iv](#), [5–7](#), [9](#), [10](#), [12](#), [14–17](#), [22–24](#), [28–30](#), [43](#), [59–61](#), [63](#), [75](#), [80](#), [124](#), [130–132](#)
- UCD** Use Case Diagram. [ix](#), [8–10](#), [16](#), [23](#), [25](#), [28](#), [38–43](#), [58](#), [61](#), [87](#), [107](#), [116](#), [122](#), [124](#), [126](#), [127](#), [129](#), [130](#)
- UML** Unified Modeling Language. [28](#)

Declaration of Academic Achievement

Chapter 1

Introduction

What came first, the chicken or the egg? It is a question people often debate over dinner or on the bus. In engineering, there is a similar debate: what comes first, the feature or the requirement? Very often, when we do engineering work, we consider what we want to build in parallel to actually building it; exploring the problem space is often overlooked in favour of exploring the solution space. As a result, an engineer may have to reflect on their requirements when they hit a wall during development. They may find that their requirements were incomplete for their feature. Perhaps the scope of their feature is much larger than they anticipated; what was originally considered a single feature could in fact be multiple features. Thus, the engineer will iterate back and forth between feature and requirement development, incrementally changing each until they reach a system state they consider complete, or at least a minimum viable product.

How do we capture this process? [Feature-Driven Development \(FDD\)](#) [1] is a method for agile development in which engineers identify features to be built together to create a system, often leveraging [MDE](#) techniques. They then iden-

tify acceptance criteria, or descriptions of each feature, to specify what needs to be done and how to know when it is complete. These descriptions serve a similar role to requirements for the features, if not written as requirements for the features themselves. Therefore, we can make the proposition that before we have requirements, we have features. Features come first; we should focus on identifying them before we start thinking about the requirements for a given system. This situation will often arise when a new product is being developed or a company is just starting up.

There is an unusual assumption in this scenario; are there no existing requirements before we begin identifying features? From another perspective, in more mature development teams and industries, it is uncommon to start from scratch without any requirements, and even rarer to jump right into identifying solutions without first understanding the problems. Thus, even if limited, there are at least some guiding requirements before engineers begin to identify features of their system that will solve their problems. Therefore, we can say that requirements come before the features. In this environment, we already have a rich history to draw on to inform any new developments.

In reality, both scenarios are likely to happen, perhaps even within the same company. Innovation in industry is constant, and new products are constantly being developed, both from scratch and iteratively on existing products. We may be able to leverage existing assets from one product to help develop an entirely new one. Conversely, we may be branching off into an entirely new domain with no existing documentation or reusable assets. Thus, we have to start from scratch to understand the problems in the new domain. Both scenarios are equally valid!

The conflicting nature of reality generates a couple of problems for develop-

ment. As a result, the relationship between requirements and features becomes ambiguous. In fact, what are the differentiating factors between a feature and a requirement? For this thesis, we will follow the definition of a feature from the original work of Kuang et al. [2, 3] for their Feature-Oriented Domain Analysis [Feature-Oriented Domain Analysis \(FODA\)](#) and Feature-Oriented Reuse Method [Feature-Oriented Reuse Method \(FORM\)](#). According to their original definition, a feature is "a prominent or distinctive user-visible aspect, quality, or characteristic of a software system or systems." Based on this definition, we could consider any portion of a system that a user can interface with a viable candidate for a feature. In fact, for the sake of inclusion, we extend this definition not only to be user-visible but also to serve as an interface, taking into account the multiple senses a user can use to interact with our system. This will expand the scope of a feature beyond visual aspects to include sound, touch, smell, and taste. Depending on the product scope, this definition could be extended to other subsystems that interface with the targeted product.

What is a requirement? This is a well-developed and studied topic. To answer, we look to van Lamsweerde [4], who outlines the purpose of requirements engineering as answering three questions: what, why, and who? What is the problem we are trying to solve? Why is it worth solving? Who should be involved in the solution? There are many more requirements engineering approaches and strategies that we can and will use to develop our methodology. As a baseline, a requirement should answer the problem we are solving, supported by sound reasoning for why it is worth solving and who is involved in solving it.

As such, what are the problems we are trying to solve with this thesis? For one, what is the relationship between features and requirements? This is

a problem because of ambiguity in how requirements and features are specified, because they can (but do not have to) be defined at different levels of abstraction, and because they are defined and manipulated using different development methods, eg, agile, FDD, PLE. Clarifying these ambiguities by concretely defining the relationship between features and requirements may help ensure we elicit and refine requirements that are correct and reasonably complete. At the same time, it can help to ensure that the features we identify are correct and reasonably complete. We define this relationship in the metamodel found in the Methodology chapter [3](#).

Another issue is that iterations between features and requirements do not happen in isolation. There is usually a lot of supporting documentation and software that surround and depend on both the requirements and the identified features. This is usually documented through traceability, most commonly in a traceability matrix. However, maintaining a traceability matrix for all incremental changes between requirements and features is extremely tedious and time-consuming. For each change, it is usually a low return for a high-effort task, until the traceability matrix is so out of sync with both the requirements and features that it is no longer usable. One implication of this is that we will need to create an entirely new traceability matrix, and thus the cycle continues. Therefore, traceability documentation is a significant challenge during iterative and incremental development.

We also examine implications for agile development styles. [FDD](#) is a feature-focused development methodology that already has established implementation rules. We want to explore possible adoption into existing development approaches. The objective here would be to explore methods for improvement rather than redefining [FDD](#) entirely, especially by tying it in with

MDE and PLE efforts. This is true at both the conceptual and tool-support levels. One such tool is FeatureIDE [5, 6], which supports feature composition of software artifacts. Or a tool like GEARS [7], which emphasizes its 3-tiered software product line methodology. These tools are further discussed in the literature review chapter 2

There is also formal methods research for PLE such as the work of Peter Höfner et al. [8, 9], which created a formal algebra for how to compute product lines and feature models. What these tools lack, however, is a self-contained way to manage traceability between features and requirements; FeatureIDE focuses on code composition, while GEARS is not cohesive. GEARS requires hooking into separate requirements engineering tools and lacks a way to handle requirements independently. Thus, we identify another problem: a lack of tool support for traceability and iteration between requirements and features.

Finally, the problem is how we accomplish all of these tasks. What is the process to ensure the features we identify are correct and sufficiently complete? What is the process to make sure that we identify requirements that are correct and sufficiently complete? How can we be sure that our mappings between requirements and features also make sense? This is another problem, as we want to ensure that the mappings between features and requirements are correct and complete for both development and assurance purposes. We want developers to be developing the correct features, and we want to be able to reason about those features so we can build assurance that the properties of our system are true.

It is important to note that while not mandatory, many PLE efforts also use modelling in their approach. There is already significant cross-pollination between the two domains. For our purposes, we want to examine these problems

primarily through the lens of [MDE](#); can we solve them using [MDE](#) techniques and processes? We support [PLE](#) through [MDE](#) techniques and processes.

In summary, these problems lead to the following research questions:

RQ.1 How can we improve the maintenance and generation of traceability between features and requirements?

RQ.2 How can we improve tool support for iterative and incremental development between features and requirements?

RQ.3 How can we combine [MDE](#) techniques for domain analysis, problem space exploration, feature development, and requirement development?

RQ.4 How can we improve the state of implementation and usability of [FDD](#)?

1.1 Motivation

In many industries, we can find companies with product catalogues that they offer to customers. Many industries also focus on developing safety-critical applications. Some areas where these two categories overlap include the automotive and medical device domains. In the automotive industry, companies offer a range of vehicles for customers to choose from. Further, each vehicle model on offer may have variants available. These vehicle model variations can be due to aesthetic differences or functional differences. There can also be variations depending on where the vehicle is sold, such as the driver’s seat location, depending on whether the country drives on the left or right side of the road.

Further, vehicles are inherently dangerous products. According to the Canadian government, 91533 vehicle collisions were reported in 2022, resulting

in 1931 fatalities [10]. It is well documented that automotive manufacturing is a safety-critical industry, and automotive companies are constantly innovating and evolving to improve vehicle safety and reduce accidents, fatalities, and injuries. Thus, we can see that the automotive industry can benefit from [PLE](#) to help manage the product catalogue, support documentation for developing safety cases (or assurance cases in general), and support iterative and incremental development as they release new vehicles year after year.

In medical device development, product variants are common, as companies may be required to fine-tune or adapt a product to different country regulations. Another point of variation can come from the intended use case of a medical device. The same device may be used to treat different diseases or ailments under different conditions, and enabling or disabling this kind of functionality can be critical for regulatory approval. It can also be part of the discovery process, as a new application for existing technology can be discovered. This event occurs frequently in medicine and medical technologies. The ability to plan out new product variants can be extremely beneficial for product planning and development.

Throughout this thesis, we will be using examples from the automotive domain. These examples aim to provide context for the work and aid in evaluating the methodology and tooling. Then, we use an example pacemaker project in the evaluation section of this thesis to examine the proposed methodology alongside an existing requirement document for a medical device.

1.2 Hypothesis

With the research questions defined and the motivation outlined, we can start making some guesses about how to answer them. Beginning with [RQ.1](#), there are several modelling techniques to use. For domain analysis and the identification of system features, we propose using the [UCD](#) from SysML [11]. As a modelling technique, it is informal; however, it is also easy to use for analysis and problem space exploration. The biggest reason to use [UCDs](#) is the need to identify the system’s actors/stakeholders and how they might use it. The idea of a use case is very similar to that of a system feature, allowing for a relatively easy mapping between system use cases and features.

Another [MDE](#) technique we use for domain analysis and problem space exploration is Goal Diagrams as outlined in Van Lamsweerde’s requirements engineering textbook [4]. Lamsweerde provides formal semantics for goal diagrams, which helps ensure we can properly reason about the system’s goals and the users’ goals. However, in spite of the formal semantics Lamsweerde has prescribed to goal diagrams, there are variant syntaxes that exist for goal diagrams that do not strictly adhere to the syntax and thus the semantics that Lamsweerde has defined. This has pros and cons. As a benefit, the reduction in formality can lower the barrier to entry, making it easier to use across professions as a back-of-the-envelope form of expression. The loss of semantics makes it difficult to use formal validation goals and the possible requirements that they can be used to derive. Given that we are more concerned at this stage with capturing knowledge, we are not concerned about the loss of formality. We found the increased flexibility more valuable, as it improves the usability of goal diagrams and their use for exploring the problem space to

elicit requirements.

The reason we chose these two [MDE](#) techniques is part of the answer to [RQ.3](#). For the rest, we turn to the Handbook of Requirements and Business Analysis by Meyer [12]. Meyer outlines several steps towards the requirement elicitation process, which also apply to feature identification. Parts of his book outline the importance of identifying the goals of both the system and the users, as well as user stories and use cases. Further, he supports the use of both formal and informal methods, leaving it to the engineer to decide when to use one over the other based on context. We support that notion, and much of the methodology we outline is inspired by his book. The [UCDs](#) created for analysis will also be used to outline user stories to refine and justify the identified features. The goal diagrams from van Lamsweerde’s requirements engineering are used to support the Goals book from Meyer in either a formal or informal capacity at the engineer’s discretion.

To further explore [RQ.1](#), we propose the following hierarchy: features shall encapsulate requirements. The reason for this proposal is two-fold. For one, in [FDD](#) we often list requirements as scoped by a feature. This presented a history supporting the concept of a feature containing requirements. As [FDD](#) is a common approach to development, we felt this would ease the conceptual learning curve of the proposed hierarchy and its implications. By encapsulating requirements within features, we can achieve feature-scoped requirement traceability, as each requirement is owned by a single feature. We will henceforth refer to this as feature-requirement traceability. And for the second point, there is more existing research and development on [PLE](#) in an [MDE](#) environment than on requirement modelling with an emphasis on traceability in an [MDE](#) context.

The defined relationship between features and requirements also helps with [RQ.2](#). By formalizing the relationship between features and requirements, tool development becomes streamlined, as we can leverage existing Object-Oriented techniques to develop a [Domain Specific Language \(DSL\)](#) that supports the iterative and incremental development of both features and requirements. We can leverage a self-contained tool to partially automate traceability maintenance when changing features or requirements in a feature or requirement model.

Finally, combining all the previous research questions, we arrive at the feasibility question [RQ.4](#). By combining automated traceability with [MDE](#) techniques and tool support for incremental and iterative development, an ideal outcome would be to enhance current [FDD](#) implementations.

In summary, we propose that features encapsulate requirements for supporting traceability. Leveraging this definition, we can build a tool that partially automates feature-requirement traceability. We can use existing [MDE](#) techniques in [UCDs](#) and goal diagrams for domain and problem space analysis, inspired by Bertrand Meyer’s approach to requirements engineering.

1.3 Organization of Thesis

The thesis is structured as follows:

Chapter 1: The introduction [1](#), where we discuss motivations for this thesis, research questions, and the hypothesis.

Chapter 2: The literature review [2](#) for this thesis, where we examine previous work and supporting efforts for traceability in [PLE](#).

Chapter 3: The methodology [3](#) proposed for this thesis.

Chapter 4: The implementation [4](#) of the proposed methodology, which introduces the tool, CyclicL, to support modelling, specification, and traceability efforts.

Chapter 5: The evaluation [5](#) of the proposed methodology. This uses a published requirement specification for a pacemaker. It analyzes the value of the proposed methodology in the medical device domain.

Chapter 6: The future work [6](#) identified for this methodology.

Chapter 7: The conclusion [7](#) for this thesis.

Chapter 2

Literature Review

There are four main research topics that support the work in this thesis: [MDE](#), [PLE](#), traceability, and requirement engineering. Individually, each of these topics has many branches of research, both academic and industrial. To narrow the selection of work to support this thesis, we sought papers that explored at least two of the four topics. As bonus topics, we also considered papers that involved a supporting tool or an exploration of adoption in industry.

We also include some definitions in this chapter to establish nomenclature that will be used within the rest of this thesis. For [PLE](#), we consider a product line, a product family, and a feature model to be synonymous ways of capturing product information by connecting features. Similarly, we consider model-based system engineering and model-driven engineering as synonymous practices.

Traceability is scoped to requirement traceability and feature traceability, though not necessarily to each other. Much of the literature focuses on traceability itself, while a smaller body of work examines it across specific phases of development. As such, we seek prior work that explores a range of trace-

ability methodologies and approaches. With these guiding topics, we use the following strings to generate our search results:

- (“model driven engineering” OR “model based engineering”) AND (“product line engineering” OR “feature modelling”) AND “tools”
- (“model driven engineering” OR “model based engineering”) AND “traceability” AND “tools”
- “traceability” AND (“product line engineering” OR “feature modelling”) AND “tools”
- “traceability” AND (“requirement modelling” OR “requirement diagrams”) AND “tools”

Results using these search strings were chosen from the initial search results from Google Scholar and the following conferences:

- International Conference on Software Engineering (ICSE).
- International Conference on Model Driven Engineering Languages and Systems (MODELS).
- Software Product Line Conference (SPLC).
- Variability Modelling of Software-Intensive Systems (VaMoS).

2.1 Inclusion Criteria

In order to deem a resulting publication to be relevant to this body of work, the following criteria needed to be met:

- The publication should use [MDE](#) techniques AND address [PLE](#) OR requirement engineering. The publication can focus on development in any applied domain, though automotive and medical device development are of particular interest.
 - Of particular interest are applications of feature modelling in industry.
 - Of particular interest are applications of [MDE](#) techniques for requirement engineering and modelling.
- The publication should explore traceability. This can be done either for requirements OR [PLE](#), though ideally the publication should explicitly explore traceability between [PLE](#) and requirements. This also includes methodologies for automated traceability.
- The publication should have tool support for traceability, [PLE](#), or/and requirement modelling.

We parse the results and separate them into the three domains: [PLE](#), traceability, and requirement engineering. Though we set up [MDE](#) as one of the topics for this thesis, it is more of a generalized one as we explore [PLE](#), traceability, and requirement engineering through the lens of [MDE](#). [MDE](#) is also used to provide tool support for feature modelling, traceability, and requirement engineering.

Another reason for separating the results in this manner is to identify synonymous definitions and specifications across domains. Looking at the topics through the lens of [MDE](#) allows us to identify patterns of similarity across publications.

As part of this work, we explore two domains: automotive and medical devices. We aim to demonstrate that the proposed methodology and implementation can be effective in both domains, given their safety-critical nature. As such, we also include some literature on development in the automotive industry and medical devices, specifically pacemakers. For the supporting literature, however, we were broader and looked across multiple industries to understand the state of practice for [PLE](#), traceability, and requirement engineering.

2.2 Product Line Engineering

[PLE](#) has become a popular approach to handling diversity and complexity in a company’s product portfolio since its original conception by Kang et al. [2, 13]. We scoped the related work to fundamental aspects of [PLE](#) and concrete implementations (tool support). As one of the objectives of this thesis is to add a few extensions to the existing theories of product families, we paid special attention to formal specifications and existing tools.

Many tools exist for feature modelling. Two examples of tools come in FeatureIDE and GEARS. FeatureIDE [5, 6], developed originally by Kästner and Thüm et al, is a feature modelling plugin tool for Eclipse that lets a user define their product software as a feature model, write code within the model, and seamlessly integrate the features together to form an executable product. It has more recently become the research community standard as a feature modelling tool. Another example for feature modelling and PLE is GEARS [7] by Big Lever, which emphasizes its 3-tiered software product line methodology. The 3-tiered approach outlines three ways to use GEARS:

handling variation and automation within a single product line; identifying reusable assets across product lines; and product evolution over time.

As a formal approach to [PLE](#), feature modelling can also be defined through algebraic methods as shown in the work of Höfner et al. [8, 9] which uses set theory as the basis for proving how their algebra works. This algebra can be extremely useful for tool development that implements feature modelling, as it enables the computation of feature models, thereby facilitating easier implementations of feature modelling and product line engineering tools.

Another direction that feature modelling has taken is evident in the work of Czarnecki et al. [14]. While still involving formalisms for developing feature models, there is a greater focus on graphical modelling rather than mathematical specification and computability. They expand on the original syntax defined in FODA and show, by using context-free grammar (in this case, meta-modelling), how to add cardinality to feature models and their relevance to feature modelling paradigms. This specification’s main benefit is handling feature duplication in contrast to the work of Höfner et al.

The similarity between feature modelling and [UCDs](#) has been previously explored in the work of Mefteh et al. [15]. They implemented a rigorous approach to transforming use cases from a [UCD](#) to a feature model and implemented it in a tool. The methodology proposed in this thesis instead relies on engineering judgement to extract the features from the [UCD](#). In the future work of this thesis, we do however want to explore more rigorous tool support for extracting features from use cases similar to what Mefteh et al. have done. Instead, the contribution of this thesis is the integration of the equivalency between use cases and features into the broader scope of feature modelling, requirement engineering, and automated traceability into a unified workflow.

We know that there is a desire for better [PLE](#) practices for industry. In a survey paper by Horcas et al. [16], they conclude that a lack of tool support for the various approaches to [PLE](#) is a roadblock to more widespread adoption in industry. This is further supported by the workshop conducted by Becker et al. [17] where they identified 7 themes to the challenges of [PLE](#) across various industries: Modularity, Product Line Scoping, Variability Management, General Process, Verification and Validation, Evolution and Maintenance, and Key Performance Indicators. There is some ongoing work to address some of these issues. Bilic et al. [18] present an [MDE](#) toolchain and methodology for managing variability in system design. Co-evolution is another problem that Jongeling et al. [19] address in Simulink models within a product line. For the automotive industry specifically, we can see some initial benefits from an implementation by Wozniak et al. [20] that uses GEARS. Their act of capturing their product line in a consistent, model-based environment helped manage complexity. The ability to automate configuration calibrations was a great benefit for their purposes as well. As these were just their initial efforts, they also called for further research and work in this domain with respect to industry practices. These publications highlight that a gap exists between academic and industrial practices due to the difficulties of widespread adoption in an industrial setting.

2.3 Requirement Engineering and Modelling

We consider requirement engineering the act of eliciting, refining, and specifying requirements for a given item, whether a product, feature, or component. However, we identified a gap in the literature regarding [MDE](#) techniques and

requirement engineering, specifically in requirement modelling. This will be a significant contribution of this thesis as we explore this gap and identify how [MDE](#) techniques can enhance existing requirement engineering approaches.

We also explore informal and formal approaches to requirement engineering. They both have merits in different situations, with informal methods being more approachable, while the formal methods inspire more confidence due to their natural rigour. We also found various definitions of functional and non-functional requirements that were helpful in informing our design decisions for this thesis.

The idea of using feature models to encapsulate requirements was partially inspired by Meyer’s book on requirement engineering [12]. His style emphasizes extensive stakeholder analysis to determine their use cases and to scope system requirements around user interactions. It emphasizes a more informal approach to requirement engineering, lowering the bar for entry and making it more approachable, while recommending the occasional “formal picnic” as he refers to it as needed. The purpose of the “formal picnic” is to supplement requirement decisions and specifications with more rigorous methodologies as needed, rather than mandating formal methods be adhered to throughout the requirement engineering process.

We also have Lamsweerde’s style of requirements [4]. There is a lot of overlap between Meyer’s and Van Lamsweerde’s requirements; however, there are some key differences in the ease of implementing the requirements processes. Van Lamsweerde’s requirement style dives deeper into the formalism and MDE approaches to requirement engineering, in contrast to Meyer’s requirement style. Lamsweerde used formal goal diagrams to capture knowledge about a product’s stakeholders and to justify emerging requirements derived

from the goals. Another difference is in the definition of a goal. Van Lamsweerde defines a goal as a “prescriptive statement of intent that the system should satisfy through the cooperation of its agents.” In contrast, a goal is defined by Meyer as the “needs of the target organization, which the system will address”. In general, Van Lamsweerde’s style requirements are a more in-depth version of Meyer’s requirements, with a more formal approach to requirement engineering, at the cost of Meyer’s approachability.

Due to the gap we discovered in requirement modelling, we focused many of our efforts on other requirement engineering resources that did not include [MDE](#) as a main component of their approach. Robertson [21, 22] heavily emphasizes stakeholder identification and involvement early on in the project. There is more emphasis on business use cases and functional vs. non-functional requirements, in contrast to Meyer’s requirements, which focus more on stakeholder use cases. Meyer’s requirements style is also structured into four books, each representing a different portion of the requirements engineering process, with a suggested structure for organizing its respective section. By contrast, Robertson’s requirement engineering approach focuses more on how requirements are written and their specific categorizations. The requirement engineering process of Sawyer and Sommerville [23, 24] discusses the different viewpoints within system requirements. Their work stresses that not all stakeholders are people; they can be anything from an organization to another system to be integrated with or interacted with. There are many more that could be talked about; however, we can generalize a pattern:

- Identify who/what you are developing for.
- Identify what/how the stakeholder interacts with a target system.

- Specify the behaviour around that interaction.

While requirement diagrams from SysML [11] are a helpful approach for requirements engineering and modelling, other modelling schemes have also been explored in this area. Use case diagrams from UML [25] have been widely explored for their value in requirements engineering. Siau and Lee ultimately concluded that use case diagrams helped communicate requirements compared to other modelling approaches [26], but had limited usability beyond communication. They have also been used by von der Maßen and Lichter [27] for software product line development, having to customize the metamodel to make it work. Wegmann and Genilloud [28] formalized use cases and use case diagrams for functional requirements in systems, though they faced difficulty scaling the model to handle more complex systems in detail and demonstrated limitations of use case diagrams. FORML, a Feature Oriented Requirement Modelling Language by Atlee and colleagues [29, 30], is another approach to feature-scoped requirements based on product families. They use superimposed state machines to express the behavioural requirements of the features that compose a product line.

2.4 Traceability

Traceability is a commonly studied topic in academia and industry. We found that traceability was commonly considered a retrospective task of product development due to the difficulty and cost of maintaining it throughout development. We found that this is due to a common philosophy that separates traceability as a task from the development process, rather than including it as a natural outcome of development. This has led to a variety of post-release

analysis techniques to build traceability artifacts.

This thesis aims to embed traceability as a natural part of development rather than a retrospective task, and we explored various automated and manual approaches to traceability. We kept the scope of traceability artifacts broad to help explore traceability across development phases beyond the feature and requirement phases.

The concept behind this thesis shares some similarities with the work of Marques et al. [31]. While both aim to automate traceability matrix generation and maintenance by using requirements diagrams, there are distinct differences in the implementations and end goals. We focused our efforts on distinct MDE environments (requirement modelling, feature modelling, use case diagrams, and goal diagrams). Meanwhile, the approach from Marques et al. aims to generate traceability between requirements, design, and testing.

Dronology, the work of Cleland-Huang and colleagues, is a project that has produced comprehensive work around and including traceability. As part of their project, they explore traceability through the lens of safety assurance [32, 33, 34, 35].

The work of Heisig et al [36] proposes a generic traceability metamodel for end-to-end traceability in software product lines. Their proposed method is designed to facilitate and enable comprehensive traceability throughout the development process from requirements to implementation across multiple MDE tools. As such, they developed a traceability metamodel to handle a variety of artifacts they anticipate will require traceability links. In contrast, our work does not focus on developing a traceability metamodel. Instead, we focus on mapping requirements to features and generating traceability from the mappings. Another example of traceability in product lines is in the work of

Tsuchiya et al. [37]. Their work focuses on recovering lost traceability links due to incremental development. They propose a framework and implement a tool that shows potential in recovering lost traceability links due to natural software maintenance over time. This is relevant because it is another approach to maintaining traceability through incremental software changes. While they focus on recovering lost links, we attempt to update the links in parallel to changes.

Similarly, the work of Kelleher [38] also required the definition of a traceability metamodel, though it was developed to handle complexity in software systems rather than product lines. Another approach can be found in the work of Maletic et al. [39], whose primary focus was the flexibility and interoperability of traceability artifacts, enabling the direct modelling of artifacts through model transformations using XML. Asuncion et al. [40] used machine learning to generate traceability prospectively and retrospectively, during development and after development. It ultimately opened the possibility of continued research but did not draw any final conclusions on the effectiveness of using ML to generate prospective traceability.

There has also been previous work on the development of tools to support traceability in feature modelling and PLE. The work of Anquetil et al. [41] they build a tool based on a generic traceability metamodel that can be applied across multiple different design artifacts. One key difference between their work and the work presented in this thesis is that they assume several key steps have already been completed: requirements engineering, variability management, UML design, and implementation. In contrast, we are specifically targeting requirements engineering and variability management as the key artifacts that need to be developed. UML design and implementation as out of scope for this

thesis. Further, Anquetil et al. apply traceability onto those artifacts after they have been created, while we aim to generate traceability directly as part of the development process.

The work of Sousa et al. [42] is also similar to the work of Anquetil et al. as they also create a generic traceability metamodel to be applied to a broad spectrum of engineering artifacts to discover and document traceability links. The key difference between these previous works and our own is that we do not have a generic metamodel to be applied after engineering artifacts have been created. Instead, we want traceability to be a fundamental part of creating the artifacts in the first place. This is a major contribution of this thesis as the majority of traceability effort takes place after the engineering artifacts have already been created, instead of integrated with the initial development. This will be further expanded upon in chapters 3 and 4

2.5 Discussion

Having reviewed a variety of publications on MDE, PLE, requirement engineering, and traceability, we have identified several patterns. While there are some MDE techniques that support both formal and informal requirement engineering, there is a lack of literature on modelling requirements themselves. Specifically, the use of SysML requirement diagrams is an underexplored theme compared to more popular modelling strategies, such as UCDs and, to a lesser extent, goal diagrams.

There are many different strategies for handling traceability, from machine learning to manual processes. We also found prior work on generating traceability for a software product line. We found that we share a similar philosophy

to Asuncion et al., who argue that traceability should be a prospective task rather than a retrospective one. While we share that philosophy, we approach it very differently, as we do not use machine learning in our traceability generation. We consider traceability should be a natural outcome of good engineering, emerging naturally as various phases of development are completed.

We generally agree with Meyer’s approach to requirement engineering, which supports a more informal and flexible approach to requirements, with some detours into more rigorous methods to supplement and support requirement specification as needed. This is the main reason we follow a development path similar to the one he presents in his handbook.

Finally, [PLE](#) and the industry require more support for the state of the practice. We have identified a desire for better [PLE](#) across multiple industries, along with the many difficulties they face in implementing it. Although we do not aim to solve all of them in this thesis, we do aim to lower the bar for entry into implementing [PLE](#) techniques in industry and to create tools to support these efforts.

The work presented in this thesis expands upon our previous work [[43](#), [44](#)] where we first introduce the concept of the feature-requirement hierarchy for traceability and the tool to implement automated traceability during the development of feature models and requirements. This has come together to form CyclicL, the methodology and its tool we will be presenting in this thesis.

Chapter 3

Methodology

In this chapter, we explain how we combine Model-Driven Engineering (MDE) and Product Line Engineering (PLE) to identify features and requirements. We then establish a hierarchy of features and requirements to facilitate a well-defined Feature-Driven Development (FDD) practice. Parts of this methodology were inspired by significant overlap between these seemingly very different domains of engineering practice. In the world of [MDE](#), [UCDs](#) are often used to capture high-level knowledge for how stakeholders will interact with certain parts of a system. While the modelling approach is informal in its semantics and syntax [\[25, 4\]](#) due to inconsistencies in how [UCDs](#) are implemented and interpreted, it has been claimed to support both technical and non-technical users alike in describing how they want stakeholders and customers to interact with their products or system.

At a high-level, the CyclicL methodology has 4 major modelling stages, shown in figure [3.1](#). During each modelling stage, we want to highlight again that our definition of incrementation is refinement work that occurs within the same modelling stage, represented by the arrows that loop to the same bubble.

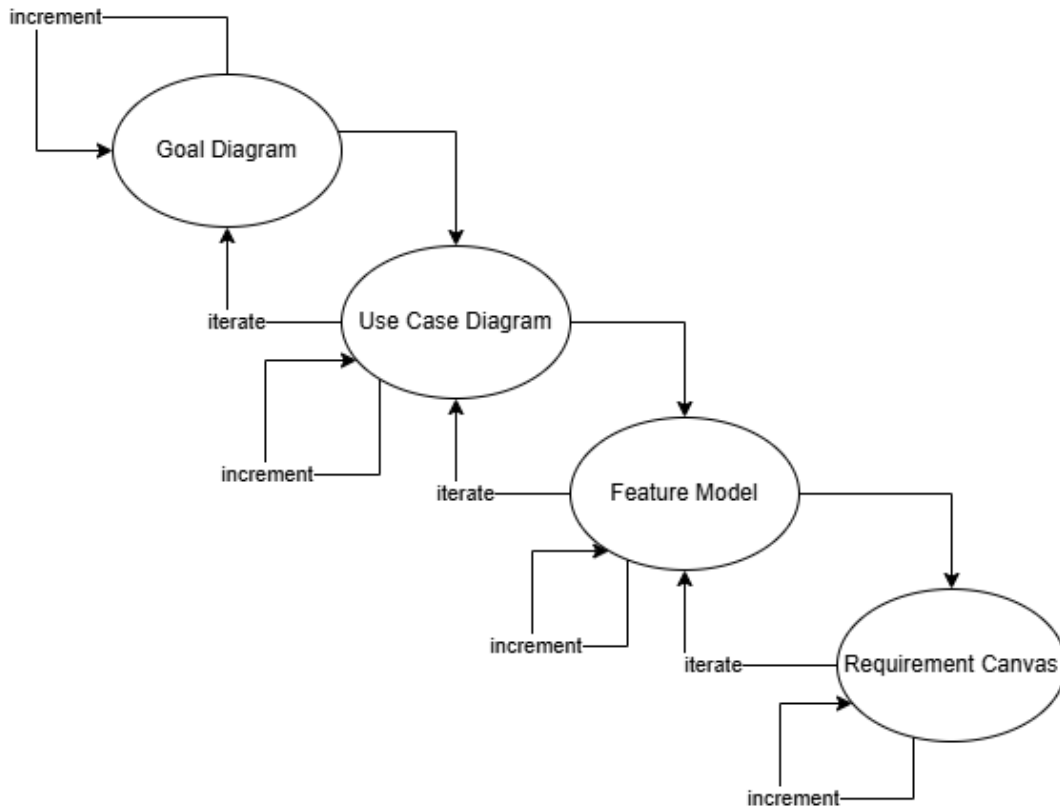


Figure 3.1: The four major modelling stages of the CyclicL workflow.

Likewise, iteration represents refinement work that is informed from feedback of a subsequent modelling stage, represented by the errors that go backwards up the modelling chain.

The overall methodology for this thesis is inspired by the Goals and System sections from Meyer’s requirement engineering book [12]. The Goals section, summarized in Figure 3.2 along with the System section, is primarily focused on capturing conceptual models of the stakeholders of the desired product and the purpose of building it. The System section is essentially a refinement of the concepts captured in the Goals section to actionable requirements ready for design and development. It dives deeper into the details of stakeholder interaction, focusing on how to use the product, while the Goals section ad-

Goals (G) G.1 Context and overall objective G.2 Current situation G.3 Expected benefits G.4 Functionality overview G.5 High-level usage scenarios G.6 Limitations and exclusions G.7 Stakeholders and requirements sources
System (S) S.1 Components S.2 Functionality S.3 Interfaces S.4 Detailed usage scenarios S.5 Prioritization S.6 Verification and acceptance criteria

Figure 3.2: List of steps from Meyer’s Handbook for requirement engineering.

addresses questions focused on why the product should exist and what it will do for stakeholders. We do not fully implement everything outlined in those sections; instead, we have adapted them to suit our method. The more detailed steps of our proposed methodology are as follows:

1. Identify stakeholders, users, and customers. Who will be impacted by the product?
2. Identify stakeholder, user, and customer goals. Why would they need the product we are developing? Why would they not need the product in question?
3. Identify stakeholder, user, and customer use cases for a designated system. Each use case should work to satisfy at least one goal. What will the product do to help the users and stakeholders?
4. Identify system features based on use cases. These are the high-level

features for our system. This is where the overlap between features and use cases can be leveraged to equate use cases from the [Unified Modeling Language \(UML\)](#) to features in [PLE](#).

5. Refine use cases to user stories. These become our feature requirements.
6. Support stories with identified stakeholder goals. We want to validate our requirements against the previously defined goals and use cases.
7. Build high-level features into a feature model. The feature model is where we begin to capture the product family.
8. Decompose features as needed in the feature model. There are iteration possibilities between the [UCD](#) and feature model here. At this point, we can begin identifying variant requirements and features. We anticipate that this step will likely lead to increased insight into the product, potentially driving decision changes that create opportunities for iteration between the [UCD](#) and feature modelling.
9. Use features to encapsulate requirements.
10. Model requirements from user stories in the requirement canvas.
11. Use behavioural specification for requirements within the modelling environment.
12. Generate bi-directional traceability between features and requirements.

In the world of [PLE](#), a critical process is the identification of relevant system features. In the original work introducing the concept of Feature-Oriented Domain Analysis (FODA), Huang et al. defined a feature as “A prominent or

distinctive user-visible aspect, quality, or characteristic of a software system or systems” [2]. While this definition has evolved beyond its original scope, the spirit of what a feature is has remained largely the same: an identifiable aspect or artifact of a system that is interacted with. For software, this can range from a single variable assignment to entire modules and components. In a hardware system, a feature could be the different materials and components used to build a base model versus a top-of-the-line trim, ranging from the smallest screws to completely different chassis materials.

Finally, in [FDD](#), the concept of a feature is also widely used for organization and task setting [1]. Features in this domain can often be client-focused tasks or actions that the software can perform. The features are used to scope the tasks required to complete product development.

Thus, the overlap becomes apparent: use cases from [MDE](#), features from [PLE](#), and features from [FDD](#) all exhibit some overlap. They all emphasize user or stakeholder interactions. They all attempt to capture domain knowledge of the interactions. Therefore, for this methodology, we use these overlaps to develop strategies for feature/use case identification, requirement elicitation, and traceability. We also establish a pseudo-equivalence between features and use cases between [PLE](#) and [MDE](#) respectively.

To demonstrate the process, we focus on the automotive domain, as it is both safety-critical and complex, with many product variants. It is a domain that combines both software and hardware. There are interactions at both the stakeholder level and within the vehicles. The combination of these factors means there is plenty of opportunity for requirement variants to emerge. This presents many challenges with project planning for vehicle development, designing hardware and software for a variety of vehicles, and maintaining docu-

mentation throughout the product lifecycle. The objective of this methodology is to capture variability earlier in development, enabling actionable planning. By capturing requirements variability, engineers can more easily anticipate and plan for building modular systems from the start of a project.

Some examples of variability in the automotive domain include adapting an engine to work across various drivetrain configurations (4x4, AWD, FWD, RWD) and an HVAC system working in 1-row, 2-row, or 3-row vehicles. Further, it is a domain that already implements various versions of [MDE](#), [PLE](#), and [FDD](#). The examples shown are not complete and focus on explaining the development process.

Another contribution of this methodology is in the level of abstraction for development. FeatureIDE [6] is mainly used for reusing software implementations. GEAR [7] is spread out across adaptable ways for a company to be introduced to [PLE](#) for design variants. However, for industries that continue to grow in complexity, capturing variants even earlier in the development process, such as requirements, becomes increasingly necessary. We aim to introduce [PLE](#) techniques earlier in the development process to capture complexity and variations at the requirement level, thereby reducing uncertainty later in the process.

In summary, once the stakeholders are identified, we want to capture what their goals are when interacting with our system. Those goals are used to guide and justify use cases. They are also used to reason about and refine requirements. By equating features and use cases, we take the use cases to begin feature modelling efforts and feature decomposition. Finally, we specify the requirements for each feature. Naturally, there is opportunity for incremental development within each stage and iterative development between them.

We anticipate that most incremental and iterative development will occur between feature decomposition and use cases, between requirement specification and user stories, and between feature decomposition and requirement specification. This is due to previous experience working with industry partners to explore how to handle product variance and decomposition. We observed significant iteration at this level as project planning changed due to shifting environmental factors (regulatory changes, economic incentives, etc.) and as engineers learned more about the product they were developing and refined their requirements and designs over time.

3.1 Domain Analysis

Before any engineering work can take place, we must answer the question of who we are building this system for. Without knowing who we are building for, it is impossible to properly identify the system’s features, as we will have no idea who will be interfacing with it. Further, without knowing who we are building for, we have no idea what goals the system will satisfy, and therefore what requirements we want to implement. This is ever more important as we consider the safety, security, and lifecycle implications of who will use our system and who will be affected by it. In the automotive domain, at least two of our stakeholders would be the driver and a pedestrian. Driver, as a category, is still quite broad; drivers come in all sorts of shapes and sizes. Would a young 20-year-old male interact with a vehicle the same way a 40-year-old female would? What about an 80-year-old, healthy male compared to a 30-year-old, overweight male? Or perhaps a 25-year-old female with dwarfism compared to a 25-year-old female with only 1 hand. In all these examples, would they all

interact with the vehicle the same way?

When we consider how they all may use a vehicle and their use cases, these will eventually be refined into features. The features identified should allow for the widest range of stakeholders to interface with the vehicle. An interesting quirk in the automotive domain is that all driver stakeholders also double as pedestrian stakeholders (buying a car does not mean you never walk anywhere anymore). Therefore, we must consider not only how they will interact with the vehicle, but also how the vehicle will interact with them. Would a blind-spot sensor detect only vehicles or also pedestrians? How big does a pedestrian need to be for the front object detection system to recognize it as a person?

As such, we must make some clarifying assumptions as part of our stakeholder identification. These assumptions are made to help with identification and justification of the goals we elicit. Not every domain may require assumptions. This will vary depending on the project's domain and scope. For automotive, we can make the following simplifying assumptions (as we anticipate they may change as development continues or new information is gathered). These assumptions are done as part of an exercise to simulate a simplified situation to help with explaining the methodology:

- We assume that drivers are at or above the legal driving age in Canada (16 years old).
- We assume that drivers are 80 years old or younger. This is a simplifying assumption to reduce concerns around driver cognition or capabilities that can deteriorate with age.
- We assume that drivers are able-bodied enough to legally operate a motor vehicle.

- Assume height between 151.895cm and 183.24cm. [45] Average range determined between 5th and 95th percentile of male and female population in Canada.
- Assume weight between 48.82kg and 106.60kg. [46] Average range determined between 5th and 95th percentile of male and female population in Canada.

3.1.1 Goals

Once we have identified our stakeholders, we then consider their goals. The goals we identify are closely tied to how we categorize our stakeholders.

The goal of a pedestrian is different than that of a driver, and a customer who purchases a vehicle is a stakeholder who can be a driver or a pedestrian (just because they own a car now does not mean they will never walk!). The stakeholder goals will likely differ significantly depending on their current role. However, the goals between various stakeholders within a category, such as a 20-year-old male or a one-armed 25-year-old female, may be quite similar. As such, identifying the goals of the categories should facilitate the elicitation of stakeholders' requirements for each role.

This is where we propose using goal diagrams to capture this knowledge and information. Goal diagrams are an effective way to capture this knowledge, as they can be formal or informal, depending on the engineer's needs. This flexibility supports the notion of formal picnics explained in Meyer's requirement engineering book [12]. The engineer can start informal if needed and can later formalize the model if required to support further analysis.

According to van Lamsweerde, "a goal is a prescriptive statement of intent

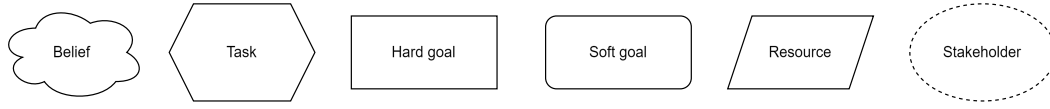


Figure 3.3: Legend of goal diagram elements.

that the system should satisfy through the cooperation of its agents”, where an agent can be a human, a device such as sensors or actuators, existing software, or new software [4]. Based on our interpretations, we equate these definitions of an agent with our system’s stakeholders, so we will continue referring to them as stakeholders. Thus, at the vehicle level, we treat the driver, pedestrian, and vehicle as agents in our system. As we reduce the scope of our system to smaller portions of a vehicle, to automatic braking, cruise control, or lane-keeping assist, we may consider other sub-systems in the vehicle as our stakeholders and consider what goals those agents may have for the new system boundary.

We propose a syntax which loosely follows the syntax from *i** Strategic Rational Diagram [47, 48]. The legend is shown in Figure 3.3. As we are initially using an informal approach, we are not as concerned with how the model elements work together or patterns. What we want to convey at this point is the goals of a stakeholder, what they might be informed by, and what they might do or use to achieve those goals. An example goal diagram at the vehicle level is shown in Figure 3.4. This goal diagram captures, at a high level, the goals a driver might have when using their vehicle.

Generally, we propose that a driver uses their vehicle to travel from one place to another. They are also likely to either carry cargo, people, or both during the commute. They may vary in terms of the amount of cargo or the number of people involved. They will also have to drive the vehicle as part of

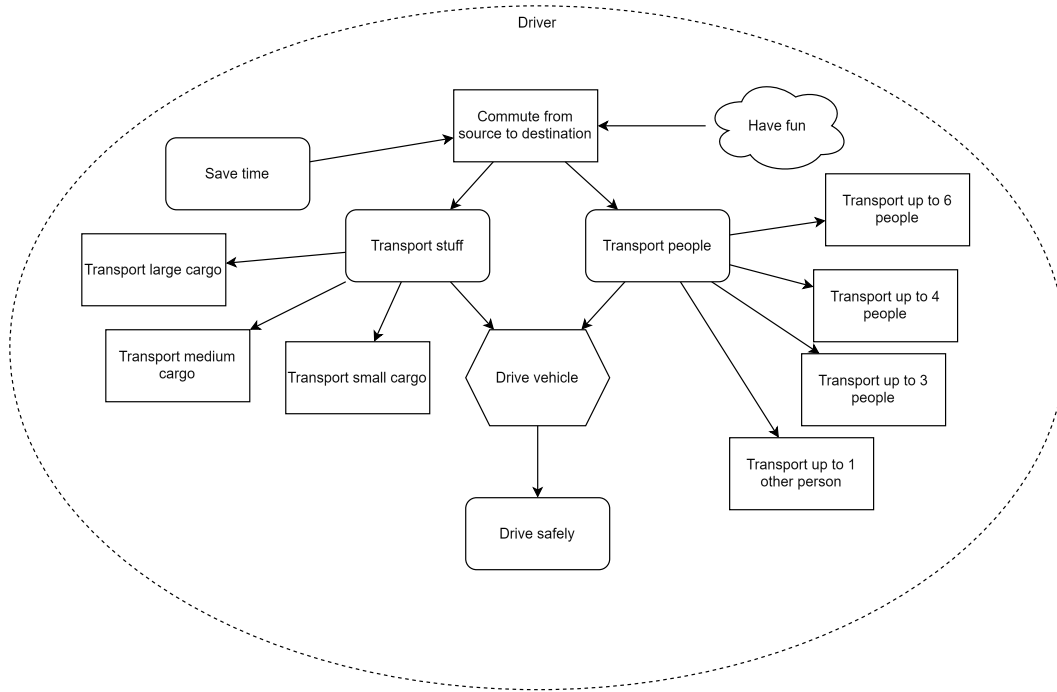


Figure 3.4: Example goal diagram outlining the goals of a driver using a vehicle.

the task, since that is still the main way people use their vehicles. As part of that task, they will typically want to drive safely to ensure they reach their destination without issue. While still relatively simple and without diving very deeply into formally defining the relationship between the model elements, we have been able to convey the goals of a driver, introduce possible variations in those goals, and highlight possible goal decompositions. It is important to note that, as this is an informal modelling approach, another engineer might propose a different goal model for what a driver might do. This is due to the inherent difficulty of repeatability and consistency in informal methodologies. Due to this same informal nature, collaboratively developed goal diagrams can still be merged with relative ease after some discussion among team members.

We will introduce more formalism later with feature and requirement mod-

elling. We believe there is little benefit to introducing that complexity at this stage of development, given the ease of use we can achieve with an informal approach to goal modelling. We found a less rigid modelling approach to be more beneficial at this stage, promoting wider adoption and understanding among both technical and non-technical individuals. More formalism can always be added by engineers or organizations as needed or desired.

3.1.2 Use Cases

The purpose of the goal diagram is to provide the context for why our identified stakeholders would want to use our system, and to provide insights into the connections between our stakeholders and the system. In other words, alongside the stakeholders' goals and our system's goals, we need to identify how the stakeholders will use our system to achieve their goals. This allows us to capture what stakeholders will use the system for, aligned with the goals of both parties. We may find during this stage that we missed some goals, which will provide context for some identified use cases. This is also the first opportunity to iterate on the methodology. As we explore the problem space more thoroughly, we hope to fill in these gaps as much as possible before we get to the feature modelling and requirement modelling.

In Figure 3.5, we show the parts of the cabin that a driver is likely to use. It is easier to justify some use cases than others based on the goal diagram we have already created. For example, the goal of 'have fun' is hard to trace to a single use case and can be ambiguous in terms of traceability and justification. Drive safely, however, can be traced to several use cases, such as brakes, gas pedal, and steering wheel. We can see that our goals of 'Transport stuff'

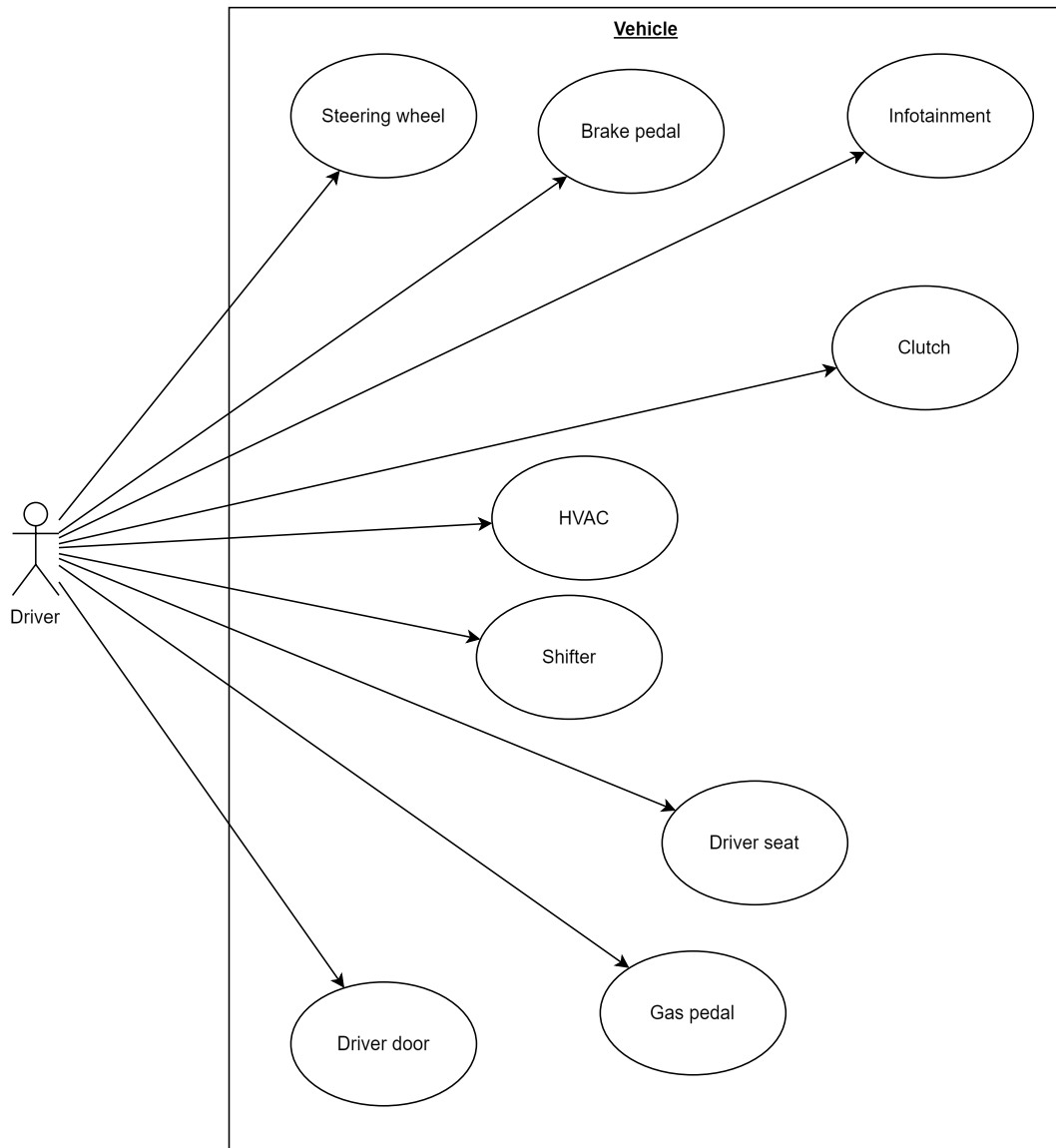


Figure 3.5: A use case diagram outlining what parts of the vehicle a driver will use.

and ‘Transport people’ are also untraceable to the current [UCD](#), as we have not specified any use cases for cargo space or passengers. This highlights the first possibility for misalignment between these two modelling efforts: system scoping. The driver’s goals focus primarily on why they would use a vehicle. As we start identifying use cases, we want to begin answering other questions. What purpose does the product serve? What problems does it solve for the stakeholder? What do they need to do with our product to solve their problem? The [UCD](#) has been written under the assumption that a driver interacts only with the driver cabin. During its inception, it did not consider how other parts of the vehicle might interact outside the driver cabin. These kinds of assumptions are typically easy to make and hard to detect.

As a helpful convention to handle complexity, we consider scoping the system boundary based on the goals from the goal diagram. Thus we can re-title the system boundary from figure [3.5](#) to ‘Vehicle - Drive Vehicle’. This allows a couple of benefits. We can treat each of these [UCD](#) boundaries as a subsystem, and having them directly tied to one or more goals in the goal diagram helps justify the existence of the newly identified subsystem.

This simple convention also helps limit state-space explosion in the [UCD](#) of a complex system, such as a vehicle, and helps scope the [UCD](#) to facilitate system decomposition. We can create another [UCD](#) to specifically target another goal: ‘Transport stuff’. As this effort is not currently supported by a tool, an external method for maintaining traceability is required. Our initial efforts for this convention have been to maintain a syntactic match or mix between the boundary title and the goal(s) that support its justifications.

For the [UCD](#) in figure [3.6](#), we label the boundary ‘Loading Cargo’ as this is the specific portion of the goal that we are focused on; the original goal of

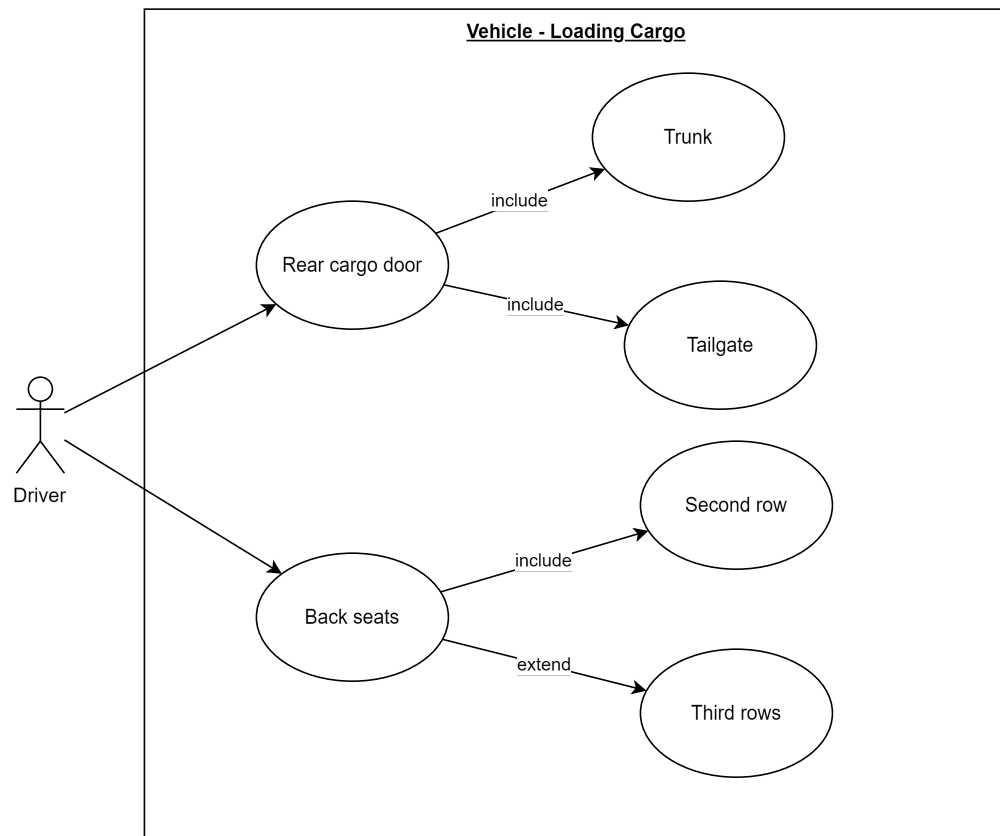


Figure 3.6: **UCD** scoped by the loading cargo goals

‘Transport stuff’ also has the implication of driving built into it. With this, we can separate the UCD by goals to identify which use cases work together to satisfy each goal. This is critical, as with this proposed methodology, the identified use cases will be treated as the features of our system. The requirements for the features can therefore also be traced directly to the goals that scope them.

3.2 Features

After identifying stakeholders, goals, and use cases, we can begin to define the system’s features. Completeness and correctness are not primary focuses during initial efforts, as we anticipate iteration and incremental changes throughout development. As such, we expect to learn something new at any step in the process that informs a conscious decision to return to a prior step to improve a previous engineering artifact.

For this methodology, we want to establish a transformation mapping between use cases and features. Therefore, the UCD serves two purposes: i) it identifies what parts of our system a stakeholder will use to satisfy a goal; ii) what features are likely to exist in our system. However, in feature modelling we can achieve greater granularity in feature decomposition than in a UCD. Usually, this mapping holds true primarily at the higher levels of the feature model. As the feature model is created, further feature decomposition is expected and encouraged more than in initial UCD efforts. As the UCD is not considered a static document, iterations between the feature model and the UCD are expected. This is especially helpful, as there is greater granularity in the relationships between features in a feature model than in use cases in

a **UCD**. Generally, the **include** and **extend** relationships in **UCDs** are meant to express mandatory and optional use cases, respectively, according to the SysML specification [11]. Those are loose definitions, however, as engineers often redefine and experiment with the semantics of those relationships. Using Figure 3.6 as an example, regardless of how the driver uses the rear cargo door, there must be either a trunk or tailgate; it is mandatory that at least one be present. Conversely, for the backseat use case, given the context of loading cargo into the backseat, there must at least be a second row in the vehicle. However, there may be a third-row option in the vehicle that is not mandatory for the use case to be satisfied, but can still be used based on an arbitrary trigger from the actor.

The relationships in feature models are well-defined and understood. Given the nature of the **UCD** specification, much of the information embedded in those relationships needs to be refined during the transformation from use case to feature. The **include** relationship can be used to define mandatory features of a system; however, it does not support enough semantics to differentiate between mandatory features that are mutually exclusive. Similarly, the **extend** relationship is not equipped to handle cardinality in optional features, where there may be a set of optional features, but a non-empty subset must be selected to complete the product. Thus, while we can equate features and use cases, the relationships between use cases and the relationships between features are less straightforward to equate. We define the relationship between use cases and features as follows:

Let $\mathcal{U}$ be the set of all use cases. (3.1)

Let $\mathcal{F}$ be the set of all features. (3.2)

And we assume $\mathcal{U} \subseteq \mathcal{F}$ (3.3)

This definition is one of the first major contributions of this thesis. We use this definition to capture the transformation process from use case to feature. This definition also allows for further feature identification beyond the original scope of the set of use cases used for initialization. This allows some flexibility, as not every new feature identified may need an origin use case. This definition captures the injective relationship between use cases and features without forcing it to be bijective.

For a simple system, it may be that no further decomposition of features is required after equating use cases to features. However, in most system development, we find that feature decomposition is necessary to create a sufficiently complete feature model and to capture all components and configuration possibilities of a system. What the mapping between [UCDs](#) and features enables is the identification of a system's initial features. As more feature decomposition occurs, we encourage iterations between feature models and [UCD](#), but this is not currently enforced. Hence, we can end up with more features in our feature model than in our [UCD](#). We can see in [Figure 3.7](#) what a possible initial feature model can look like based on the previous [UCDs](#).

As an informal approach to drawing a feature model, we use black circles for mandatory features and white circles for optional features. We have defined the clutch as optional, along with the rear door and back seat features. This

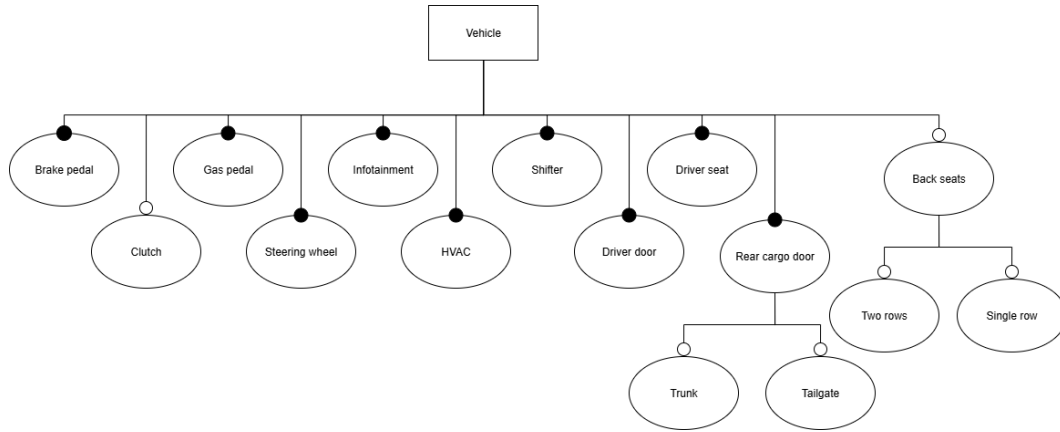


Figure 3.7: Initial attempt at a feature model based on use cases identified in [UCD](#).

is where traditional feature modelling takes over. These are optional features of our vehicle, but no composition relationship has yet been defined among them or with the other optional features of our system. We can now define the conditions for combining the various features to create our system, also known as a product, as we transition to the world of [PLE](#). The various versions of the vehicle we can create, as shown in Figure 3.7, are called product variants. These variants are determined by the optional features available in a given product.

At a glance, one product variant might be a vehicle with no rear seats, a trunk, and a clutch (a typical coupe). Another might be a vehicle with one row of seating in the back, no clutch, and a tailgate (a truck, for example). We can see what these variants might look like in Figure 3.8. It shows just the optional features that become mandatory for the new variants. As previously mentioned, we can define rules to ensure the product variants are semantically and syntactically valid. For example, we could have:

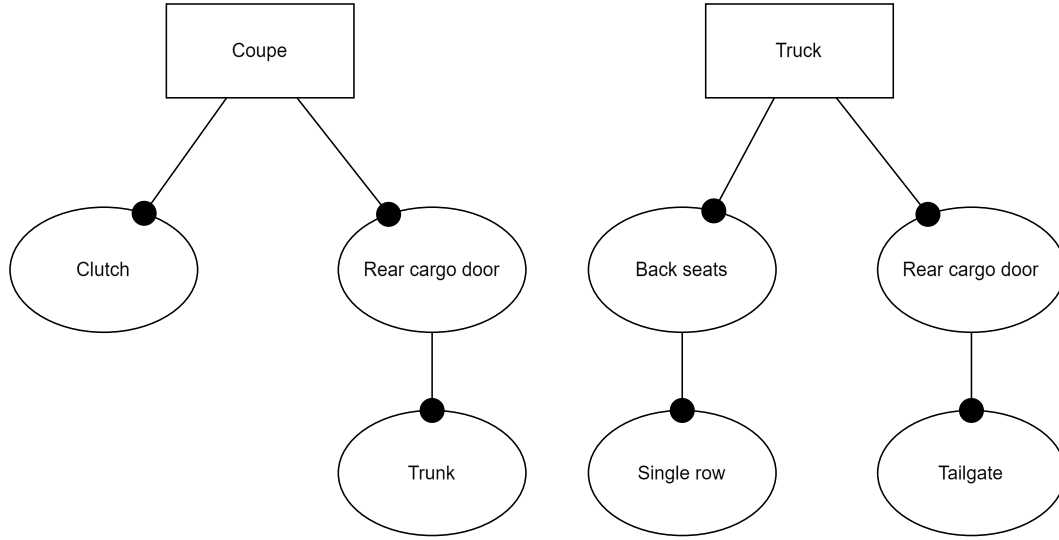


Figure 3.8: Product variants for the initial vehicle feature model. The model is simplified to only the optional features required for the variant.

$$\text{vehicle.backseats} == \text{false} \rightarrow \text{vehicle.clutch} == \text{true} \quad (3.4)$$

$$\text{vehicle.rearcargodoor.tailgate} == \text{true} \rightarrow \text{vehicle.clutch} == \text{false} \quad (3.5)$$

These conditions state that whenever we have no back seats in a vehicle, we must include a clutch pedal, and whenever a tailgate is selected, there will be no clutch pedal. This means all coupe product variants are manual, and all truck product variants are automatic. Naturally, there are requirements for each of these features that we need to capture to support development of these product variants. Since the features map to the use cases, and the use cases are supported by the goals we have outlined, the features are also supported by those goals. Next, these need to be refined to more usable requirement specifications.

3.3 Goal Refinement and User Stories

The goals from the goal diagram are used to justify the use cases and features, as they can be refined into requirements. This refinement process takes shape by determining how the goals are intended to be satisfied and what they are intended to accomplish. Using the goal of ‘Transport stuff’, we rewrite it as a high-level requirement: the vehicle shall enable the transportation of items for the stakeholder. We ask the next question: how will the vehicle enable the transportation of items? If there is a goal decomposition, we can also correlate the answer to this question with the decomposed goals. How will the vehicle enable the transportation of items? The vehicle shall have a large/medium, small cargo space. What is a large/medium/small cargo space? This question is generally up to the engineers and can be decided arbitrarily or justified with research data.

For writing these user stories, we have adapted the use case template from the Meyer requirement handbook [12]. The goal outlines what the user wants to happen. The use case indicates what the user can interact with to achieve a goal. The story connects the dots, explaining how the user can use the system to achieve their goal.

While these help provide requirements from the vehicle perspective, they do not help determine requirements from the stakeholder’s perspective. To this end, we want to decompose our high-level requirements, derived from our goals, into user stories in Meyer’s style. We thus create the following user story around the high-level requirement that the vehicle shall enable the transportation of stuff for the stakeholder:

- **Use Case:** Rear cargo door

- **Actor:** Driver, passenger
- **Trigger:** Stakeholder wants to transport stuff
- **Success Scenario:**
 1. Stakeholder identifies access latch to rear cargo area.
 2. Stakeholder uses access latch to open rear cargo door.
 3. Stakeholder is able to lift stuff into rear cargo area.
 4. Stakeholder fits all stuff into rear cargo area.
 5. Stakeholder is able to close rear cargo door.
- **Secondary Scenarios:**
 1. Stakeholder unable to open rear cargo door.
 2. Stakeholder unable to lift stuff into rear cargo area.
 3. Stakeholder unable to fit stuff into rear cargo area.
 4. Stakeholder unable to close rear cargo door.
 5. Stakeholder gets limb or end appendages stuck in door when closing.

The door does not close, and the stakeholder has suffered harm.
 6. Stakeholder gets limb or end appendages stuck in door when closing.

Door does close, and stakeholder has suffered harm.
- **Success postcondition:** Stakeholder is safely able to load stuff into rear cargo area.

From this user story, we can identify many additional requirements related to the rear cargo door, as well as the goal this story is connected to. The

points from the user success scenario become the functional requirements for the feature. The secondary scenario outlines other requirements for the feature, both functional and non-functional. Finally, the success post condition highlights the satisfaction of the goal if all requirements are met. In summary, from this user story we can identify the following requirements for the rear cargo door and its sub-features:

- Stakeholder shall be able to identify rear cargo door access mechanism.
- Stakeholder shall be able to open the rear cargo door.
- Stakeholder shall be able to load rear cargo space with stuff.
- Stakeholder shall be able to close rear cargo door.
- Door system shall detect an object obstructing the door-closing path.
- Door shall remain open when object is detected obstructing door path.
- Stakeholder shall be able to safely load stuff into rear cargo area.

3.4 Requirement Modelling and Feature Modelling

With goals, use cases, high-level requirements, and some refined requirements, we can begin modelling our requirements. We refer to this environment as our Requirement Canvas. This environment is heavily inspired by the requirement diagram outlined in the SysML specification [11]. We adopted the modelling format and relationship between requirements for the requirement canvas. The requirement canvas diverges from its SysML counterpart in its

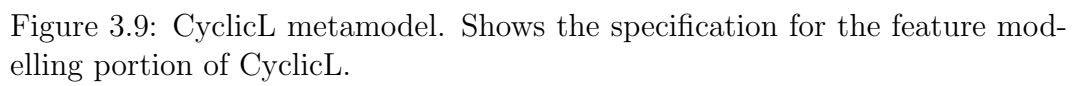
focus on traceability and implementation details. We wanted to allow for different requirements specifications within the requirement canvas element. We also have different attributes owned by our requirement types in the requirement canvas. The requirement canvas has several main objectives:

- Provide an environment for modelling requirements in a way that emphasizes traceability between features and requirements. This is meant to be a bi-directional relationship to enable traceability in both directions. This is also meant to automatically propagate changes in both directions.
- Provide users with the ability to specify requirement behaviour in greater detail with a specification language. The language of choice we use at this time is the Gherkin specification language [49]. This will be further explained in section 3.6.
- Support categorizing requirements. Instead of just having a generic requirement, we want the ability to specify early on whether a requirement is functional or non-functional. We further categorize the non-functional requirements into more specialized categories.

With these in mind, a metamodel was created to specify how to create models within our newly identified requirement canvas. The metamodel for the requirement canvas is shown in Figures 3.10 and 3.11. In this metamodel, we also captured specifications for the feature modelling in Figure 3.9 and the relationship between features and requirements in Figure 3.12. This metamodel has also been used to develop a domain-specific modelling language named *CyclicL*. With *CyclicL*, we aimed to show the feasibility of the proposed

methodology. This includes the overall objectives of supporting iterative and incremental development and partially automated traceability generation and maintenance.

The requirement canvas portion of the metamodel was created after several iterations to identify the elements needed to support the three main objectives of the modelling environment. An important distinction between this specification and the SysML requirement model specification is the requirement types we have defined. We deviate from the SysML requirement model specification to specifically support traceability with features from the feature model to specific types of requirements. Based on our requirement canvas specification, we identified four requirement categories: **Functional**, **Qualitative**, **Constraint**, and **Safety**. Functional requirements follow their traditional definition; things that a product or system must do. Qualitative requirements are those that affect how a product or system accomplishes its function. These include the look, feel, and usability, as well as humanity and some performance requirements that are not categorized as functional. Constraint requirements are those that impose limitations of some type on a product or system. These include operational, environmental, maintainability, support, security, cultural, political, and legal requirements. These requirement definitions can be found in James and Suzanne Robertson’s Volere requirements [22]. Finally, safety requirements are critical because they identify all requirements related to the safety of users or stakeholders involved in the function of a product or system. We highlight safety requirements as a separate requirement category as our target domain for CyclicL is safety-critical development. This also gives us the flexibility to adjust the safety requirement type profile to match safety integrity level ratings based on the target domain.



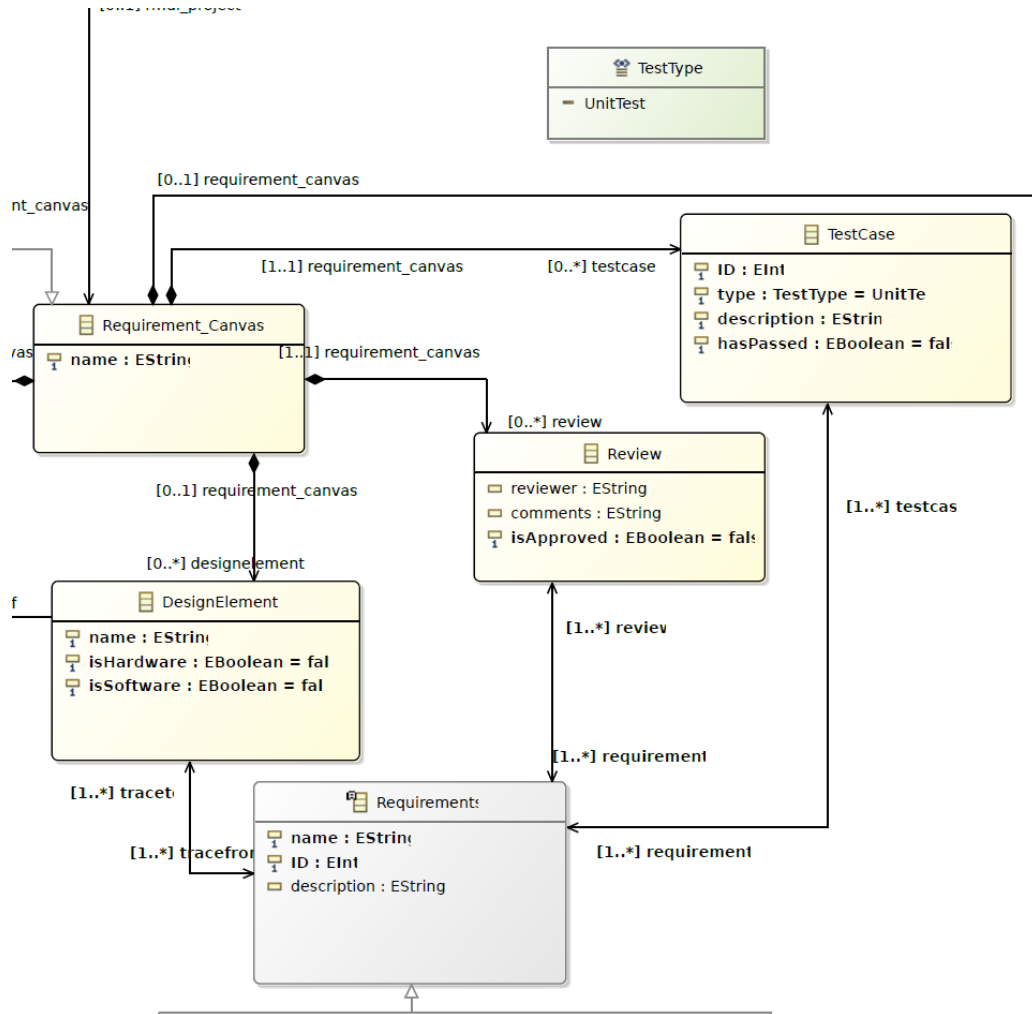
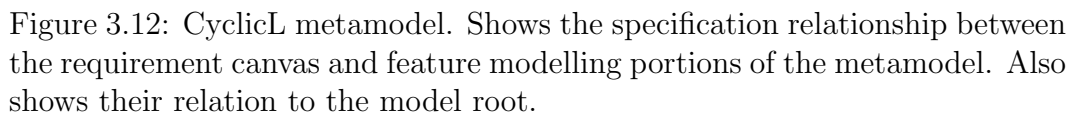


Figure 3.10: CyclicL metamodel. Shows the specification for the first half of the CyclicL requirement canvas.



We also added three more classes besides the requirements: **DesignElement**, **Review**, and **TestCase**, further increasing the divergence from the SysML requirement model specification. The test case and review classes are for verification and validation bookkeeping, respectively. This is also one of the reasons why we include design elements within the requirement canvas. The test cases are meant to determine if a requirement has been verified against its traced design elements, while the review represents whether the requirement has been validated. Currently, verification and validation are expected to occur outside CyclicL because they are out of scope for initial development. We have an enum **TestType** tied to the test cases. This enum will eventually allow users to categorize the tests they may want to apply to their requirements. At this time in development, only **UnitTest** is implemented as a type, but more can be added in future revisions.

The **DesignElement** is perhaps a more controversial addition. There is a risk of polluting the requirements by adding design elements this early in development. The main reason we add them to the metamodel at this stage is, once again, to enable traceability. Design elements in the requirement canvas are meant to be black boxes to mitigate this potential issue. We have limited the class to determining if a design element is hardware, software, or neither (undetermined when building the requirements or an integrated system). Another reason for having design elements within the requirement canvas is to allow a potential path for future development, specifically design generation based on the requirements specified in CyclicL.

For the feature modelling portion of the metamodel shown in Figure 3.9, we tried to remain conceptually in line with previous work [2, 3, 13]. The **FeatureModel** element is the container for the feature model, which is com-

posed of **FeatureEntity** type elements. There is the **Root** element, which will form the actual root of the feature model. This is limited to a single **Root** element per model. The root of the model then decomposes to the **Element** type via the three relational associations: **Alternative**, **Mandatory**, and **Optional**. These three relationships are once again bi-directional to enable traceability in either direction throughout the feature model.

Finally, we have the **ProductVariant** type class, which inherits from **FeatureModel**. This is what allows users to define their product variants at the highest level and have all the same semantic capabilities as the regular feature model while still allowing us to define more specific semantics around the product variants, such as allowing us to have a zero-to-many multiplicity on how many product variants we can have per feature model. This is a design decision we made to facilitate our implementation of feature modelling in this methodology which is not strictly inline with other feature modelling approaches or tools.

This specification is loosely based on the work of Peter Höfner et al. [8, 9], which uses set theory as the basis for proving how their feature modelling algebra works. It is also loosely inspired by FeatureIDE [5, 6], a feature modelling plugin tool for Eclipse. The main reason we recreated the specification for feature modelling within CyclicL is to explore the implications of using features to encapsulate requirements, shown in Figure 3.12. This is a key step in the process, as it enables what we dub feature-requirement traceability. As this proposed methodology suggests that features be used to encapsulate requirements to support feature-driven development efforts, having a way to represent that relationship in a formal specification, such as a metamodel, was impactful for both the development of CyclicL and the development of the

concept.

The complete metamodel can be found in appendix [A.4](#).

3.5 Feature-Requirement Traceability

Feature-requirement traceability is a main contribution of this thesis. As previous steps have identified the system’s features and requirements, this relationship is integral to the process. This relationship is captured by the inheritance connection between the `Requirement_Canvas` and the `FeatureEntity` classes in the metamodel. By allowing every model element in the feature model to double as a requirement canvas, each feature can contain all modelled requirements. Thus, the individual features are encapsulating the various types of requirements, their associated approvals, and their associated design elements.

The decision to implement encapsulation as we did was heavily influenced by the desire to tie together two separate modelling approaches and maintain traceability between the two modelling environments. The relationship we specified allows for several key benefits. First, it provides a mechanism for direct traceability between the requirements and their respective features. Since every requirement will be ‘owned’ by a feature, we can easily trace the relationships between requirements within a feature, between features, and across the entire system. Secondly, this modelling approach minimizes the number of connections needed between the two modelling environments to a single inheritance relationship while still supporting all our traceability objectives. Thirdly, this allows us to model product variance at the requirement level. Specification of product variance can help reduce unexpected changes and capture knowledge of anticipated variance early in product lifecycle de-

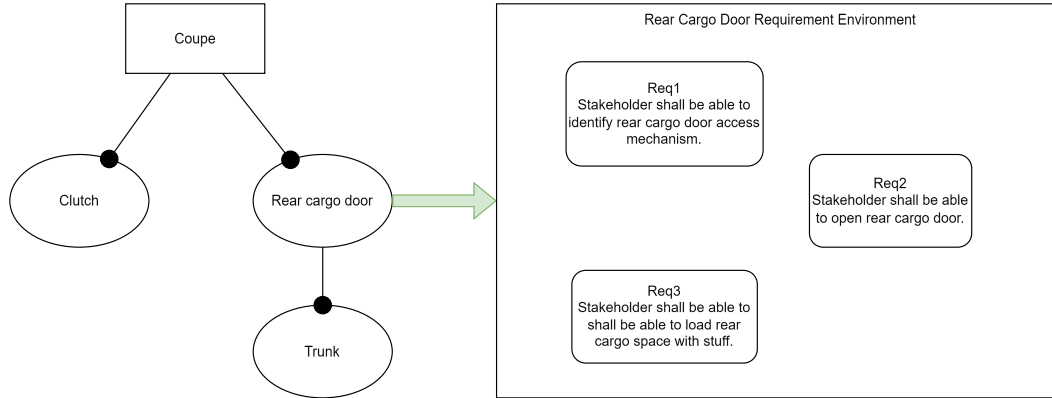


Figure 3.13: Example of how a feature can be opened up to display encapsulated requirements.

velopment. Further, this can support change-impact analysis across product variants and their specifications, facilitating product maintenance.

Some anticipated downsides to this extension include the possibility that it may not be compatible with existing feature-model composition approaches. Since every feature element now doubles as a requirement canvas, this can break conventional feature composition strategies, forcing us to create new ones. Another downside is the need to handle requirements that span multiple features. Due to the encapsulation strategy we are proposing, CyclicL does not support cross-cutting requirements across features at this time. There might be ways users circumvent this, leading to duplicated or inconsistent requirements across several features.

In Figure 3.13, we show the intention of having the features encapsulate the requirements. There is an explicit method for information hiding, and access to the requirements necessitates diving into a feature. This is where we transition from feature modelling to the requirement canvas. Within the canvas, we can then define our requirements and specifications for the owning feature.

3.6 Requirement Specification

Using goal and use case diagrams, we showed how to refine requirements using user stories. In the requirement canvas, we intend to express these requirements as a behavioural specification. This is beyond just refinement, however; as part of the requirement canvas, we also enable requirement specification. Currently we support behavioural specification through an interpreted implementation of Gherkin syntax [49]. The Gherkin style of specification is a structured, natural-language approach to specifying behaviours. It follows a pseudo-predicate-logic system using three main keywords: GIVEN, WHEN, and THEN. The GIVEN keyword is used to express behavioural preconditions. The WHEN keyword is used to express events that trigger some new behaviour. These triggers can be either internal or external to the specified system. Finally, the THEN keyword specifies behavioural postconditions.

The reasons for using Gherkin-style specifications in the requirement canvas are two-fold. We want the specification environment to be both easy to use and read. Gherkin provides a low barrier to entry for specifying requirements and reading them in a semi-formal, natural-language style. This increased flexibility is meant to help both technical and non-technical people specify system requirements. Secondly, while the flexibility is good, we still wanted some constraints for how specifications can be written. Rather than imposing arbitrary constraints ourselves, Gherkin provided a nice, off-the-shelf solution. Despite our current implementation not being completely faithful to the original specification, it is enough for a proof of concept.

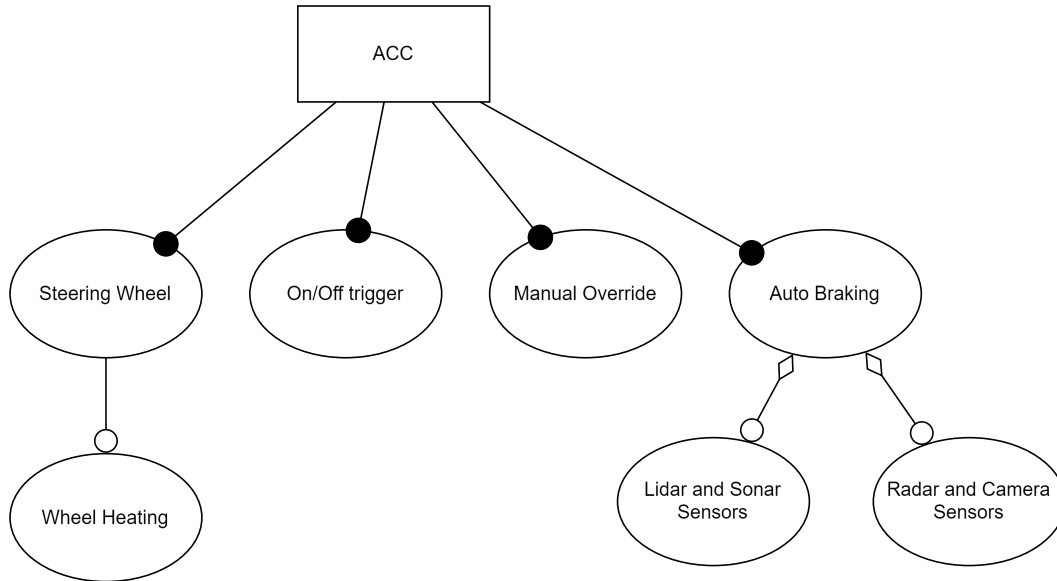


Figure 3.14: Simplified example of an adaptive cruise control feature model.

3.7 Requirement Based Feature Identification

Due to the relationship between features and requirements assumed and defined in this methodology, there are alternative ways to identify features. The variation points in this approach are the requirements defined in the features. As a result, variant requirements can potentially identify new features previously missed by the UCD. Thus far, this methodology has primarily been top-down in its approach to identifying features through domain analysis. However, it is expected that variant requirements will be identified during the elicitation process. This introduces the possibility for a bottom-up approach to identify features. Variant requirements elicited should still be supported by the goal diagram. Naturally, through this bottom-up portion of the methodology, it is encouraged to iterate on previous steps as engineers incrementally follow the approach.

In Figure 3.14, we can see a simplified feature model of an Adaptive Cruise

Control (ACC) system. For the auto-braking feature, two alternative features are available beneath, denoted by an aggregation symbol at the source and an open circle at the target. Sensor-fusion requirements based on possible variant products (such as radar and camera versus lidar and sonar implementations) generate alternative features under the auto-braking feature. Any requirements that apply to both are expected to be abstracted into a feature for auto-braking. This scoping is intended to reduce cross-cutting concerns arising from requirement dependencies.

In summary, the proposed methodology provides a process for combining [MDE](#) and [PLE](#) to improve requirement elicitation, requirement specification, feature identification, and product planning. It also provides a metamodel to specify a language that demonstrates the feasibility of the process and the potential to support automated traceability documentation generation and maintenance throughout a product's lifecycle.

Next, we will present a tool implementation of CyclicL to demonstrate the language's feasibility and the potential tool support for the methodology.

Chapter 4

Implementation

In the previous chapter, we explored a methodology for leveraging common points between [PLE](#) and [MDE](#) for domain analysis, requirement engineering, and feature modelling. We have also introduced a language, CyclicL, to demonstrate how requirement modelling can be combined with feature modelling to improve traceability via a defined hierarchy between features and requirements.

In this chapter, we explore an implementation of CyclicL that demonstrates the tool’s support for the language. Through this implementation, we aim to demonstrate the capability of automatic traceability generation and maintenance from good modelling practice, and the feasibility of iterative and incremental development using the proposed methodology with tool support.

As a proof of concept, we have implemented a [DSL](#) using Sirius [\[50\]](#) and Xtext [\[51\]](#) to show the feasibility of automated traceability between features and requirements. This is a core component of the methodology, as we require the means to specify feature models and requirements to capture their inter-relationships to enable automated traceability. By implementing a [DSL](#), we

can enforce syntactic and semantic rules defined by the metamodel to create verifiable models that support incremental, iterative development.

The tool for CyclicL also provides an environment in which we have explored potential composition strategies. We anticipate that, as a result of designing the language of CyclicL to focus on feature-requirement traceability, there may be some side effects. By implementing the language in a tool, we can explore some of these side effects, such as the impact it may have on composition techniques from [PLE](#). Some modifications may be required as a result of the relationship we have introduced between features and requirements.

4.1 Implementation Objectives

4.1.1 Goals and Limitations

In [Figure 4.1](#) we capture some anticipated goals for our end-users. In conjunction with our own objectives, we use these to support our high-level requirements for CyclicL. At the time of writing, the scope of CyclicL is focused only on the traceability portion of the methodology; we have scoped user goals for inclusion after they complete an independent domain analysis. We assume that users will complete the other steps of this methodology (goal diagrams, [UCD](#)) using other means prior to or as part of an iterative process when using CyclicL. This is a limitation of CyclicL for now, as we do not currently support modelling or traceability from the domain analysis to the identified features and requirements. We strongly encourage users to complete the prior steps of domain analysis before modelling the features and requirements, as it helps the development process build a better conceptual model and understanding

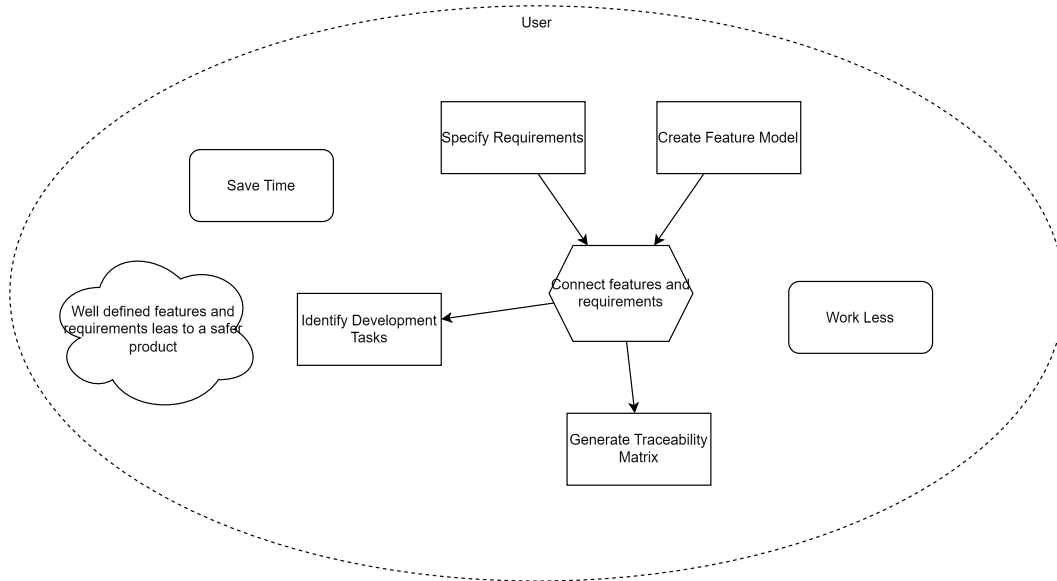


Figure 4.1: Anticipated goals of users for CyclicL.

of the intended product [2].

We have several key goals we aim to achieve by implementing a tool for CyclicL.

- Enforce syntax and semantic rules of CyclicL when creating models.
- Explore impacts of traceability generation and maintenance when creating and changing models within the tool.
- Enforce automated traceability as a side effect of model creation.
- Explore the impact of feature-requirement traceability on both feature modelling and requirement modelling.
- Reuse as many specifications and definitions from previous work as possible.

We expect the main users of the tool to be engineers, but not limited to software engineers. To be successful, the tool needs to allow users to create

feature models to define product families. It also needs to have a way to define requirements. Since CyclicL is also an implementation of the methodology, it must also implement the relationship we have defined between features and requirements. Since we want to support iterative and incremental development, the tool for CyclicL also needs a way to enhance the generation and maintenance of traceability matrices for requirements and features. Finally, the implementation needs to be robust to change, supporting future changes such as traceability to use cases and goals, and automated processes such as change impact analysis and product composition.

Based on our objectives, the tool for CyclicL has two main modelling environments that are distinct and connected, as demonstrated in the abstract syntax in figures 3.9, 3.10, and 3.11. The first is the feature modelling half of the tool. The purpose of this modelling environment is to allow the user to build a product family, describing their features and how they are related. The second modelling environment is the requirement canvas. This portion of the tool enables users to specify their requirements, associate requirements to design elements, and assign reviewers and test cases to requirements. This is where traceability is captured between the feature and various requirements aspects.

Another objective we had during development was reuse. As we explored and implemented traceability in the spirit of PLE, we sought to identify as many reusable aspects as possible from previous specifications and tool implementations. Thus, CyclicL is also inspired by FeatureIDE [5, 6] and GEARS [7] for their abstract and concrete syntaxes. This prevented us from having to reinvent how to build a feature modelling tool so we could focus more efforts on the requirement canvas and the traceability matrices.

4.1.2 Requirements

We extracted high-level requirements from previous work on building a product line and on a coffee cup to guide initial development plans for CyclicL [44]. We also aim to support iterative and incremental development life cycles. We generally define an iterative change as one based on feedback from later stages of development or from stakeholders. These are usually larger changes that may arise while cycling between phases of development. In the context of CyclicL, we want to generate and maintain traceability between features and requirements as users iterate between feature modelling and requirement modelling. Examples of iterative change include a new version of a development artifact or its redesign. Incremental changes are usually smaller in scope and bound within a single development phase, such as feature modelling *or* requirement modelling. Examples of incremental changes include refactoring, updating a definition, or changing a connection between components. While we have been using these definitions to guide our development philosophy, they may not be shared by others. This may present a challenge to the adoption or usage of CyclicL. Therefore, we included ease of use and adoption as another goal. Our initial list of requirements was as follows:

- Tool shall provide the user with an environment for defining requirements.
- Tool shall provide the user with an environment for defining features.
- Tool shall allow the user to specify design elements.
- Tool shall allow the user to relate one or more requirements to one or more design elements.

- The tool shall allow the user to use features to capture and own requirements (ownership in this context refers to a compositional relationship between features and requirements, where features are composed of requirements).
- Tool shall automatically create and maintain a traceability matrix.
- The tool shall facilitate the iterative development of the requirements canvases (*note*: this remains ongoing work).
- The tool shall facilitate the incremental development of the requirements canvases.
- The tool shall aim to promote familiarity and ease of use to increase adoption rates.

Along with these requirements, we also use the prior specifications for encapsulating feature requirements. We treat the requirements encapsulated by a feature as attributes and consider them feature requirements, thus scoped by the feature. As the exploration of requirement modelling started with the requirement diagrams from SysML, we reused as much as possible from the original specification. We then focused more on traceability and integration with feature modelling, diverging from the original specification until we arrived at what we now refer to as our requirement canvas. For a rough idea of what we wanted the requirement canvases to look like, we sketched requirements for a disposable coffee cup with a handle, shown in [Figure 4.2](#).

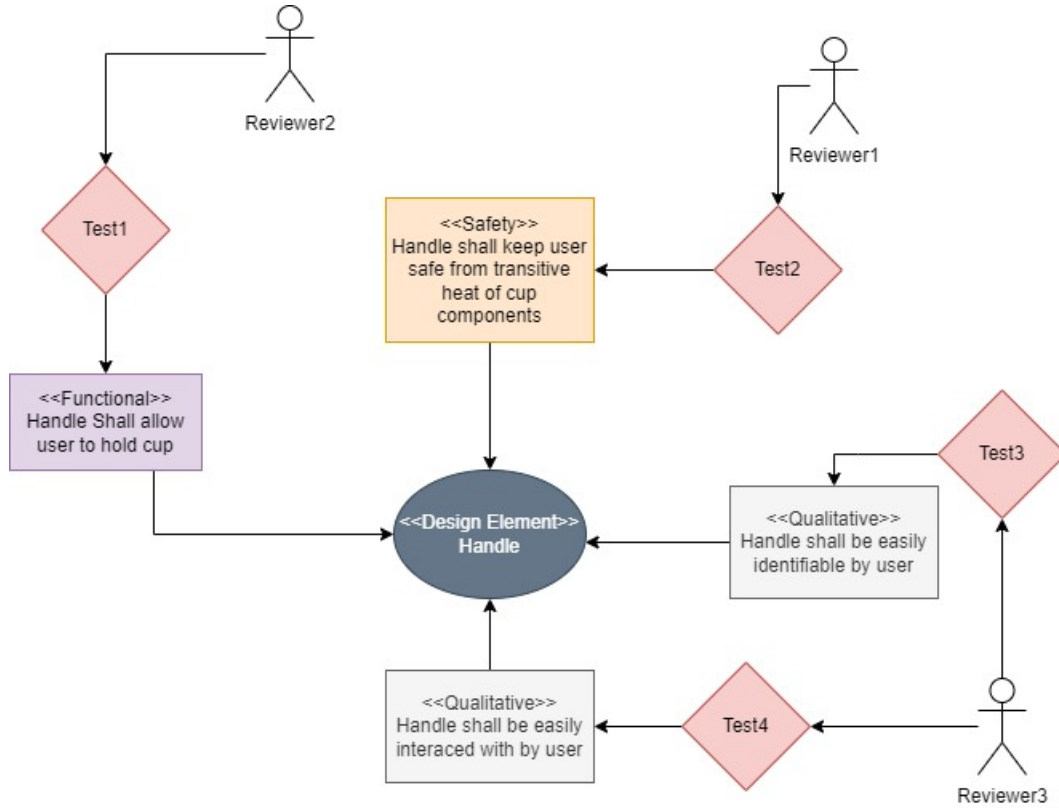


Figure 4.2: Example of a requirement canvas concrete syntax. The example uses a disposable coffee cup with a handle as the diagram’s focus.

4.2 Abstract Syntax

The metamodel shown in Figure A.1 served as the specification for building the tool for CyclicL. The abstract syntax can be roughly divided into two main parts: the feature model shown in Figure 3.9 and the requirement canvas shown in Figures 3.10 and 3.11. This aligns with our requirements for CyclicL. The requirement canvas portion of the metamodel is not a complete implementation of the SysML specification for requirement diagrams. The SysML specification served as a source of inspiration for the requirement canvas created in the tool for CyclicL. CyclicL has a greater emphasis on traceability and requirement specification that is not explicitly captured in SysML. Similarly, we have gone

a step further by specifying different types of requirements in our implementation, a detail not captured in the original SysML specification. We therefore drew heavily on the SysML requirement diagram specification, but did not limit ourselves to faithfully implementing it in our requirement canvas. This is why we refer to the requirement modelling portion of the tool as a requirement canvas rather than a requirement diagram. This captures our deviation from the SysML specification.

4.2.1 Feature-Requirement Encapsulation

To enable features to encapsulate requirements, the `FeatureEntity` class inherits from the `Requirement_Canvas` class. This implementation is what allows the features to encapsulate the requirements. We chose the inheritance relationship because it allows features to share the semantics of a requirement canvas while also allowing us to define a feature independently of a requirement canvas syntactically. Using another relationship for implementation, such as a composition relationship, would have introduced a strict ownership semantics that we did not find added value, and instead would have added an extra layer of abstraction between features and requirements, cluttering the traceability with useless information. Through this inheritance relationship, the features own the requirements directly without any abstraction. Further, the inheritance specification forces a 1-1 relationship between requirements and features, meaning that every new set of requirements requires a new feature to be encapsulated, reinforcing the convention of a unique feature per unique set of requirements. This relationship is highlighted in Figure 4.3.

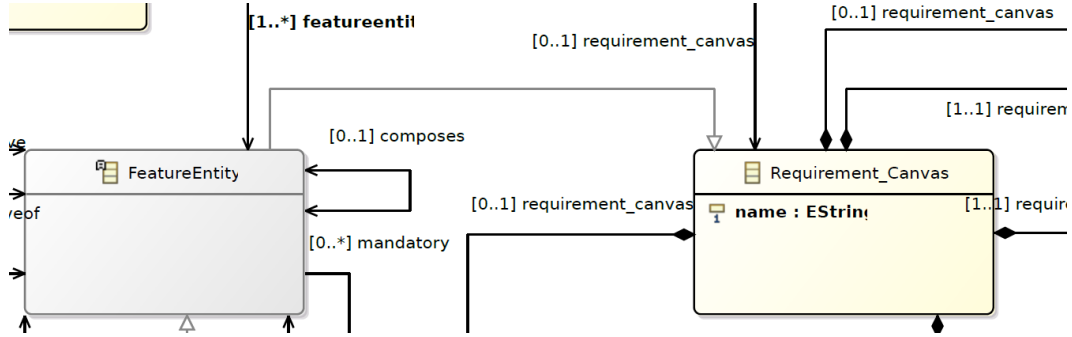


Figure 4.3: Specification of the inheritance relationship between the requirement canvas and the feature entities in CyclicL.

4.2.2 Feature Model Specification

The feature modelling specification of the tool is loosely inspired by the formal approach from Peter Höfner *et al.* [cite hofner2006feature,hofner2011algebra](#), with a small extension to account for requirement encapsulation. We use this specification rather than creating a new one to leverage prior proofs and work on formal specifications for feature modelling. It saved us time and let us begin development much sooner. We consider the relationship between features and requirements within a product family as shown in Figure 4.4. Using this definition, we treat each feature as a set of requirements, where each requirement within the feature is unique. As stated in the definition, however, it does not account for duplicate requirements in separate feature elements. This will be addressed in future iterations.

Specifying the hierarchy between features and requirements, both in a metamodel and using a set-theoretic approach, provides a better understanding of the mechanisms that link the two entities. The idea is that this will help specify and implement mechanical processes, for example, change impact analysis in the future using a formalized language like the Object Constraint Language (OCL) [\[52, 53\]](#), or another formal approach. This should help val-

- Let $\mathbb{F}$ be a set of arbitrary elements that we call features. (4.1)
- We can call a collection (set) of features a product. (4.2)
- The set of all possible products is $\mathbb{P} \stackrel{\text{def}}{=} \mathcal{P}(\mathbb{F})$ (4.3)
- A collection of products (an element of $\mathcal{P}(\mathbb{P})$) (4.4)
- is called a product line (or product family) (4.5)
- This model does not capture feature duplication (4.6)
- Let $\mathbb{R}$ be a set of arbitrary elements that we call requirements. (4.7)
- We can call a collection (set) of requirements a feature. (4.8)
- The set of all possible features is: $\mathbb{F} \stackrel{\text{def}}{=} \mathcal{P}(\mathbb{R})$ (4.9)
- This model does not capture requirement duplication. (4.10)

Figure 4.4: Definition of the relationship between features and requirements.

idate the mechanism and streamline development, as we can use one of its derivatives, such as [Eclipse Validation Language \(EVL\)](#) or [Acceleo Querying Language \(AQL\)](#), baked into Eclipse.

There are three defined relationships for features: mandatory, alternative, and optional. These are specified with bidirectional references to allow visibility in either direction from any feature. This decision was made to enhance traceability in the modelling environment and simplify development of the traceability matrices. The multiplicities are also defined with 0..1 at the top end to limit how the feature composition will work. This limits the number of nodes a leaf element can be connected to, constraining the models that can be built and simplifying feature composition. We also highlight the requirement set owned by the null feature based on the composition relationship from `RMDL_Project` to `Requirement_Canvas`. The justification for this structure is to provide an extra space for requirement specification that can later be refined by cutting and pasting the requirements into a canvas encapsulated by

a proper feature. To prevent instantiated projects from being cluttered with too many disconnected drawing spaces, we limit the multiplicity to $[0..1]$.

4.2.3 Requirement Canvas Specification

The requirement canvas portion of CyclicL was inspired by SysML requirement diagrams [11]; however, we did not implement it according to its original specification. The main reason for the divergence was our focus on implementing the feature-requirement hierarchy and specifically on developing for feature-requirement traceability. As a result, we decided to iterate on requirement diagrams as a means of exploration for traceability and requirement specification, leading to the inception of the requirement canvas. During exploration, we made several decisions at both the requirements and design levels for CyclicL that led to its current state at the time of writing. This includes the classifications for functional and non-functional requirements, the graphical syntax implemented, and the specification language embedded in the model. We explain these differences in the following passages.

Next, for the requirements in the requirement canvas, we identified four requirement categories: **Functional**, **Qualitative**, **Constraint**, and **Safety**. These are shown in Figure 4.5.

Functional requirements follow their traditional definition: things that a product or system must do. Qualitative requirements are those that affect how a product or system accomplishes its function. These include the look, feel, and usability, as well as humanity and some performance requirements that are not categorized as functional. Constraint requirements are those that impose limitations of some type on a product or system. These include operational,

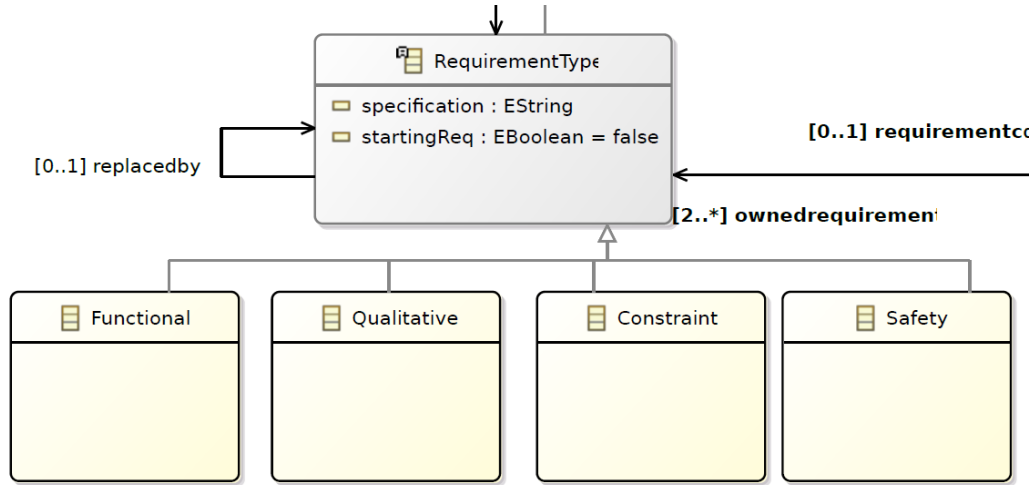


Figure 4.5: Specification for the 4 requirement types in CyclicL.

environmental, maintainability, support, security, cultural, political, and legal requirements. These requirement definitions can be found in Robertson Volere requirements [22]. We found a benefit for more specific categories beyond just non-functional requirements, as it lets the user be more intentional with their specification. We find this especially beneficial for identifying safety requirements separate from the other traditional non-functional requirements. The safety requirement category is critical for our purposes, as it identifies all requirements related to the safety of users or stakeholders involved in the function of a product or system. We highlight safety requirements as a separate category to target CyclicL towards safety-critical development. It is important to note that requirement categorization relies heavily on system or CyclicLfeature scoping. For example, we hypothesize that in a safety feature, a safety requirement may be considered a functional requirement due to the feature scope. A functional feature, however, may contain safety requirements that differ distinctly from its functional requirements. This distinction relies heavily on the engineer creating the model and the model’s context.

The effect of having these classifications front and center for the end user is that it forces them to consciously decide what kind of requirement they are creating at the time of development. Anecdotally, we have noticed minimal emphasis on the type of requirement identified. The process of determining the type of the requirement is often treated as tech debt, or is assumed by default to be a functional requirement. In this way, the end user may still assume that the requirement is a functional requirement; however, that detail is now explicitly part of the requirement specification, rather than left to interpretation at the head of the user and left up to interpretation by others during development, until the requirement type is explicitly decided.

We also added three more classes besides the requirements: **DesignElement**, **Review**, and **TestCase**, shown in Figure 4.6.

The test case and review classes are for verification and validation book-keeping, respectively. We also have an enum **TestType**. This enum allows users to categorize the tests users may want to apply to their requirements. At this time in development, only **UnitTest** is implemented as a type, but more can be added in future revisions. This is also one of the reasons why we include design elements within the requirement canvas. The test cases are meant to determine if a requirement has been verified against its traced design elements, while the review represents whether the requirement has been validated. Currently, verification and validation are expected to occur outside CyclicL because they are out of scope for initial development. Design elements are also meant to be black boxes as we do not want to pollute our requirement environment with design aspects. We have limited it to determining if a design element is hardware, software, or neither (undetermined when building the requirements or an integrated system). Another reason for including

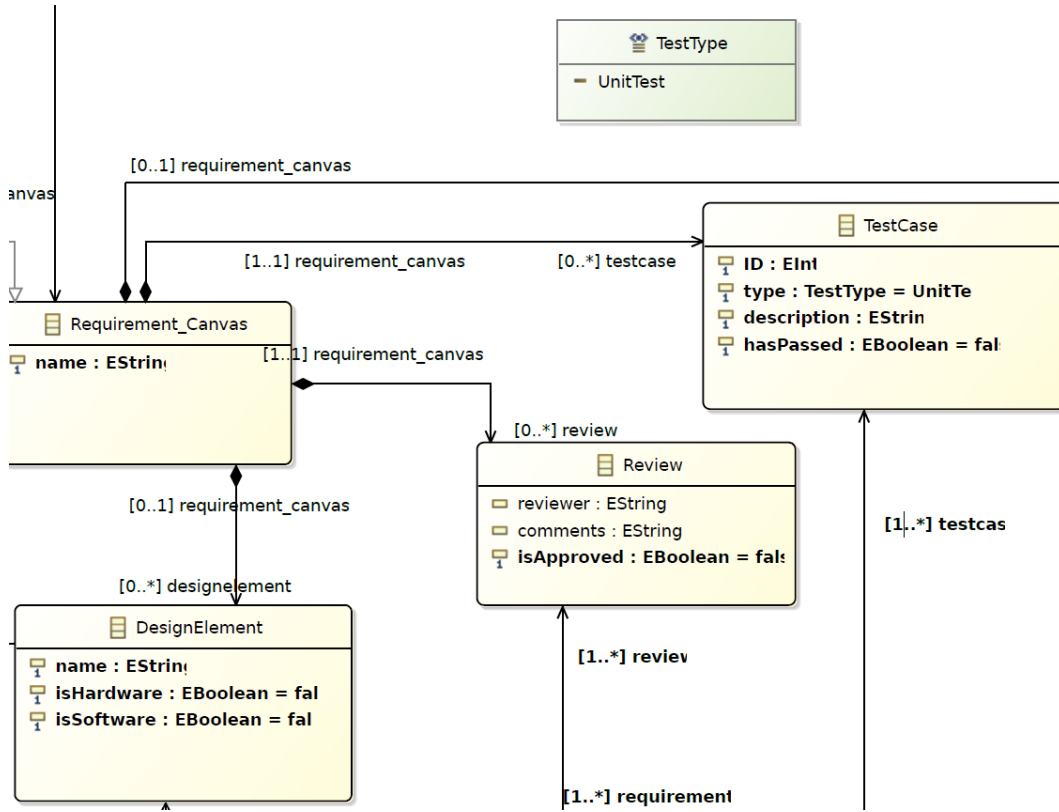


Figure 4.6: Specification for design, test, and review in CyclicL.

design elements in the requirement canvas is to provide an opportunity for future development of design generation based on the requirements specified in CyclicL.

Further, our specification allows for another interesting scope. We can have the 'null feature' scope. This is the requirement canvas that isn't owned by any feature model. This is enabled via the composition relationship from the project root class down to the requirement canvas class, shown in Figure 4.7. Due to its cardinality of 0..1, CyclicL allows the end user to create a requirement canvas for a no-feature requirement.

There are a couple of reasons why this can be helpful. Referring back to the scenario where we already have requirements before we start identifying

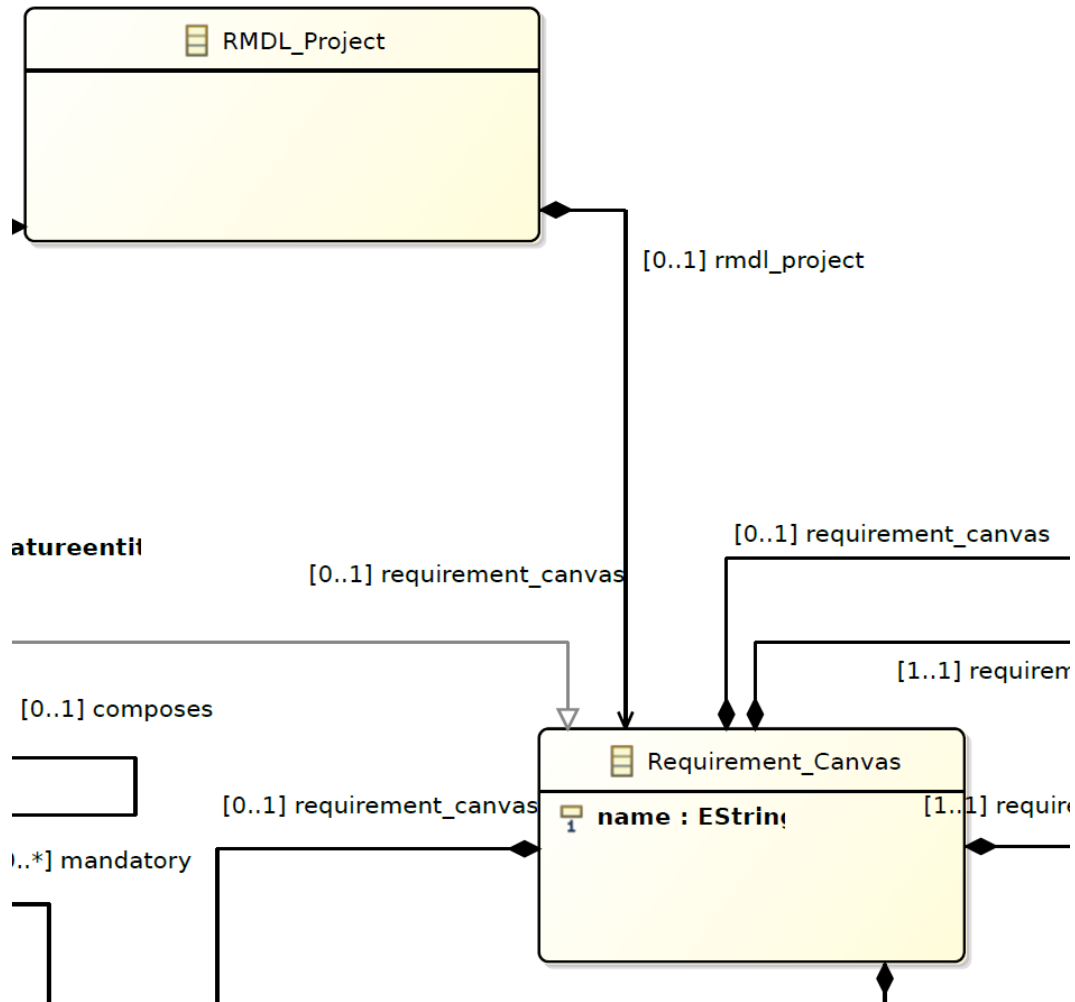


Figure 4.7: Specification for the requirement canvas owned by null. This provides a sort of drawing space or backlog for requirements that users have not yet sorted into a feature.

features, having a featureless requirement canvas allows the user to define their requirements right away. This way, they are present in CyclicL from the start and can simply move them into the appropriate feature at a later date. A simple way to ensure that all their requirements have been assigned to features is to keep the requirement canvas empty until the features have been identified.

Another possible scenario is the identification of new requirements during

product lifecycle development. They may not already have an owning feature identified. Rather than not specifying them, using the featureless requirement canvas allows the end user to fully specify the requirement and use the canvas similarly to a bank. The user can then return to the bank to withdraw the requirement once an owning feature has been identified.

4.3 Concrete Syntax

The design decisions for developing the concrete syntax for the requirement canvas follow the ideas from the Physics of Notation [54]. Various colours and shapes were used to maintain a bijective relationship between the concrete syntax and intended semantics. For example, in Figure 4.8, all of the requirements have the same shape to show they have commonality as they inherit from the **Requirement** type, but use different colours to differentiate themselves from each other. Similarly, **Test Cases** and **Reviews** are dynamically coloured to indicate whether they pass/fail and are approved/unapproved, respectively. Finally, the **Design Element** symbols dynamically change colour if they are stereotyped as software, hardware, or black-boxed.

For the feature modelling portion of CyclicL, we attempted to maintain a syntax as close to traditional as possible. This was to satisfy the ease-of-use and familiarity requirements. Maintaining the original syntax for feature modelling to the best of our ability during implementation should provide some intuitive familiarity to users familiar with feature modelling, while allowing a new user to learn relatively easily by leveraging previous work and documentation about PLE. Given Sirius’s default limitations, there are still differences.

Figure 4.9 shows an example product family created in CyclicL. The model’s

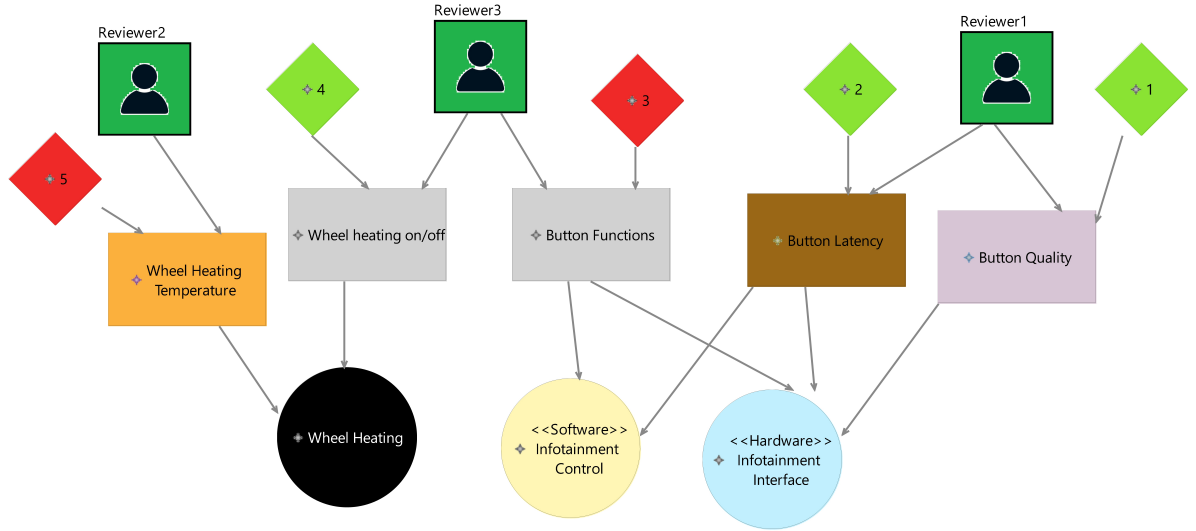


Figure 4.8: An example of the concrete syntax for the requirement canvas using requirements from the automotive domain.

root is shown in the grey box, and the model’s features are shown in white circles. The mandatory reference uses an arrow and a black diamond at the target to represent the AND relationship. The optional relationship uses a circle with a plus at the target to represent OR relationships. The alternative reference uses an arrow and a white diamond at the target to represent XOR relationships. In a top-down modelling layout, the reference source is at the top, and the reference target is at the bottom. We aimed to maintain a syntax as similar as possible to feature modelling conventions and to maintain a bijective relationship between the syntax and semantics.

Along with the graphical portion of CyclicL, we also implemented an Xtext-based [51] specification language. The implemented specification language is based on the Gherkin specification language [55, 49]. We chose this as the specification language for our requirements for a few reasons. Gherkin uses a structured natural language for specifications, thus making it relatively approachable for users when defining their requirements. Further, despite using

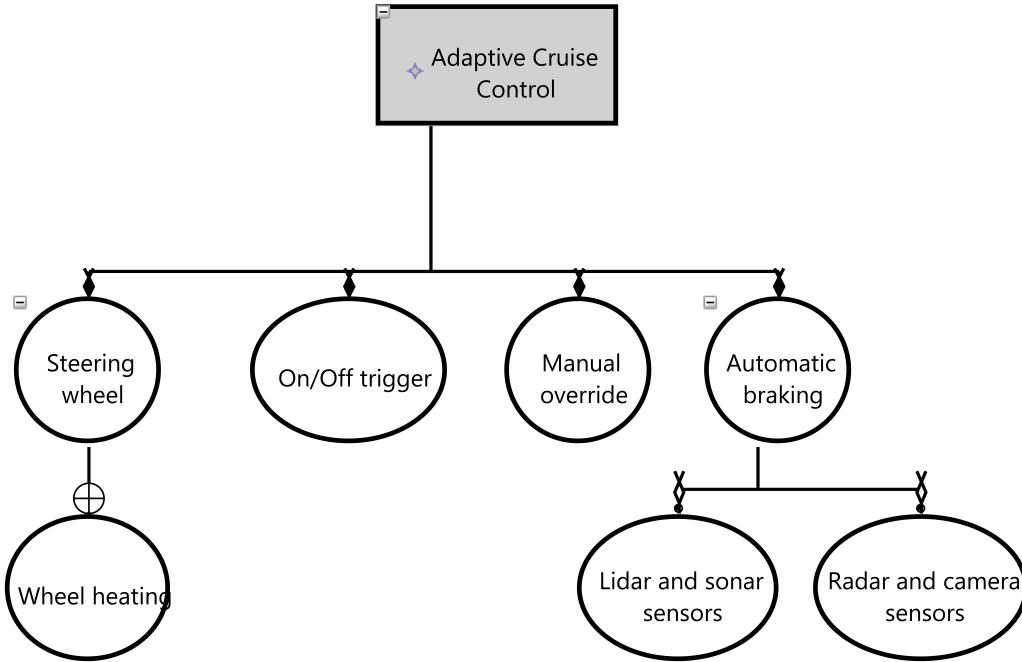


Figure 4.9: An example of the concrete syntax for the feature model using features for an adaptive cruise control system from the automotive domain.

natural language, it employs keywords to provide a predicate-logic structure to the specifications. The keyword ‘Given’ is used for preconditions, ‘When’ is used for events, and ‘Then’ is used for postconditions. The combined approach of adding predicate structure to natural language makes it relatively easy to specify requirements and to implement the language within CyclicL. This helps satisfy the ease-of-use and adoption requirement, as it makes the tool more approachable to learn and implement in a development process. We found that Gherkin struck a nice balance between formal and informal approaches to specification, aligning with Meyer’s requirement style, which inspired some of this methodology.

An example of what the concrete syntax is for the traceability matrices is shown in figure 4.11. This is a feature-requirement traceability matrix. The

```
Given{
  Precond FeatureExists: "Steering wheel has heating feature."
  Precond CarIsOn: "Vehicle is on."
}
When{
  Event UserTurnsOnHeating: "User interacts with heating
                             interface to turn on/off wheel heating."
}
Then{
  Postcond StateChange: "Wheel heating boolean changes state
                         to on or off depending on current state."
}
```

Figure 4.10: Gherkin specification for Wheel Heating on/off functional requirement. The high-level requirement description is: User shall be able to turn wheel heating on/off.

requirements are all in the first column. Subsequent elements in each row are directly traceable to the requirements via the relationships specified in the metamodel and are accessed via AQL queries. For example, to get the `Requirement` ID value, we use the query `aql:self.ID`. Since each row is each row is focused around the requirement in the first column, the only way to display anything in the traceability matrix is to have the model elements somehow be traceable to the requirement via any of the relationships defined in the metamodel in appendix A.4.

The design decisions of what to show in the traceability matrices are to show as much information as possible about the element from the first column. It is possible that some information may be removed in the future or new types of traceability matrices will be identified as needed. In the example from figure 4.11, this also includes the feature that owns the requirement to ensure we capture the hierarchical relationship in the matrix based on the same specification used for the modelling.

Feature-Requirement Traceability Steering Wheel X											
Requirement ID	Requirement Description	Requirement Specification...	Feature	Design Element	Test Case ID	Test Case Description	Test Case Type	Test Case Passed	Approval	Comments	Reviewer
2	Buttons shall allow user to interact with infotainment control.		Steering Wheel	[Infotainment Control, Infotainment]	[3]		[UnitTest]	[false]	[true]		[Reviewer3]
3	Wheel heating on/off	Given{	Steering Wheel	[Wheel Heating]	[4]	Test that heating element can be turned on/off	[UnitTest]	[true]	[true]		[Reviewer3]
1	Button Quality	Interface buttons shall have tactile feel to them	Steering Wheel	[Infotainment Interface]	[1]	[Test facility of buttons]	[UnitTest]	[true]	[true]		[Reviewer1]
5	Button Latency	Button shall be easily swappable in the event of a failure.	Steering Wheel	[Infotainment Control, Infotainment]	[2]		[UnitTest]	[false]	[true]		[Reviewer1]
4	Wheel Heating Temperature	Wheel heating shall remain within acceptable limits.	Steering Wheel	[Wheel Heating]	[5]		[UnitTest]	[false]	[true]		[Reviewer2]
Properties X											
Functional Wheel heating on/off											
Main											
Requirement Specification											
Semantic											
Given{											
Precond FeatureExists: "Steering wheel has heating feature."											
Precond CarIsOn: "Vehicle is on."											
}											
When{											
Event[] UseTurnOnValidation: "If user interacts with heating interface to turn on/off wheel heating."											

Figure 4.11: An example of traceability matrix concrete syntax following the Adaptive Cruise Control features and requirements.

4.4 Design Decisions

A major design decision for CyclicL was using our own implementation for feature modelling. Other tools such as FeatureIDE already exist and are native to Eclipse. We decided to use our own implementation to avoid constraints imposed by an existing tool, so we can focus on the traceability aspects of the tool. Another reason was to allow us some flexibility in implementing the new hierarchical relationship between features and requirements. Further, this gave us the flexibility to explore how we might navigate the created models and traceability matrices.

Another major design decision was diverging from the requirement diagrams of SysML to the requirement canvas. This was done to emphasize traceability of requirements and features as a primary function for this methodology and reinforce it in the implementation of the tool. Due to the time and scope of this thesis, the Goal Diagram and Use Case diagram were not included in the tool implementation of CyclicL.

We decided to rely on the majority of syntactic options available in Sirius for domain specific modelling development. This was both to speed up development and reduce the potential cost of restarting development, which we did several times as we iterated on the tool implementation. This is also true for the traceability matrices that are generated based on the models.

4.4.1 Composition Implementations

The relationship between features and requirements makes traditional composition techniques for PLE a challenge. Given our convention that a variant requirement necessitates a new feature, the path to composing features and

requirements to a product is relatively straightforward at this time. Currently, the approach focuses on the features themselves in the composition, assuming each feature has unique requirements. If a requirement applies to multiple features, it should be abstracted up a level in the tree to a higher feature. This currently reduces cross-cutting concerns and dependencies between requirements. This allows users to define the constraints and rules governing how the features within a model can compose into a product (e.g., every car needs 4 wheels). More complex analysis is planned for future work; however, as a proof-of-concept, CyclicL has the facilities to support such development. With the `Product_Variant` class in the metamodel [A.1](#), we provide the facilities for a user to manually create instances of their product. Also, because of the composition relationship we use, all product instances are owned by the feature model in which they are created. The inheritance relationship also means they don't need to be new semantic elements to represent the various features and requirements across product variants. We can leverage model slicing to create those new variants and instances. Naturally, the inheritance also allows all the traceability mechanisms to apply to the product variants the same way they do to the parent feature model.

4.5 Summary

In summary, the tool for CyclicL has shown promise for enforcing the syntax and semantics we desire. We implemented the concrete syntax while maintaining a bijective relationship between the metamodel definitions and the concrete syntax in the tool. Our design decisions were justified, and we satisfied the tool objectives we outlined before beginning development. In the next chapter,

we will explore a preliminary analysis of the implementation’s capabilities.

Chapter 5

Evaluation

Throughout this thesis, we have used the automotive domain as the basis for illustrative examples and to motivate the design of CyclicL and its features. This is because, within the automotive domain, we can identify two main issues that this methodology addresses: how to handle variants within a product family and maintain traceability across it. For automotive manufacturers, there are variants in the customer-facing product, evident in the different vehicles offered, the model-year differences between vehicles, and the various trims available within a single vehicle model. There are also variants that can exist internally to the manufacturer as they go through development and decompose the vehicle into its components: engine, transmission, cooling, and heating variants that are used across platforms, for example. Managing these product families across multiple levels of abstraction requires product family traceability to track the requirements constraints that drive the various design and implementation decisions.

For the evaluation portion of this thesis, we look towards the medical domain and medical device development. There is significant overlap between

medical device development and automotive manufacturing, as both involve safety-critical systems that must comply with various standards and regulations across geographic regions to sell their products to customers. The medical device we will be using for this evaluation is a pacemaker. From a safety-critical perspective, medical devices such as pacemakers tend to have a greater impact on individual health and safety, which is often reflected in medical regulations. Regional legislation may affect product development, and the products themselves may have variants tailored to the capabilities of target stakeholders, in this case, individual patients. As this research has been carried out in Canada, we reference medical device guidelines and classifications based on Health Canada regulations [56, 57] as we are from a Canadian institution. According to these guidelines, an implantable rate-responsive pacemaker is classified as a class 4 medical device. This is the most critical classification a medical device can receive, as it is considered a closed-loop system used to control the treatment of a patient’s condition. Thus, a pacemaker has high safety-critical implications. This justifies high levels of scrutiny for the requirements and potential variations, as there must be a great deal of confidence in the product lifecycle before being implanted in a patient.

This evaluation will use an existing requirement document from Boston Scientific A.2 for pacemaker development. We will assess how this methodology and CyclicL can be used to improve an existing requirement set and establish traceability between features and requirements. We will also consider potential integration points with development processes to evaluate the adaptability of the methodology and CyclicL and to minimize disruption to existing documentation and development strategies.

This evaluation aims to show feasibility and applicability over a full em-

pirical evaluation. Ultimately, to fit within the scope of a thesis, this is a proof-of-concept. Industrial and user evaluation, scalability testing, and quantitative analysis against existing technologies were out of scope for this thesis.

5.1 Knowledge Capture & Domain Analysis

The first step in this thesis is to examine how domain analysis is conducted. This part of the evaluation examines how we can enhance existing knowledge-capture approaches by introducing domain analysis techniques from our methodology. Using the proposed processes, along with CyclicL, we will assess the ability to identify gaps in the original pacemaker requirement document, potential integration points within the existing development process, and general enhancements to engineering practices.

We also want to identify what a pacemaker is. As previously identified in accordance with Canadian guidelines [56, 57] and the Boston Scientific requirements document A.2, we have an implantable rate-responsive pacemaker. The purpose of this pacemaker is to treat various forms of cardiac arrhythmia, or abnormal heart rhythms. There are three types of arrhythmia:

- **Arrhythmia:** A condition wherein the heart beats irregularly or abnormally
- **Bradycardia Arrhythmia:** A condition one can have is a Resting Heart Rate (RHR) that is abnormally slow. The question of “how slow is abnormally slow?” may differ from person to person because the heart’s RHR tends to decrease with age and in individuals with higher fitness levels. For most adults, a slow RHR is less than 50 or 60 Beats per

Minute (BPM). Symptoms of bradycardia may include fatigue, dizziness, fainting, heart palpitations and breathing problems.

- **Tachycardia Arrhythmia:** A condition wherein one has an RHR that is abnormally fast. For most adults, a fast RHR is above 100 BPM.

A pacemaker system typically comprises two main subsystems: the pulse generator and the leads. The pulse generator, also known as the “PG”, contains the battery of the pacemaker and the circuits that deliver the pacing stimuli. It provides protection for the pacemaker’s internal circuits and the patient’s heart from each other. There is a connector from the pulse generator to the leads subsystem. The leads are insulated wires that carry electrical stimuli from the pulse generator to the heart. They are the first component of the pacemaker to be implanted during surgery, usually via the subclavian vein. The electrodes at the end of the lead are attached to the heart.

Finally, as this is an implantable closed-loop system, a Device Controller-Monitor system is paired with the pacemaker. This device is used by physicians, nurses, and technicians to interrogate and alter the pacemaker’s behaviour according to patient needs. It provides a means of remotely transmitting programming data to the pacemaker while it is implanted and of receiving telemetry data from the pacemaker.

A more in-depth description of what a pacemaker is can be found in the appendix [A.3](#). The document provides greater detail on the Cardiac Conductive System and its interaction with the pacemaker.

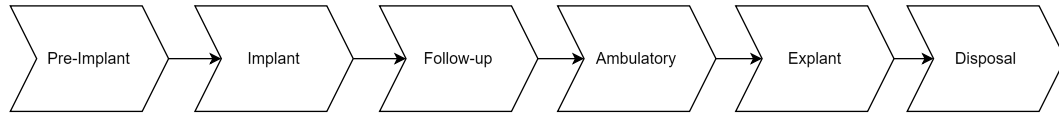


Figure 5.1: Pacemaker lifecycle as outlined by Boston Scientific requirements.

5.1.1 Goal Diagrams

The full requirement set by Boston Scientific for a pacemaker [58] is available in the Appendix A.2. Though not explicitly stated as domain analysis, knowledge capture occurs in chapters 1 and 2 of the pacemaker requirements, which are all captured in natural language. In fact, all the requirements specified by Boston Scientific are captured in natural language. These two chapters introduce the scope of the requirements, identify some stakeholders, and define the system and subsystems. We found it sufficient for engineers to have an idea of why the system is needed, who they are building it for, and where it is intended to be used. This comes from anecdotal accounts from industry engineers. They want to be able to justify their time and effort in building a product that has a purpose, even if they do not have all the details of the finished product. We now attempt to improve this domain analysis using the goal diagram and UCD approaches outlined in our proposed methodology.

We follow the same pacemaker lifecycle outlined in the requirement document shown in Figure 5.1. The lifecycle provides context for our stakeholders' goals throughout the pacemaker's life. We find that the document explains what is required of the pacemaker at each stage of its lifecycle. The document, however, does not explain what each stage actually is. The stakeholder roles, use cases, or goals are also not defined or outlined for each stage throughout the product lifecycle. This can reduce the engineers' understanding of the product and impact confidence in the requirements specified for product de-

velopment.

The lack of context and knowledge can create uncertainty during development. The extra definitions or knowledge of stakeholder goals may not always directly impact product features or requirements; however, the ability to draw this conclusion from a body of knowledge is important for increasing confidence in creating more complete and correct sets of features and requirements. To accomplish part of this domain analysis, we conducted a casual interview with a doctor to provide perspective and insight into stakeholder goals beyond the engineer responsible for development. Ultimately, this was a benefit, as the doctor helped provide context missing from the original pacemaker specification. Some questions asked were the following:

- From the perspective of the attending doctor, who are the stakeholders involved when a patient has a pacemaker?
 - A follow-up question was: what goals would the identified stakeholders have?
- What goals do you have as the attending doctor when caring for a patient with a pacemaker?
- What are your goals as a doctor in general?
- What goals would you share with the combined health care team for a patient with a pacemaker?
- In your experience, what goals does a patient with a pacemaker typically have? Why would they want or not want a pacemaker?

The first step we took in this domain analysis was to provide more information about what happens at each stage of the pacemaker lifecycle.

1. **Pre-Implant:** This stage is where the patient is assessed to determine if a pacemaker is required for treatment. This stage includes risk/benefit discussion, patient risk tolerance, and informed patient consent. The patient has the right to refuse care and may deny getting a pacemaker altogether. It may also be determined that, though the patient may need a pacemaker, it would not be in their best interest to have it implanted.
2. **Implant:** This stage is the surgery. The patient undergoes surgery to implant the pacemaker.
3. **Follow-up:** This stage occurs directly after the pacemaker is implanted. The patient is assessed for surgical safety (correct pacemaker implantation, no infection, no other surgical complications, etc.). This is the follow-up assessment to ensure the surgery was successful and to identify any unforeseen complications after the surgery is complete.
4. **Ambulatory:** This stage is the regular day-to-day life of the patient post-surgery. This includes regular check-ups with the patient's medical team. Regular pacemaker assessment is performed at this stage. If necessary, pacemaker maintenance or updates may be required. In extreme or unusual cases, pacemaker removal may be required or refused.
5. **Explant:** This stage is where a pacemaker is removed from the patient. A cost-benefit analysis is performed with the patient to determine whether removing the pacemaker is more beneficial than leaving it in. Also, the pacemaker has to be removed after a patient's death.
6. **Disposal:** This stage is when a pacemaker has reached the end of its life and is safely disposed of.

These definitions are helpful to engineers to get a better understanding of the complete product lifecycle after development. It helps inform the goals of the various stakeholders, providing insight into why the requirements are specified as they are. Further, these additional definitions helped us identify additional stakeholders beyond those in the original specification. Some new stakeholders were also identified during the interview with a doctor that were not captured in the original pacemaker requirement document. The original list of stakeholders is the following:

- Patient
- Doctor
- Nurse
- Technician

After following through on the domain analysis, we produced an extended list of stakeholders:

- Patient
- Patient Family
- Doctor
- Hospital
- Nurse
- Technician
- Manufacturer

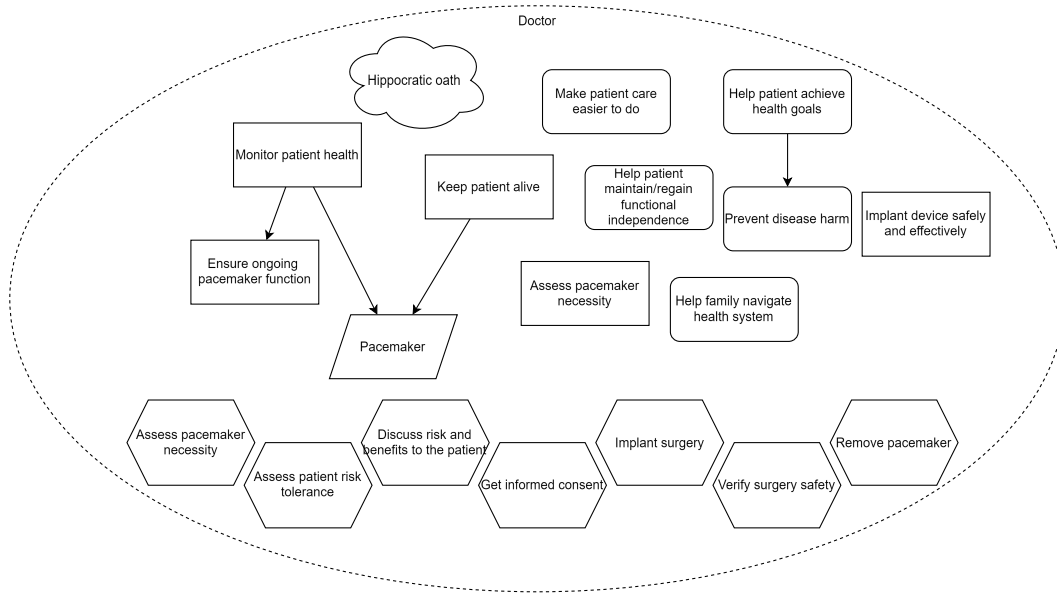


Figure 5.2: Doctor goals in the context of the pacemaker system.

The patient family was identified as another stakeholder during our interview with the doctor. They also share some of the goals identified for both patient and doctor stakeholders. One of the goals for doctors is to help the patient family navigate the health care system. Identifying this goal reinforced the idea that the patient’s family are also stakeholders of the pacemaker system, as they are likely to be involved with the patient’s quality of life. This ties into another goal we identified for the patient: to regain or maintain a desired quality of life. The goals of the doctor and patient stakeholders are shown in figures 5.2 and 5.3, respectively.

One important patient belief to identify is **We all die eventually**. This leads to the patient goal of the right to refuse care. It is critical to recognize patient autonomy in general. In the specific case of the pacemaker, a patient can refuse to have it implanted or explanted. Refusal to implant a pacemaker is less common, but informed consent is an important part of the process before a doctor can implant one. More commonly, especially in the setting of

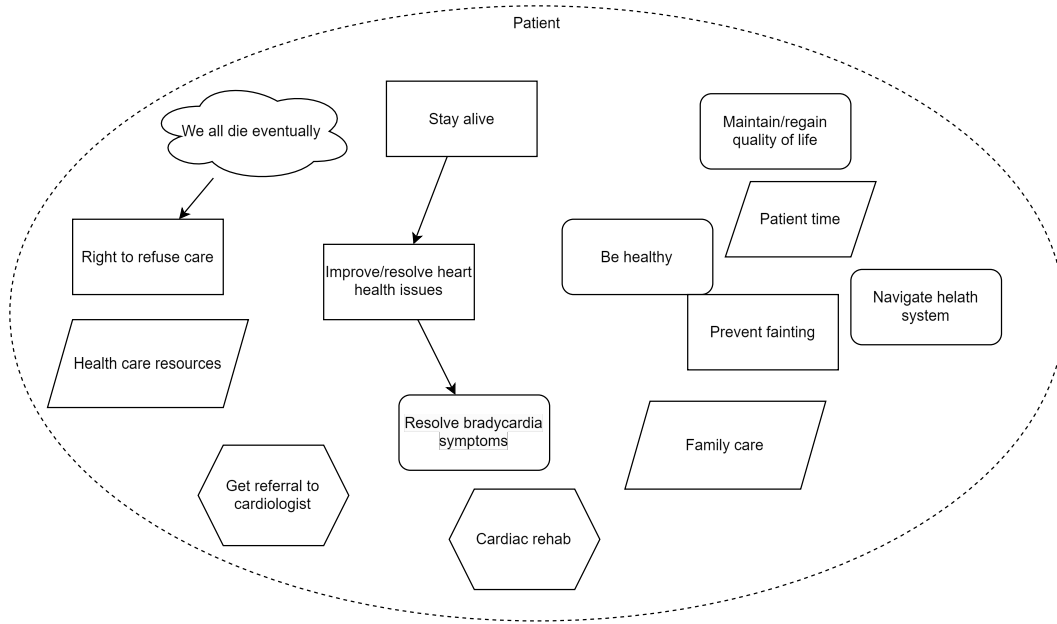


Figure 5.3: Patient goals in the context of the pacemaker system.

pacemaker failure, the patient may refuse to have the pacemaker explanted. This can be for a multitude of reasons, but often it is simply because the patient is old and does not want surgery. We point this out as a gap in the original specification, as this can have requirements and design implications.

We also identify what the goals are for the nurse and technician stakeholders. This helps clarify the roles they would play in the pacemaker lifecycle. During the ambulatory stage, the patient will spend a lot of time with the nurse to monitor the pacemaker’s health and function. The technician may be called to troubleshoot a bug or, in extreme cases, to dispose of the pacemaker. Disposal of a pacemaker requires returning it to the manufacturer due to the risk of battery explosion if improperly handled. We also add the hospital’s goals, though the hospital is mostly where the resources are sourced. There can be more goals for the hospital in general, but for this context we limited the goals to the scope of a patient with bradycardia and the pacemaker lifecycle.

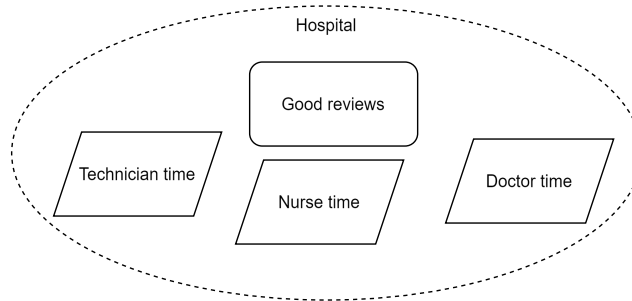


Figure 5.4: Hospital goals in the context of the pacemaker system.

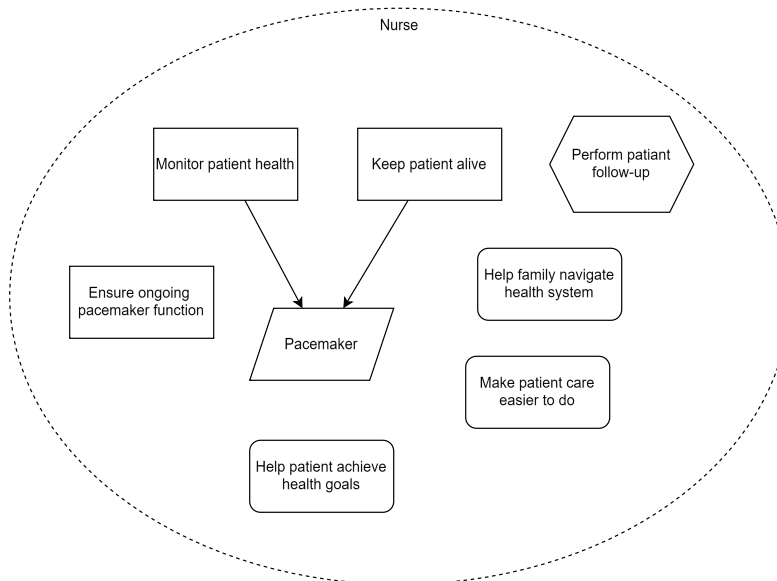


Figure 5.5: Nurse goals in the context of the pacemaker system.

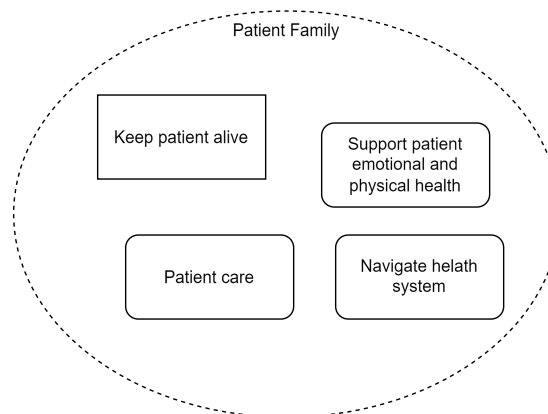


Figure 5.6: Patient family goals in the context of the pacemaker system.

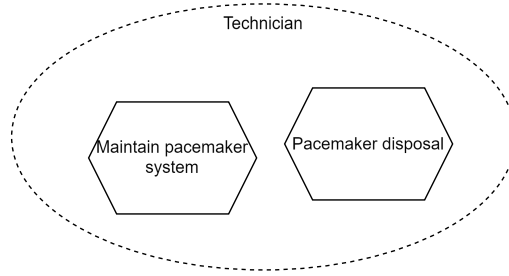


Figure 5.7: Technician goals in the context of the pacemaker system.

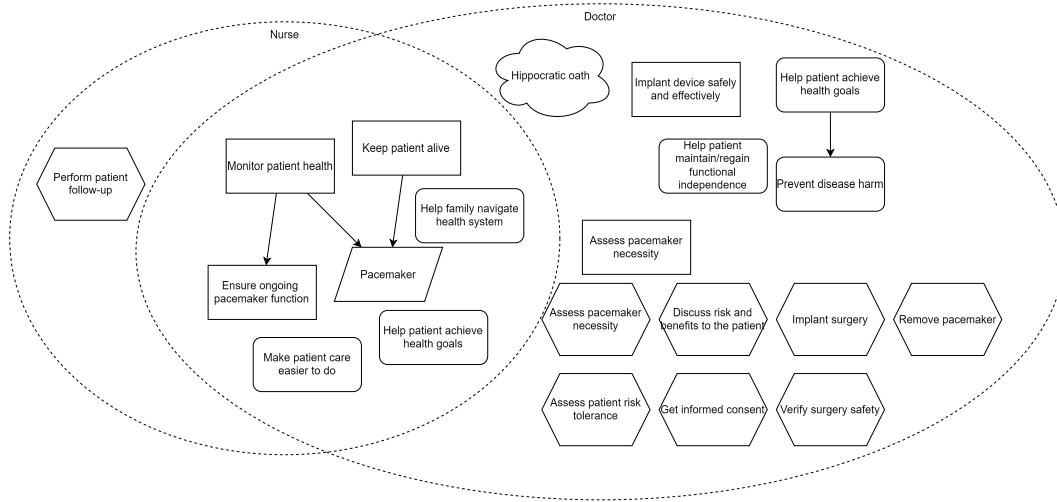


Figure 5.8: Combined nurse and doctor goals.

Along with the hospital, we identified another stakeholder beyond the scope of the original pacemaker requirement: the patient family. The reason we include the patient family as a stakeholder is that they are also relevant to the pacemaker lifecycle as a potential resource for the patient. The patient's family will also likely share the goal of helping care for the patient by performing tasks the medical team is unable to; helping with the patient's day-to-day activities and in the community.

We identified many shared goals among stakeholders. We have them as separate bubbles for the sake of displaying in this thesis, but another potential benefit of the goal diagrams is that we can create Venn diagrams that explicitly

show the overlap between stakeholders and reduce copied goals. We show the combined goals of the doctor and nurse stakeholders in Figure 5.8.

Thus far, the process of modelling our goals has identified two additional stakeholders not identified in the original requirement document. We have provided insight into the goals for each stakeholder, along with supporting beliefs, tasks, and resources. We can claim that we have reduced some uncertainty about the goals stakeholders have for the pacemaker product lifecycle. We also claim that we have provided additional context on the stages of the pacemaker lifecycle to help inform and support decisions on the requirements and subsequent development.

5.1.2 Use Case Diagrams

The next step is to identify the use cases for each of these stakeholders and compare them with the uses identified in the original requirement document. We have identified a variety of tasks (from the goal diagrams) for the stakeholders, which loosely break down into two categories: tasks that require the pacemaker and those that are human-supplemented. According to the methodology and our primary objective of developing the product, we want to identify the use cases for which stakeholders will need the pacemaker system. These use cases should also be traceable to one or more goals for the stakeholders. This is critical to establishing end-to-end traceability for the entire product lifecycle. For our most immediate task, the traceability helps to justify the identified use cases based on stakeholder goals.

In Chapter 2: System Definition of the pacemaker requirements document, the components comprising the pacemaker system are defined: the pacemaker

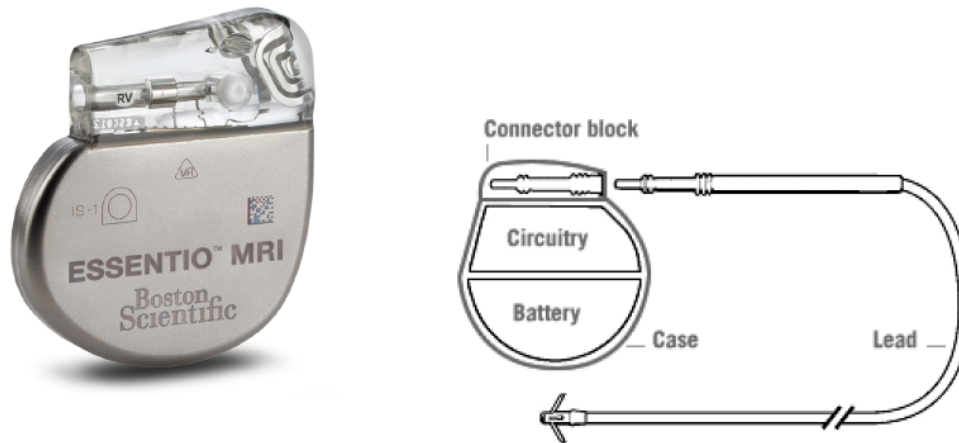


Figure 5.9: Image of a pacemaker pulse generator on the left. Diagram of a pacemaker device with leads. Extracted from the supplement document ‘What Is A Pacemaker’ found in [A.3](#).

device (also known as the pulse generator or PG), the Device Controller-Monitor and associated software, and the device leads implanted in the patient. Those three components become the boundaries of our UCDs. This is an easy integration point between our methodology and the existing documentation. We can save some rework by reusing the system and sub-system decompositions from the original document. The document does not explicitly state the features or use cases for each device, but there are overview sections we can use as a starting point for determining the use cases.

The PG device overview provides the following information [58] summarized from the PACEMAKER document found in [A.2](#):

- Monitor and regulate patient heart rate.
- Detect and provide therapy for bradycardia conditions.
- Provide programmable, single- and dual-chamber, rate-adaptive pacing, both permanent and temporary.

- In adaptive rate modes, an accelerometer is used to measure the physical activity resulting in a sensor-indicated rate for pacing the heart.
- Programming and interrogation via bi-directional telemetry from the DCM. Allows the physician to change the device's operating mode and parameters non-invasively after implantation.
- Provide the following historical data:
 - Sensor output data.
 - Atrial and ventricular rate histograms.
- In conjunction with DCM, provide diagnostic features including:
 - Real-time telemetry markers
 - EGMs
 - P and R wave measurements
 - Lead impedance
 - Battery status tests

The DCM overview includes two subsystems: a hardware platform and pacemaker application software. It also outlines the DCM's role. The DCM is the primary implant, pre-discharge EP support, and follow-up device for the pacemaker system. It can be used in the operating room, the doctor's office, and the electrophysiology lab. The DCM communicates with the PG using its software and hardware sub-systems. The DCM overview provides the following information:

- Program and interrogate a pacemaker.

- Command delivery of “Pace-Now” pace.
- Acquire and show diagnostics (history) and lead signal measurement information.
- Acquire and show sensor history and trending information.
- Show visible and audible indications of communication between the DCM and PG device, including beeping and LED’s for alerting the operator to error conditions.
- Acquire and show multi-channel monitoring including surface electrogram and telemetered signals (e.g. EGM, annotated event markers).
- Print reports and strip charts.
- Monitor battery status.
- Output to external strip chart recorders.
- Manage windows for display of text and graphics.
- Set the date and time.

The lead system overview provides the following information:

- The lead system implanted in the patient allows the device to sense the intrinsic electrical activity of the heart and deliver pacing therapy.
- The leads are connected to the PG via its header. All IS-1 bipolar leads are supported.

Based on the various overviews, we can begin creating the UCDs. A helpful insight the overviews provide that has been missed thus far is how the subsystems will interact with each other. Within the UCDs, we can model this by treating the various subsystems as stakeholders in the diagram environment. Based on the overview of the PG device, we can show the UCD in Figure 5.10. Three actors are explicitly outlined in the original specifications: the doctor, the patient, and the DCM. In this interpretation, we treated sub-bullet points as mandatory use cases of the original use case. This is shown in the ‘Provide diagnostic features’ use case and its various included use cases. The same strategy is repeated for the ‘Provide historical data’ use case.

In the original specification, the programmable pacing is listed within a single bullet point. However, when we attempted to model that point as a use case, we found it was highly overloaded. It inadvertently hid a lot of information behind a single use case. How many types of pacing are there? How many of the types of pacing need to be implemented and programmable in the pacemaker? How are the types of pacing related to one another? Does each type need to be implemented independently, or can we identify reusable portions of the system based on the requirements? By breaking it up and modelling it in a UCD, we explicitly state the information that was previously hidden in the specification. We can identify that both single- and dual-chamber pacing will be reused by the rate-adaptive part of the use case. Although not explicitly mentioned in this section of the original specification, it implies that the pacemaker should support both rate-adaptive and non-rate-adaptive pacing. This is further supported by the table on page 17 (see Figure 5.11) of the original specification, which outlines the pacemaker’s operating modes.

We took a further liberty with the following ‘an accelerometer is used to

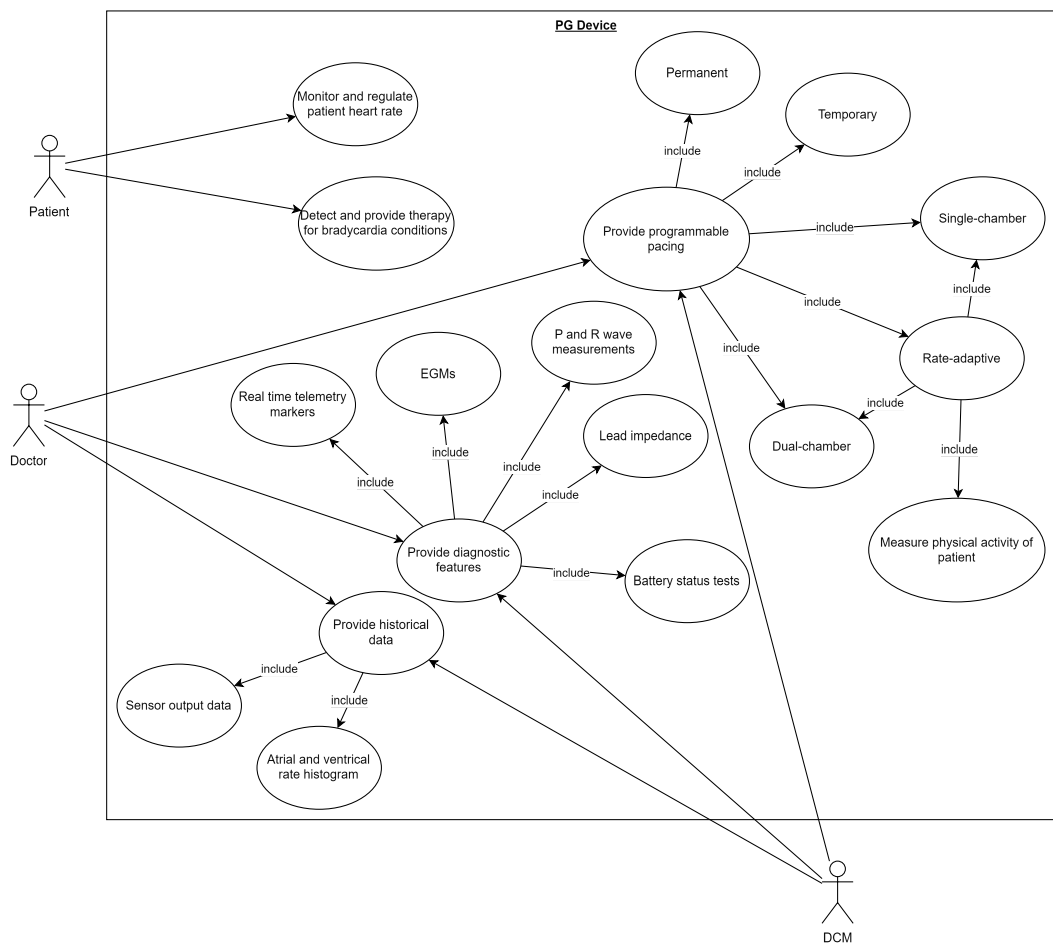


Figure 5.10: Use Case Diagram for the PG device based on the original pace-maker specification.

	I	II	III	IV (optional)
Category	Chambers Paced	Chambers Sensed	Response To Sensing	Rate Modulation
Letters	O–None A–Atrium V–Ventricle D–Dual	O–None A–Atrium V–Ventricle D–Dual	O–None T–Triggered I–Inhibited D–Tracked	R–Rate Modulation

Table 2: Bradycardia Operating Modes

Figure 5.11: Intended operation modes for the pacemaker system.

measure the physical activity resulting in a sensor-indicated rate for pacing the heart’ use case. Since it is still related to rate adaptive pacing, we model it as another included use case along the chain. Thus we can show how the doctor interacts with the ‘Provide programmable pacing’ use case and all of its included use cases along the relationship chain. These connections are not obvious from the natural language requirements in the original document, but become much clearer when the UCD is introduced as part of the process. It forced us to consider who was interacting with the use cases and how the identified use cases might imply knowledge that could be beneficial for further requirement specification or design. We have identified a possibility for reusable artifacts extremely early in the development lifecycle. We have also refined our use cases with minimal overhead to the original document.

In Figure 5.12, we have created another UCD enhanced by the knowledge gained from the goal diagram. We identified an additional actor that has use cases: the nurse. The nurse is never explicitly stated as an actor when using the PG device in the original specification. In our goal diagram, we have identified significant overlap between the nurse and doctor stakeholders. Aside from programming the pacemaker, which the specification explicitly states should

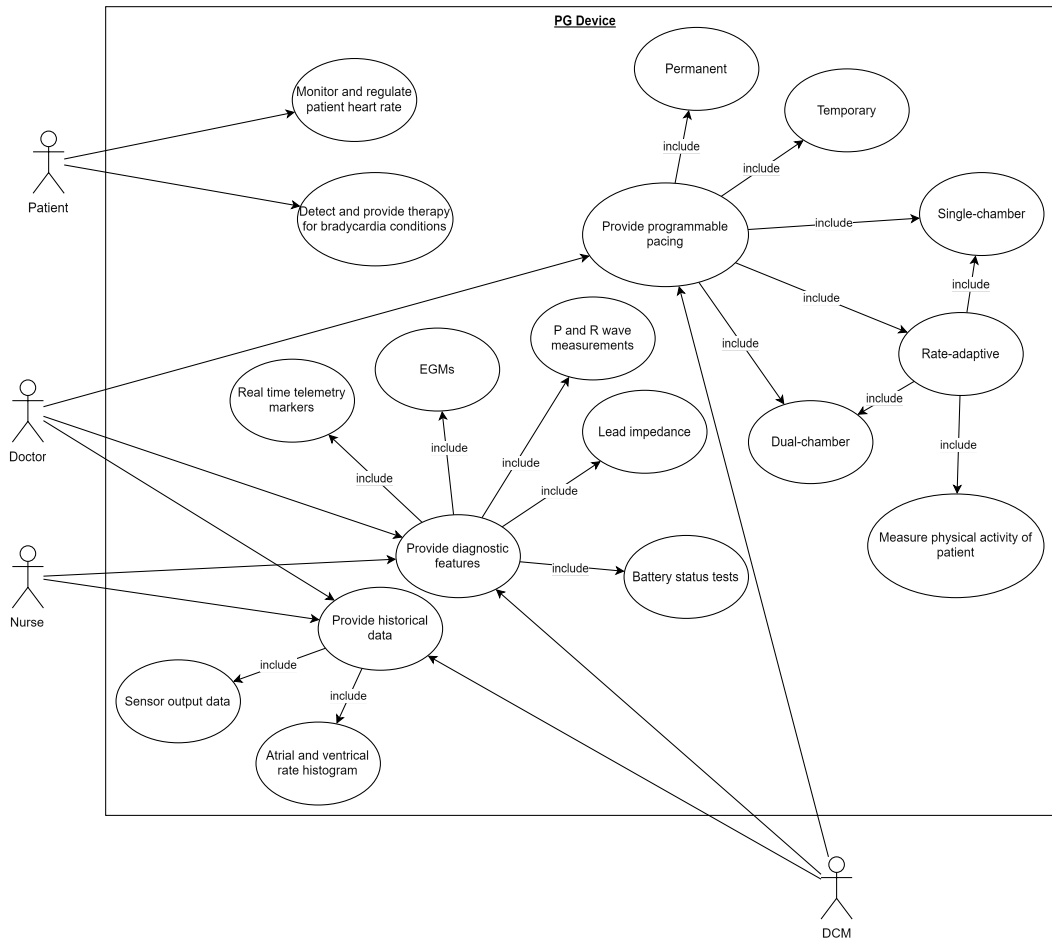


Figure 5.12: Use Case Diagram for the PG device based on the original pacemaker specification enhanced with information from the goal diagram knowledge capture.



Figure 5.13: Use Case Diagram for the DCM based on the original pacemaker specification. This diagram combines the hardware and software sub-systems into a single boundary.

be done by the doctor, the nurse also has a unique task of patient follow-up that requires them to access the patient’s historical and diagnostic data.

Next, we see in Figure 5.13 a possible interpretation of the information in the DCM overview in a UCD. While the original specification states that the DCM subsystem is further decomposed into a hardware platform and a software application, it does not distinguish between the two when specifying the use cases. In this initial modelling attempt, we try to define the use cases for the DCM with its components combined within the system boundary. We also discovered another actor during this UCD: the Strip Chart Recorder.

This actor is related to the singleton use case ‘output to external strip-chart recorders’. This UCD was particularly challenging to create due to all the conjunctive sentences in the specified features. In particular, the ‘Acquire and show diagnostics (history) and lead signal measurement information’ use case is difficult to fully capture who it is for. The language is confusing, as using the word diagnostics makes the statement seem like this feature is for the Technician. However, because it then specifies history in brackets, similar to what is specified in the ‘Acquire and show sensor history and trending information’ use case, we ultimately decided it is more likely to be used by the doctor and nurse actors. Interestingly, likely due to the overlap between the two stakeholders, the doctor and nurse actors in this model are associated with all the same use cases. Given their overlapping goals, we have greater confidence in the model’s correctness.

It is more challenging to have confidence in the UCD of the DCM system due to the combined nature of the hardware and software sub-systems specified in the requirements. They are coupled in section 2.2.2 of the PACEMAKER document, which makes it difficult to identify what a software use case is and what a hardware use case is, as they are currently written. More information may come to light during refinement; however, at this point in development, it is difficult to gain further insight into how the subsystems could be decoupled.

The leads subsystem is relatively straightforward to model given the limited use cases identified. It identified two actors that will use it: the patient and the PG device. However, we have included the DCM as an actor for this system as well. We justify this because the DCM is the interface that allows human actors, such as doctors or nurses, to program or interrogate the pacemaker.

Overall, creating the UCDs for the pacemaker has given us insight that

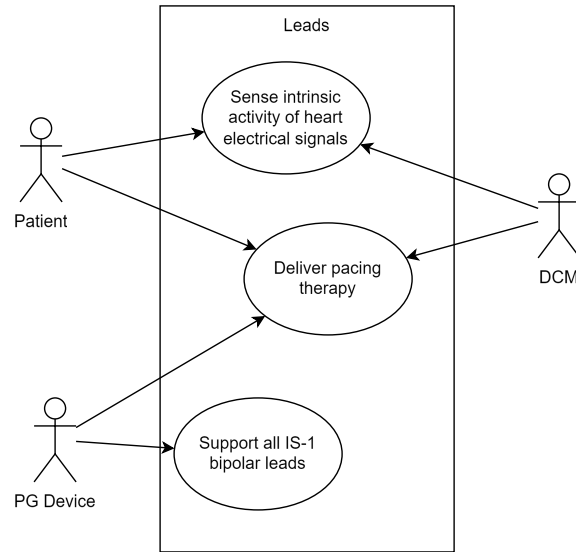
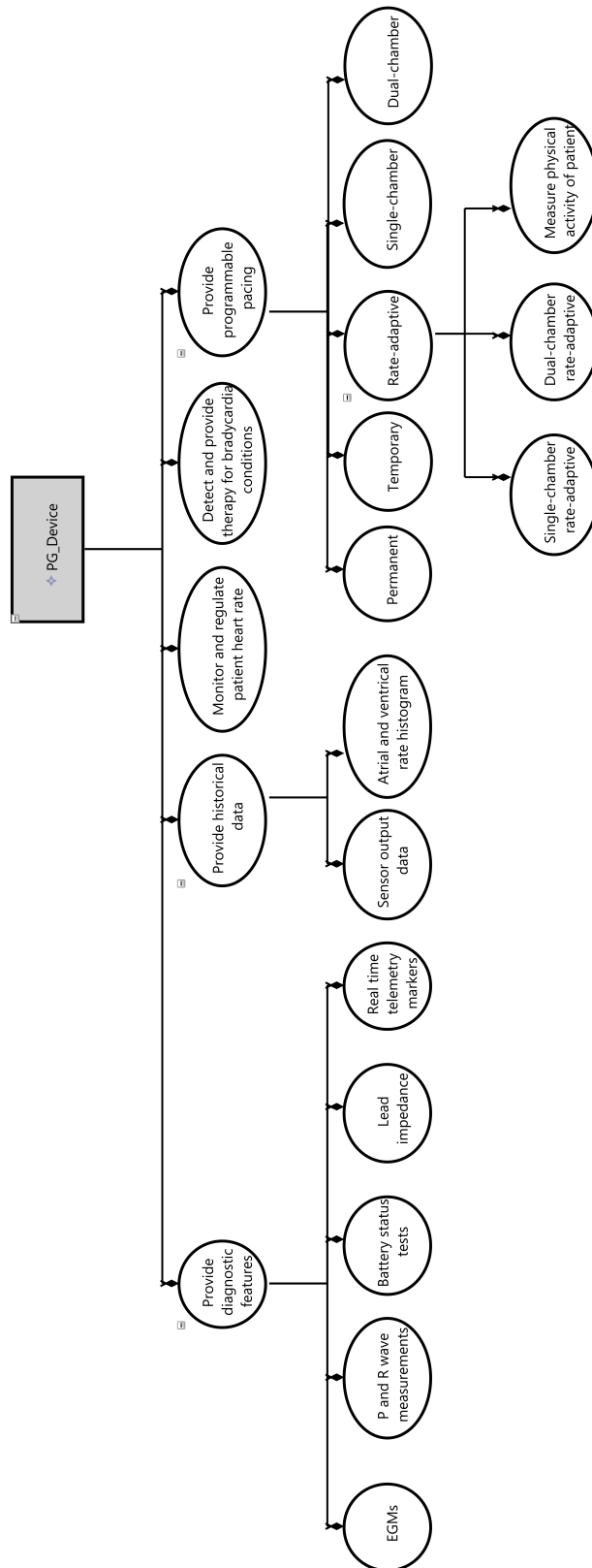


Figure 5.14: Use Case Diagram for the leads based on the original pacemaker specification.

would have been overlooked if we had relied solely on the original specification. We have identified potentially reusable parts of the system in the rate adaptive pacing. We have identified additional actors that were otherwise implied or overlooked in the original specification. We have identified potential pain points due to high coupling among the DCM subsystems. Finally, we have identified interactions among the various components of the pacemaker system. We also noticed during this process that a lot of interpretation is required to create the models. This can introduce inconsistencies, as the results depend heavily on the engineers who execute this process.

5.2 Product Family Development

One of the benefits of this methodology becomes apparent at this stage of development. Leveraging the equivalency we established between use cases and features, all the use cases we used in our UCDs are also the features we



106

will use in our feature models. In Figure 5.15 we show a potential product family based on the UCD for the PG_Device. We say 'potential' because there is variability in how engineers interpret the information from the UCD and goal diagram to create the feature model. For example, single- and dual-chamber pacing appear twice within the product family because both rate-adaptive and non-rate-adaptive pacing are required. There is overlap between the two features, but we put the rate-adaptive pacing under that feature and the non-rate-adaptive pacing on a separate level of the model.

Based on the UCD for the PG_Device, every use case is necessary for the system, and we identified the reuse of single- and dual-pacing across rate-adaptive modes. As a result, there is no potential for variability in the final product, since all features are mandatory for completing the system. This provides a starting point for developing the product. We do not take this model as complete, as we expect to learn more about the system through the requirement refinement process. As we refine the model, this presents opportunities for iteration between the phases of this approach. As more features are identified, there can be benefits in iteratively linking the feature model and UCD to explore how they relate to the actors and other features in the system.

5.2.1 Composition

Composition is a key component of the modelling strategies for this methodology. It is through composition that we encapsulate feature requirements; each requirement is owned by a specific feature of the product. This prevents floating requirements that cannot be traced to a specific feature. It also im-

plicitly forces each feature to be well-defined, at least based on the quality of the requirements it owns. The composition relationship between features and requirements in this methodology enables us to achieve feature-requirement traceability as an almost by-product of good engineering practice.

Due to the current metamodel definition of *CyclicL*, feature composition differs slightly. Since we state that all requirements are scoped by their owned features, we do not support cross-cutting requirements. When a feature composition is required to define a product based on the feature model, it becomes more of a checklist rather than more traditional composition weaving. This is by design, as we want each feature to be self-contained with respect to its requirements. This does not exclude dependencies between features, however. Feature dependency validation can still occur during the composition process if properly implemented. At this time, feature composition is not an automated process in *CyclicL*. It requires the user to manually select which features from the product family to add to a product variant they define. Despite being a manual process, the tool specification helps ensure that when a feature is added to a product, all owned requirements are added as well to obtain a valid result, thanks to the natural semantics of composition in SysML [11].

5.3 Requirement Engineering and Refinement

With a product family derived from the UCD, we can start exploring use cases and refining them into more well-defined requirements. We do this by taking the use cases and refining them into user stories. A user story outlines the steps a user (a stakeholder) would need to take when using a product (a use case) to achieve a goal or task, as outlined in the goal diagrams. We use the

following structure when refining a use case to a user story:

- **Use case:** Provide programmable pacing
- **Actor:** Doctor, DCM
- **Precondition:** Pacemaker PG_Device is active and can connect to the DCM.
- **Trigger:** Doctor wants to program pacemaker to mitigate patient bradycardia symptoms via DCM interface.
- **Success Scenario:**
 1. Doctor accesses PG_Device via DCM.
 2. Doctor verifies they are connected to the correct PG_Device.
 3. Doctor selects the desired pacing operation mode.
 4. Doctor sets the various parameters based on the chosen pacing operation mode (pace pulse characteristics, rate sensing settings, delays, etc).
 5. Doctor pushes programmed mode and parameters to the PG_Device.
 6. Doctor disconnects DCM from PG_Device.
 7. Doctor logs off from DCM.
- **Secondary Scenarios:**
 - Doctor connects to the wrong PG_Device via DCM.
 - The doctor makes a mistake when programming the PG_Device via DCM.

- Communication fails or is interrupted during push from DCM to PG_Device.
 - Doctor is unable to access the PG_Device via DCM.
- **Success Postcondition:** PG_Device is correctly programmed and pacing the patient’s heart correctly and safely.

The steps outlined in the success scenario are the functional requirements for the use case. They outline what the actor needs to do to accomplish the task. The outcomes in the secondary scenarios provide alternative results of the actor’s actions. These alternative outcomes can be safety hazards, security hazards, system malfunctions, or user errors. It is important in this step to ensure that ‘what’ the stakeholder has to do to achieve their goal is captured, not ‘how’.

The secondary scenarios outline the justifications for mitigating requirements to improve the chance that the actor will achieve the success postcondition, in this case that the PG_Device is correctly programmed and that the patient’s heart is paced correctly and safely. In the context of our methodology, mitigating requirements are most commonly categorized into safety, constraint, or qualitative requirements. At this time, we have not ruled out that a mitigating requirement may lead to a functional requirement; however, based on our experience, we support the convention that they are better suited to non-functional requirements. The secondary scenarios come together to answer the question of ‘how’ the stakeholder will accomplish their objective, typically by creating requirements that reduce the likelihood of the stakeholder reaching a secondary scenario.

5.4 Tool Supported Development

Having completed many steps in this methodology, we have developed tool support for the remaining steps. The tool support for CyclicL was developed using Sirius [50] based on the metamodel specification from A.1. We can see how the tool supports our objectives of traceability through our feature-requirement encapsulation and the implementation of the various relationships between model elements from the specification.

With these requirements identified, we can begin modelling them in the requirement canvas. In CyclicL, the requirement canvas is embedded within the product family’s features. This is the feature-requirement encapsulation that we outlined in the methodology. In Figure 5.16, we show how a user can create an embedded requirement canvas based on our tool implementation.

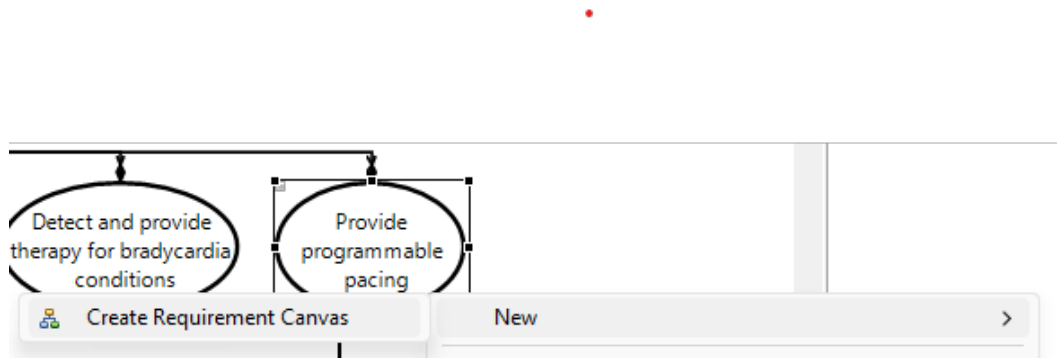


Figure 5.16: When right-clicking on a feature, a context menu appears allowing users to create a requirement canvas that is embedded in the selected feature.

In Figure 5.17, we show all the requirements created in the model as previously identified to match the refinement of the user story. In Figure 5.18, Figure 5.19, and Figure 5.20, we also show the Gherkin specifications for the expected behaviour of the first three requirements.

The requirement entities in the canvas have facilities that allow us to specify

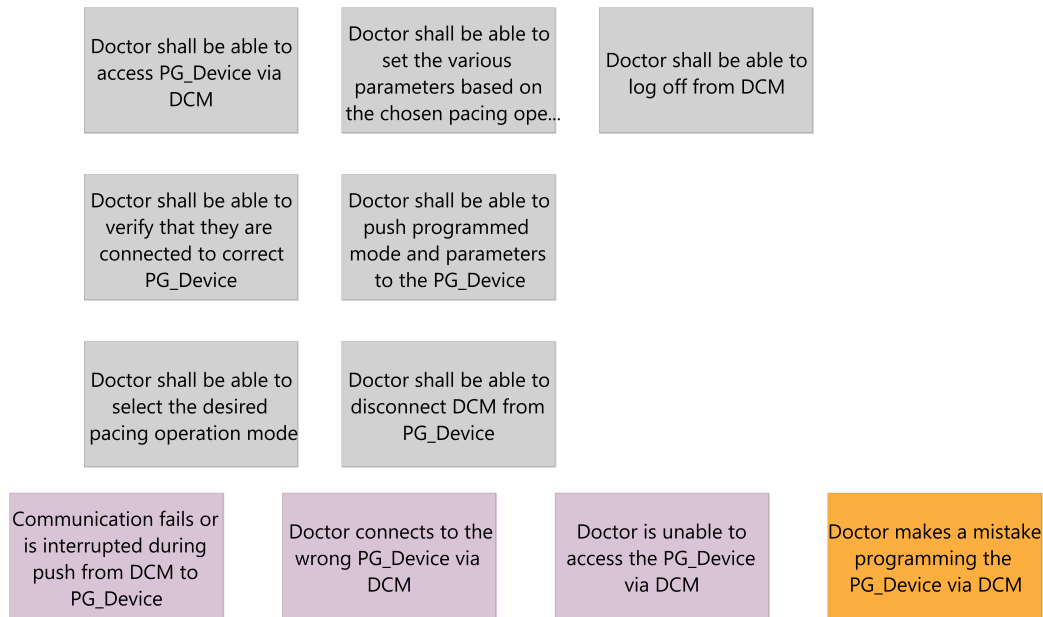


Figure 5.17: Requirements for the ‘Provide programmable pacing’ use case.

```

Given{
    Precond one: "The pacemaker PG_Device is active and can
                  connect to the DCM"
}
When{
    Event one: "Doctor wants to program pacemaker to mitigate
                patient bradycardia symptoms via DCM interface"
    Event two: "The doctor attempts to access the PG_Device via
                DCM"
}
Then{
    Postcond one: "The doctor is able to connect to a PG_Device"
}

```

Figure 5.18: Gherkin specification for the first requirement of the PG_Device.

our requirements in greater detail. Based on the user stories, we refined the use cases into requirements necessary to satisfy the successful postcondition, and we generated an implied ordering of the functional requirements. We can see this ordering reflected in the Gherkin specifications. The postconditions for

```
Given{
  Precond one: "The doctor is able to connect to a PG_Device"
}
When{
  Event one: "The doctor wants to verify they are connected to
             the correct device"
}
Then{
  Postcond one: "The PG_Device shall communicate to the doctor
                via DCM identification to allow the doctor to verify
                device identity"
}
```

Figure 5.19: Gherkin specification for the second requirement of the PG_Device.

```
Given{
  Precond one: "The doctor has verified they are connected to
               the correct device"
}
When{
  Event one: "The doctor wants to change or select the desired
             pacing mode"
}
Then{
  Postcond one: "The doctor shall be able to select from the
                available pacing options"
}
```

Figure 5.20: Gherkin specification for the third requirement of the PG_Device.

one requirement often are reused as the preconditions for the next. The tool does not make this relationship explicit, but the specification does allow for it. This provides some flexibility to engineers using this methodology in how they specify relationships between their requirements. These requirements are identified based on the user stories we created in a previous step. Thus, the ordering and identification of these requirements are a natural consequence of refining our use cases to explore how they help our stakeholders satisfy their

tasks and goals.

Another benefit of these ordered requirements is that they can support planning for feature development. Engineers can more easily identify dependencies between requirements within a feature and plan which stages of development must be completed sequentially and which can proceed in parallel. This lends confidence to the claim of this methodology helping with FDD. The feature is well defined by the requirements; each requirement can be clearly specified by the engineers, and plans can be made to determine the development order to complete the feature. This includes satisfying functional, qualitative, constraint, and safety requirements alike.

Another part of the requirement canvas in our tool implementation is the ability to assign test cases and reviewers to specified requirements. At this point in development, the purpose of those model elements is primarily book-keeping. A further benefit of using the Gherkin syntax is the many existing libraries that support it for testing. At this time, we do not natively support testing in the tool. We do, however, want to keep track of which requirements have been satisfied and which have not. Additionally, we also want to keep track of which ones have been reviewed and approved. As part of the requirements engineering process, it is important to track which requirements are ready for development and which are still undergoing refinement. We show, based on our tool implementation, how this can appear for the pacemaker in Figure 5.21. Naturally, once these model elements are added and connected to requirements, they are automatically updated in the traceability matrix.

At this stage, we have created a functional behavioural specification for the “Provide programmable pacing” use case that extends beyond the original specification. We have justified our requirements using the use cases from

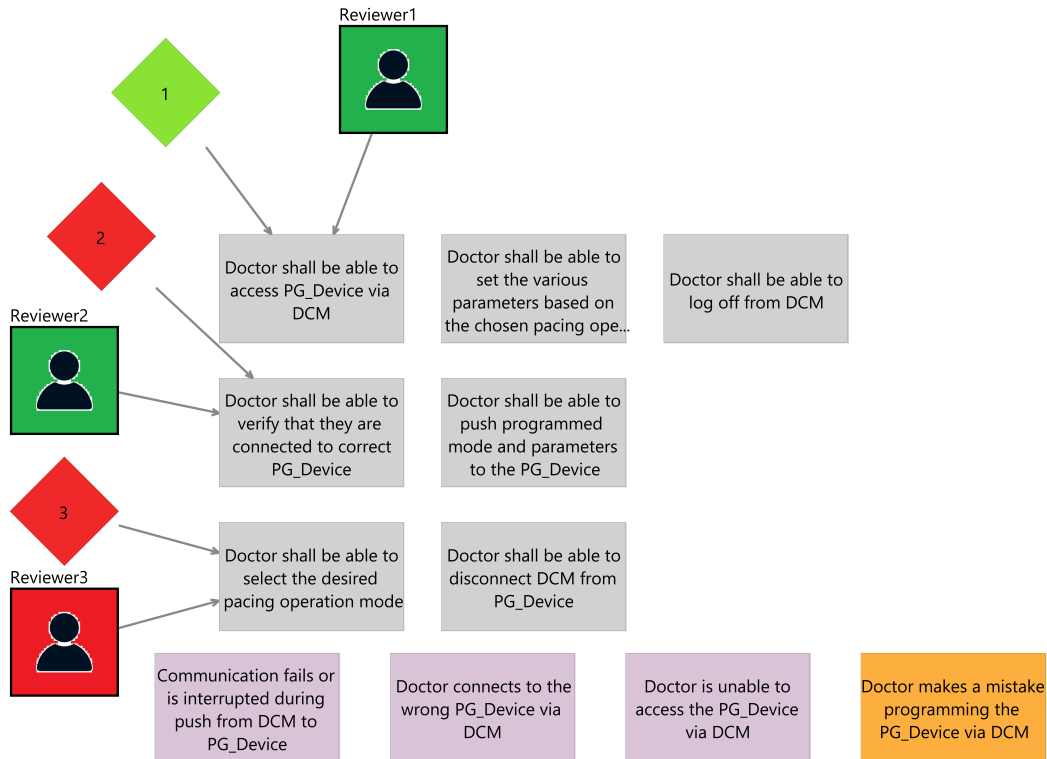


Figure 5.21: Various stages of approval and testing for the requirements.

Requirement ID	Requirement Description	Requirement Rationale	Requirement Specification	Feature	Design Element	Test Case ID	Test Case Description	Test Case Type	Test Case Passed	Approval	Comments	Reviewer
1	Doctor shall be able to access PG_Device via DCM		Goal	Provide programmable pacing	1	101	101	UnitTest	101	101		Reviewer1
2	Doctor shall be able to verify that they are connected to correct PG_Device		Goal	Provide programmable pacing	1	102	102	UnitTest	102	102		Reviewer2
3	Doctor shall be able to select the desired pacing operation mode	Pacing options are AAF, VV	Goal	Provide programmable pacing	1	103	103	UnitTest	103	103		Reviewer3

Figure 5.22: Showing how traceability is automatically maintained throughout requirement approval and testing.

the [UCD](#) which are in turn supported by our goal diagrams. We have also identified qualitative and safety requirements that were not identified in the original specification. We have demonstrated flexibility in specifying these requirements using the Gherkin syntax. This has shown the benefit of a semi-formal approach, as we can specify our functional requirements if desired, helping with project planning and scoping. This also shows potential integration points with existing [FDD](#) development methods. The natural language also makes the specification process relatively easy to learn and adopt, while allowing meaningful specifications that support development. Finally, we have shown that all of these efforts automatically create a traceability matrix for us, with no extra steps or inputs required during execution.

As we have a minimum viable product as part of this methodology, we can also compare it against other existing technologies. In summary, FeatureIDE is typically an implementation-focused feature-modelling tool. GEARS is a design-focused feature modelling technology that emphasizes capturing variation points in design. DOORS is a requirement database technology that supports manual creation of traceability links and can generate graphical representations of those links. The tool for CyclicL differs from these tools in that it integrates requirement-modelling with feature-modelling in a way none of the previously mentioned technologies currently do. Compared to DOORS, the traceability links are a first-class citizen of CyclicL, while in DOORS they are optional. Unlike GEARS and FeatureIDE, CyclicL attempts to capture requirements variation before moving to design or implementation details. It tries to capture changes in rationale that could lead to variations in how to solve a problem for a customer, which ultimately result in different feature and requirement variants. Overall, CyclicL and its implementation fill in a niche

in development that does not conflict with other existing technologies.

5.5 Iterative and Incremental Change

For the purposes of this methodology, iterative changes are understood as cycles between development phases (features and requirements, requirements and design, and design and implementation). The idea behind this definition is that lessons learned during a subsequent development phase can inform and justify repeating a development task from a previous phase. In this methodology, if a need or gap is identified during the requirements phase, it may necessitate repeating a process in the feature modelling phase. A need or gap between the requirement modelling and feature modelling phases could be that some requirements do not fit into any features. It may require a review to ensure that the identified features are complete and correct.

Incremental changes are changes within a single development phase. We used this definition to limit the scope of what constitutes an incremental change. This was to support our development efforts for tool implementation and to simplify requirements for incremental changes compared to iterative changes. Currently, our tool implementation of CyclicL supports maintaining traceability matrices through incremental changes as defined. As engineers work through the steps of a development phase, they will continually add, modify, or remove artifacts until they reach a satisfactory result. We consider all of these incremental changes until they get enough completed to move on to the next phase of development.

Our implementation supports three traceability matrices for CyclicL: project traceability, feature-feature traceability, and feature-requirement traceability.

	Requirement ID	Requirement Description	Requirement Rationale	Requirement Specificati...	Feature
◆ Doctor shall be able to access PG_Device via DCM	1			Given{	Provide programmable pacing
◆ Doctor shall be able to verify that they are connected to correct PG_Device	2			Given{	Provide programmable pacing
◆ Doctor shall be able to select the desired pacing operation mode	3	Pacing options are: AAT, VV		Given{	Provide programmable pacing
◆ Doctor shall be able to set the various parameters based on the chosen pacing operation mode	4	This refers to the pace puls...			Provide programmable pacing
◆ Doctor shall be able to push programmed mode and parameters to the PG_Device	5				Provide programmable pacing
◆ Doctor shall be able to disconnect DCM from PG_Device	6				Provide programmable pacing
◆ Doctor shall be able to log off from DCM	7				Provide programmable pacing
◆ Doctor connects to the wrong PG_Device via DCM	8				Provide programmable pacing
◆ Communication fails or is interrupted during push from DCM to PG_Device	10				Provide programmable pacing
◆ Doctor is unable to access the PG_Device via DCM	11				Provide programmable pacing
◆ Doctor makes a mistake programming the PG_Device via DCM	9				Provide programmable pacing

Figure 5.23: Feature-requirement traceability during the middle of development for a pacemaker first half.

Design Element	Test Case ID	Test Case Description	Test Case Type	Test Case Passed	Approval	Comments	Reviewer
□	[1]	□	[UnitTest]	[true]	[true]	□	[Reviewer1]
□	[2]	□	[UnitTest]	[false]	[true]	□	[Reviewer2]
□	[3]	□	[UnitTest]	[false]	[false]	□	[Reviewer3]
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□
□	□	□	□	□	□	□	□

Figure 5.24: Feature-requirement traceability during the middle of development for a pacemaker second half.

Project traceability and feature-requirement traceability are defined to focus on traceability of the requirement entities. Project traceability is the matrix that tracks the entire project. Feature-requirement traceability is a traceability matrix that is scoped to a single feature. The Feature-feature traceability matrix represents the feature model that is defined within the project.

These matrices are a second syntactic representation of the modelling environments present within the tool implementation. Since they are semantically equivalent, any change to the requirements in the requirement canvas or to the features in the feature modelling environment is automatically reflected in their respective traceability matrices. They are semantically equivalent because the traceability matrices are based on the same metamodel that is used to specify the modelling environments. As a result, the matrices are a complete reflection of the modelling environment. The first column of the matrix is type class, while everything within a row is directly traceable to the element of the type class in the first element of the row. For example, in figure 5.23, the first column is of type **Requirement**, and all the elements within the same row are found based on the traceable relationships that are defined for the **Requirement** class from the metamodel. The relationships are specified for display in the table using queries from AQL [59] on the user-created models. For this reason, for every model element that is added, to a requirement element, it will appear in one or more of the requirement traceability matrices.

Due to the shared source of the traceability matrices and the modelling environments, they are tightly coupled. Therefore, correctness of the matrices is completely bound to the correctness of the models created. If the relationships created between elements in the modelling environments are incorrect, then the traceability matrices will also reflect incorrect traces between ele-

	Mandatory Features	Optional Features	Alternative Compositions	Builds up to	Optionally used by	Alternatively builds up to
◆ PG_Device	[Provide historical data, Pro] []		[]			
◆ Provide historical data	[Sensor output data, Atrial] []		[]	PG_Device		
◆ Provide diagnostic features	[Real time telemetry marker] []		[]	PG_Device		
◆ Sensor output data	[]	[]	[]	Provide historical data		
◆ Monitor and regulate patient heart rate	[]	[]	[]	PG_Device		
◆ Detect and provide therapy for bradycardia conditions	[]	[]	[]	PG_Device		
◆ Provide programmable pacing	[Permanent, Temporary, Ra] []		[]	PG_Device		
◆ Atrial and ventricular rate histogram	[]		[]	Provide historical data		
◆ Real time telemetry markers	[]	[]	[]	Provide diagnostic features		
◆ EGMs	[]	[]	[]	Provide diagnostic features		
◆ P and R wave measurements	[]	[]	[]	Provide diagnostic features		
◆ Lead impedance	[]	[]	[]	Provide diagnostic features		
◆ Battery status tests	[]	[]	[]	Provide diagnostic features		
◆ Permanent	[]	[]	[]	Provide programmable pac		
◆ Temporary	[]	[]	[]	Provide programmable pac		
◆ Rate-adaptive	[Measure physical activity] c []		[]	Provide programmable pac		
◆ Measure physical activity of patient	[]	[]	[]	Rate-adaptive		
◆ Dual-chamber	[]	[]	[]	Provide programmable pac		
◆ Single-chamber	[]	[]	[]	Provide programmable pac		
◆ Single-chamber rate-adaptive	[]	[]	[]	Rate-adaptive		
◆ Dual-chamber rate-adaptive	[]	[]	[]	Rate-adaptive		

Figure 5.25: Feature-feature traceability during the middle of development for a pacemaker.

ments. This reinforces one of the objectives of the CyclicL methodology which is that good engineering practice will lead to good traceability as a natural by-product. Further, by having two different viewpoints of the same elements and relationships, the tool provides different ways for the stakeholders to interact with the features and requirements to allow for better refinement of both over incremental changes.

The tabular environments can also define new requirements. The limitations are that test cases and reviewers cannot be created in the tabular environment. This is by design, as it forces users to manually create those traces explicitly in the modelling environment and reflect on what test cases are being created and who is approving the requirements.

5.6 Discussion

We currently have limited support for the defined iterative changes in the tool for CyclicL. While changes to an existing model are automatically reflected in the traceability matrices, their impact cannot be assessed in the tool at this time. Based on the metamodel for CyclicL, however, the mechanisms to support change impact analysis are all present. This is due to the model's inherent traceability and to the requirement that explicit connections between elements be present for a model to be considered syntactically valid under our current conventions. A more in-depth change-impact analysis that performs both syntactic and semantic analysis would be desirable to inspire greater confidence in the results.

The CyclicL implementation is also currently only scoped to feature and requirement modelling. It does not currently support [UCDs](#) or goal diagrams. As

a result, we do not automatically generate and maintain traceability to those phases of development. While this does pose a challenge to our goal of end-to-end traceability, we wanted to focus on the feature-requirement traceability and build more confidence in that portion, given the novelty of the feature-requirement encapsulation, before adding the goal-to-use case-to-feature-to-requirement traceability, as there is less novelty in that specification. There was also no other obvious connection point between goals, use cases, and features, as we found we could use features to encapsulate requirements. This lack of completion also hampers our efforts to support iterative changes across the methodology's steps and development phases. This also represents a limitation of the CyclicL methodology. Because we do not connect goal diagrams and use case diagrams with our other modelling specifications, we lose another benefit of this methodology: traceability as a by-product of good engineering practice. This is explored in more depth in the future work chapter, as we aim to determine how to effectively integrate these four modelling strategies into a single specification. By having all the modelling environments connected within CyclicL, we will have greater confidence in tracing iterative changes and assessing change impact more comprehensively across the various stages of development.

As part of this evaluation, we demonstrated how following the CyclicL methodology led to improvements of the requirements on top of identifying the features and generating traceability. While not one of the main contributions of this thesis, it is important to note the bias present within this evaluation given our own intimate knowledge of all aspects of the CyclicL methodology. Result may vary when this methodology is used by a broader scope of people. This is however somewhat mitigated by the incremental and iterative nature

of this approach as the expectation is for the stakeholders to learn more about the processes and the product they are developing throughout the development lifecycle and refine all the engineering artifacts produced by CyclicL over time. CyclicL has demonstrated the potential to improve the feature modelling and requirement engineering stages of development by combining existing [MDE](#) and [PLE](#) techniques, but it is still highly dependent on the qualifications of the people executing the steps of the methodology.

Overall, CyclicL shows great promise for addressing gaps in current requirements engineering practices. The methodology provides strong support for traceability across the various stages of execution. Each step naturally flows into the next, and every subsequent step can be directly supported and traced to preceding work. Currently, there is a gap between the goal diagram, the [UCD](#), and the feature and requirement modelling environment due to a scope limitation in the initial specification. Despite this current limitation, we have shown the potential for traceability between the various modelling environments. The current challenge is finding the optimal mechanisms for connecting the various modelling environments.

Further, once all the modelling environments can be traced to each other mechanically, this will advance the goal of a more comprehensive change-impact analysis. It will also enable end-to-end traceability to support argumentation and justification of the specified product's safety and security, enhancing safety and security assurance. In this current iteration of CyclicL, we have demonstrated how to automate traceability between features and requirements. We have also justified our various classifications of requirements, in line with previous work, while further refining them to help with specific development goals for the product. We have shown how to flow from domain

analysis through to requirement elicitation and how to justify each decision along the way.

Chapter 6

Future Work

While the results of our proposed methodology and tooling are promising, there are limitations. We needed constraints to manage the scope of work for a PhD thesis and for technical reasons. Although the CyclicL tool is self-contained in terms of features and requirements, there is currently no modelling environment for the [UCD](#) and goal diagram explained in the methodology. As a result, there is no supported traceability between the [UCD](#) and the goal diagram, the feature model, or the requirement canvases. This makes comprehensive traceability from requirements through features to their sources difficult. It also leaves a gap in iterative change analysis, because there are currently no facilities for change-impact analysis between feature modelling and the [UCD](#). As users learn more about their product, we anticipate and encourage them to iterate through the various phases of development in this methodology, and we want to support that computationally as well.

Using this methodology to support the development of safety and security assurance cases is another avenue we would like to explore. Because the methodology emphasizes traceability, it may support the development of assur-

ance cases. This possibility partly explains the design decision to include the **Safety** type of requirement in the CyclicL metamodel. It would include traceability to the eventual modelling environments for UCDs and goal diagrams in CyclicL.

Another interesting avenue of research for the PACEMAKER requirements was explored by Wang et al. [60], which examines the assumptions made in the PACEMAKER requirements document [58]. A potential avenue for future work would be to integrate the analysis process for identifying assumptions in system requirements with the CyclicL methodology, enabling traceability to those assumptions. This would help ensure that assumptions made during development are identified, documented, and traceable throughout the product lifecycle.

The implementation of the tool for CyclicL is still incomplete, as we have only demonstrated the minimum viable product’s capabilities. We still need to add many functions to make the tool more complete. There are currently no constraints in the tool to ensure that all user-created models are valid according to the metamodel. While the tool offers flexibility, there is room for stronger validation to provide more robust analysis support for users creating models. It is, after all, a trade-off between flexibility and error checking.

We are currently relying on conventions and good engineering to create semantically meaningful models, but maintaining them at scale within a company or industry is very difficult. Another avenue of development would be to allow users to create these models both textually and graphically. Very often, adoptability of a tool, especially an MDE tool, is difficult because many engineers are unfamiliar with MDE techniques and approaches. Creating a textual environment for these models may improve the usability and approachability

of CyclicL for industry use. Next, we would want to implement consistency checking within the tool. This would remove much of the burden from the engineer to ensure that their features and requirements are consistent across features. This is a unique challenge, as the requirements are specified using the Gherkin behavioural approach and may require us to re-evaluate the specification language we implement within the requirement canvases of CyclicL.

Other future work tasks mentioned throughout this thesis include integration with existing test compilers, integration with previous feature modelling tools, and general UI enhancements. Because there are many Gherkin test environments, building an integration bridge with CyclicL would help with verification tracking and traceability. Integration with another feature modelling tool, such as FeatureIDE, would provide a basis for exploring the impact of the feature-requirement hierarchy defined in this thesis on existing feature weaving and composition techniques. Finally, as mentioned in the evaluation section, industry and user validation, scalability testing, and quantitative and qualitative comparisons against existing technologies remain as future work for continued analysis of CyclicL as both a methodology and a tool.

Chapter 7

Conclusion

What comes first, the requirement or the feature? In this thesis, we have attempted to outline an answer to this question. Incidentally, and perhaps unsurprisingly, the answer is both, and it depends. Both realities are possibilities for companies in industry as they start new product development from scratch or try to reuse previous work for a new model. So we support both of them.

The ultimate answer, however, is neither. The stakeholder use cases and goals come first as we conduct a domain analysis and problem space exploration. We have shown that, by following Meyer’s requirements engineering approach, we can leverage both informal and formal [MDE](#) techniques to perform domain and problem space analysis. We found that goal diagrams were an easy-to-use, informal approach to capturing knowledge of what stakeholders really want to do, which helped inform why they would want to use the product we developed. The [UCD](#) was important for establishing how they would use our product. Identifying what interfaces the stakeholders would need from the product to accomplish their goals. Goal diagrams and [UCDs](#) formed the core of our informal modelling techniques for domain analysis and

problem space exploration.

Feature modelling and our requirement canvas leveraged more formal approaches to modelling and problem space exploration; with `tool`, we could enforce syntactic and semantic rules when creating the models. Equating features and use cases also helped to establish requirements for our features as we refined the use cases into user stories. With our proposed feature-requirement hierarchy, those requirements are immediately scoped and owned by the features, making an easy transition from problem space exploration to requirement specification. We have demonstrated both formal and informal approaches to domain analysis, problem space exploration, and requirement development while using [MDE](#) techniques. We have explored several processes for accomplishing these tasks in multiple environments. Finally, we have demonstrated how this methodology can be used to identify gaps in existing documentation and how to improve requirements specification and identification. Thus, we conclude that we have satisfied [RQ.4](#)

[FDD](#) is a well-established development strategy. A major contribution of this thesis is the proposed feature-requirement hierarchy, which establishes that features own their requirements. Another contribution is our equating of use cases from [UCDs](#) to features in [PLE](#). Our methodology refines use cases into user stories to define requirements for each. Because the use cases and features are aligned, those requirements also apply to the feature. When combined with the proposed feature-requirement hierarchy, we end up with well-defined requirements for each feature. An added bonus we discovered was a natural ordering that emerged in the requirements, driven by the use of user stories as a refinement process. These all contribute to well-defined, scoped features and potentially aid project planning by providing an implied ordering

of requirements from the user stories. These are all possible ways to enhance [FDD](#) development environments, satisfying [RQ.1](#)

One of the main contributions of our methodology is the encapsulation of feature requirements. By defining this hierarchy, we have shown how it facilitates traceability between features and requirements. We also removed ambiguity in the relationship between features and requirements. We have demonstrated how this hierarchy enables greater traceability granularity through our implementation of *CyclicL*. We were able to define a formal metamodel to capture this hierarchy and implement a tool to show satisfiability of this proposal. Through *CyclicL*, we demonstrated the benefit of this hierarchical relationship between features and requirements in semi-automated maintenance and generation of traceability matrices. This was completed as a by-product of following the methodology and naturally using *CyclicL* to create models and specify our requirements. We thus got the traceability matrices without any manual intervention or extra steps in the engineering process. We conclude that this reduces much of the time and effort that would traditionally have been spent manually creating or maintaining engineering data traceability, satisfying [RQ.2](#).

Finally, *CyclicL* has demonstrated the potential benefits of a self-contained [PLE](#) tool that embeds requirements, in the context of iterative and incremental development. We have shown that *CyclicL* can support incremental development of both feature and requirement models. By leveraging [Eclipse Modeling Framework \(EMF\)](#) and *Sirius*, we demonstrated the possibilities enabled by a tool that maintains traceability across both requirement and product changes, without reliance on external support. While the current tool still has limitations in supporting iterative changes, we have demonstrated how future

development can continue to address these shortcomings. Due to the current limitations in the tool, we conclude that we have partially satisfied [RQ.3](#) with CyclicL as a tool that supports iterative and incremental traceability development in parallel with feature and requirement development. We give it a partial designation as there are still limitations on how CyclicL can support iterative changes under our current definition of a change between development phases.

Overall, our methodology has demonstrated improvements in an existing requirement document, identified avenues for potential improvements in requirement engineering and feature modelling, and shown potential for integration with existing development practices. Our contributions include the following:

- The integration of automated traceability with domain space exploration, feature modelling, and requirement engineering.
- The definition of a clear hierarchy between features and requirements.
- A tool supported approach enabling incremental and iterative development of feature models and requirements.
- A unified workflow between various [MDE](#) and [PLE](#) techniques for feature modelling and requirement engineering.

CyclicL and its supporting methodology are ripe for further exploration and continued development to support safety-critical development in further domains and potentially as a more generalized approach to requirement engineering across industries.

Appendices

Appendix A

A.1 Full Tool Source Code

<https://github.com/t-chiang/CyclicL>

A.2 Boston Scientific Pacemaker Requirements

PACEMAKER System Specification

Copyright ©2007 Boston Scientific

January 3, 2007

DISCLAIMER: This document is provided for academic purposes only, specifically demonstration of formal methodologies and engineering education.

Boston Scientific disclaims any use of information contained herein for any product whatsoever, disclaims safety, disclaims efficacy, and disclaims fitness for purpose. Any person or entity who uses this document for anything assumes complete responsibility for its use.

Permission to copy is granted, provided the copyright declaration and this disclaimer are included.

Please direct any comments or questions to: **PACEMAKER@sqr1.mcmaster.ca**

Contents

1	Introduction	5
1.1	Scope	5
1.2	Acronyms	5
2	System Definition	7
2.1	System Overview	7
2.2	System Components	7
2.2.1	Device Overview	8
2.2.2	Device Controller-Monitor (DCM) Overview	8
2.2.3	Lead System Overview	9
2.3	Indications and Contraindications	9
2.4	User Characteristics and Operational Environment	10
2.4.1	Hospitals/Physician Users	10
2.4.2	Device Usage with Other Equipment	10
2.5	System Operational Life Phases	10
2.5.1	Pre-Implant Phase	11
2.5.2	Implant	11
2.5.3	Predischarge Follow-up	12
2.5.4	Routine Follow-up	12
2.5.5	Ambulatory	12
2.5.6	Explant	12
2.5.7	Disposal	12
3	System Requirements	13
3.1	Model Type	13
3.2	Device Controller-Monitor (DCM)	13
3.2.1	DCM User Languages	13
3.2.2	DCM User Interface	13
3.2.3	DCM Utility Functions	14
3.2.4	Printed Reports	14
3.2.5	Strip Chart Recording Support	15
3.2.6	DCM-PG Telemetry	15
3.3	Lead Support	15
3.4	Pacing Pulse	16
3.4.1	Pulse Amplitude	16
3.4.2	Pulse Width	16
3.4.3	Rate Sensing	16
3.4.4	Sensitivity Adjustment	16
3.5	Bradycardia Operating Modes	16
3.5.1	No Response To Sensing (O)	17
3.5.2	Triggered Response To Sensing (T)	17
3.5.3	Inhibited Response To Sensing (I)	17
3.5.4	Tracked Response To Sensing (D)	17
3.6	Bradycardia States	17

3.6.1	Permanent State	17
3.6.2	Temporary Bradycardia Pacing	17
3.6.3	Pace-Now State	18
3.6.4	Magnet State	18
3.6.5	Power-On Reset (POR) State	18
3.7	Magnet Test	19
3.8	Implant Data	19
4	Diagnostics	21
4.1	Measured Data	21
4.2	P and R Wave Measurements	21
4.3	Lead Impedance Measurement	21
4.4	Battery Status	21
4.5	Threshold Test	22
4.6	Bradycardia History	22
4.6.1	Rate Histograms	22
4.6.2	Rate Trending	23
4.6.3	Recording Duration and Time Stamp	24
4.6.4	Sensor Trending	24
4.7	Real-time Electrograms	24
4.7.1	Electrogram Viewing	24
4.8	Real-time Electrogram Event Marker Annotations	25
4.8.1	Atrial Markers: AS AP AT TN	26
4.8.2	Ventricular Markers: VS VP PVC TN	26
4.8.3	Marker Modifiers: () -Hy -Sr $\uparrow\downarrow$	26
4.8.4	Augmentation Markers: ATR-Dur ATR-FB ATR-End PVP $\rightarrow$	26
4.9	Faults and Error Handling	27
4.9.1	Faults	27
4.9.2	Errors	27
5	Bradycardia Therapy	28
5.1	Lower Rate Limit (LRL)	28
5.2	Upper Rate Limit (URL)	29
5.3	Atrial-Ventricular (AV) Delay	29
5.3.1	Paced AV Delay	29
5.3.2	Sensed AV Delay	29
5.3.3	Dynamic AV Delay	29
5.3.4	Sensed AV Delay Offset	30
5.4	Refractory Periods	30
5.4.1	Ventricular Refractory Period (VRP)	30
5.4.2	Atrial Refractory Period (ARP)	30
5.4.3	Post Ventricular Atrial Refractory Period (PVARP)	30
5.4.4	Extended PVARP	30
5.4.5	Refractory During AV Interval	31
5.5	Noise Response	31
5.6	Atrial Tachycardia Response (ATR)	31

5.6.1	Atrial Tachycardia Detection	31
5.6.2	ATR Duration	31
5.6.3	ATR Fallback	32
5.7	Rate-Adaptive Pacing	32
5.7.1	Maximum Sensor Rate (MSR)	32
5.7.2	Activity Threshold	32
5.7.3	Response Factor	32
5.7.4	Reaction Time	33
5.7.5	Recovery Time	33
5.8	Hysteresis Pacing	33
5.9	Rate Smoothing	33
A	Programmable Parameters	34
B	Measured Parameters	35

List of Tables

1	Model Type and Lead Port	13
2	Bradycardia Operating Modes	17
3	Battery Status and Therapy Availability	22
4	Intervals Associated with Histogrammed Events	23
5	Event Marker Annotations	25
6	Programmable Parameters for Bradycardia Therapy Modes	28
7	Programmable Parameters	34
8	Measured Parameters	35

1 Introduction

This System Specification for PACEMAKER defines the functions and operating characteristics, identifies the system environmental performance parameters, and characterizes anticipated uses of the system.

1.1 Scope

This document identifies the functions that the system must perform and provides a description of these functions and their primary interactions.

1.2 Acronyms

AP	Atrial Pace
AS	Atrial Sense
ARP	Atrial Refractory Period
ATR	Atrial Tachycardia Response
AV	Atrial-to-Ventricular
BOL	Beginning Of (battery) Life
BPM	Beats Per Minute
cc	Cardiac Cycle(s)
CCI	Cardiac Cycle Interval
DCM	Device Controller-Monitor
ECG	Electrocardiogram, external heart signals
EGM	Electrogram, internal heart signals
EOL	End Of (battery) Life
EP	Electrophysiology, electrophysiologist
ERN	Elective Replacement Near
ERT	Elective Replacement Time
HRL	Hysteresis Rate Limit
ICD	Implantable Cardio-Defibrillator
IS-1	Industry Standard lead type 1
LRL	Lower Rate Limit
MSR	Maximum Sensor Rate
NSR	Normal Sinus Rhythm
OR	Operating Room
PG	Pulse Generator
POR	Power-On Reset
PMT	Pacemaker-Mediated Tachycardia
ppm	Pulses Per Minute
PVARP	Post-Ventricular Atrial Refractory Period
PVC	Premature Ventricular Contraction
SIR	Sensor Indicated Rate
SRD	Sustained Rate Duration
URL	Upper Rate Limit
VP	Ventricular Pace

VS	Ventricular Sense
VRP	Ventricular Refractory Period

2 System Definition

2.1 System Overview

This PACEMAKER System Specification describes the PACEMAKER-specific programming application and pulse generator (PG). The PACEMAKER system supports the following needs of patients that require bradycardia pacing support:

- Implantation
- Ambulatory
- Follow-up
- Explantation

The PACEMAKER system:

- Provides dual chamber, rate adaptive bradycardia pacing support
- Provides historical data on device performance
- Provides user diagnostics through brady analysis functions

The bradycardia analysis functions permit the following pacing measurements and tests to be performed:

- Lead impedance
- Pacing threshold
- P and R wave measurement
- Battery status
- Temporary brady pacing
- Motion sensor trending

2.2 System Components

The PACEMAKER system consists of three major components:

- Device (also called the pulse generator or PG)
- Device Controller-Monitor (DCM) and associated software
- Leads

A standard cardiac “donut” magnet is a minor system component.

2.2.1 Device Overview

The device monitors and regulates a patient's heart rate.

The device detects and provides therapy for bradycardia conditions.

The device provides programmable, single- and dual-chamber, rate-adaptive pacing, both permanent and temporary.

In adaptive rate modes, an accelerometer is used to measure physical activity resulting in a sensor indicated rate for pacing the heart.

The device is programmed and interrogated via bi-directional telemetry from the Device Controller-Monitor (DCM). This allows the physician to change the operating mode or parameters of the device non-invasively after implantation.

The device provides the following history data:

- Sensor output data
- Atrial and ventricular rate histograms.

The device, in conjunction with the DCM, provides diagnostic features including:

- Real time telemetry markers
- EGMs
- P and R wave measurements
- Lead impedance
- Battery status tests

2.2.2 Device Controller-Monitor (DCM) Overview

The Device Controller-Monitor (DCM) is the primary implant, pre-discharge EP support, and follow-up device for the PACEMAKER system. The DCM is capable of being used both in the OR, physician's office, and the EP lab. The DCM communicates with the PG using a communication protocol and supporting hardware. The DCM consists of the following:

- A hardware platform
- PACEMAKER application software

The DCM has the following features:

1. program and interrogate a PACEMAKER
2. command delivery of "Pace-Now" pace
3. acquire and show diagnostics (history) and lead signal measurement information.
4. acquire and show sensor history and trending information.

5. show visible and audible indications of communication between the DCM and device, including beeping and LED's for alerting the operator to error conditions.
6. acquire and show multi-channel monitoring including surface electrocardiogram and telemetered signals (e.g. EGM, annotated event markers)
7. print reports and strip charts.
8. monitor battery status.
9. output to external strip-chart recorders.
10. receive cursor positioning and button inputs.
11. manage windows for display of text and graphics.
12. set the date and time.

2.2.3 Lead System Overview

The lead system implanted in the patient allows the device to sense intrinsic activity of the heart's electrical signals and delivers pacing therapy to the patient's heart.

The leads are connected to the PG via its header. All IS-1 bipolar leads are supported.

2.3 Indications and Contraindications

The PACEMAKER system is indicated for patients exhibiting chronotropic incompetence and who would benefit by increased pacing rates concurrent with physical activity. Generally accepted indications for long-term cardiac pacing include, but are not limited to, sick sinus syndrome; chronic sinus arrhythmias, including sinus bradycardia, sinus arrest, and sinoatrial (S-A) block; second- and third-degree AV block; bradycardia-tachycardia syndrome; bundle branch block; and carotid sinus syndrome.

Patients who demonstrate hemodynamic improvement from atrioventricular synchrony should be considered for one of the dual-chamber or atrial pacing modes. Dual-chamber modes are specifically indicated for treatment of conduction disorders that require restoration of rate and of atrioventricular synchrony, including varying degrees of AV block; low cardiac output or congestive heart failure related to bradycardia; and certain tachyarrhythmias.

Use of the PACEMAKER pulse generator in DDDR and VDDR modes may be contraindicated (1) in patients with chronic atrial tachyarrhythmias (atrial fibrillation or flutter), which may trigger ventricular pacing, or (2) in the presence of slow retrograde conduction that induces pacemaker-mediated tachycardia (PMT) which cannot be controlled by reprogramming selective parameter values. In DDDR, DDIR, and AAIR modes, atrial pacing may be ineffective in the presence of chronic atrial fibrillation or flutter or an atrium that does

not respond to electrical stimulation. In addition, the presence of clinically significant conduction disturbances may contraindicate the use of atrial pacing. Unipolar pacing is contraindicated for patients with an implanted cardioverter defibrillator (ICD) because it may cause unwanted initiation or inhibition of ICD therapy.

2.4 User Characteristics and Operational Environment

Patients with arrhythmias may be users of portions of the system such as the magnet.

2.4.1 Hospitals/Physician Users

The PACEMAKER device and leads are implanted by physicians and hospital staff with varying degrees of experience. Follow-up of the patients is typically performed by nurses or technicians under the supervision of a following physician.

2.4.2 Device Usage with Other Equipment

The device functions with the following equipment:

- The ECG monitors.
- The DCM.

The device meets industry standards for electrical safety. The device functions with the following ancillary equipment present in ORs and EP labs which can be sources of EMI or direct energy interference:

- A fluoroscope.
- Anesthesia machines.
- Patient water blankets.
- Electrocautery devices.
- External defibrillators.
- Blood pressure monitors.
- Pulse oximeters.

2.5 System Operational Life Phases

The system's life cycle is shown below. Each phase in the cycle is described in more detail following.

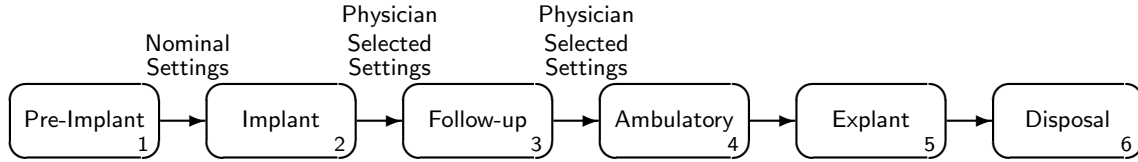


Figure 1: PACEMAKER Life Cycle

2.5.1 Pre-Implant Phase

The system is manufactured using good manufacturing practices as specified by the FDA and other appropriate regulatory bodies. During manufacturing, nominal settings for the PG are set.

2.5.2 Implant

During implant, the PACEMAKER device is placed into the patient. During implant, the DCM is used to:

1. Interrogate the system
2. Review battery status
3. Test the PACEMAKER in the patient
4. Setup the appropriate parameters
5. Program the system before implantation
6. Evaluate ventricular and atrial lead signal amplitudes, impedances, and pacing thresholds.

The procedure to implant a PACEMAKER device and lead system consists of these steps:

1. Checking status of all equipment to be used during implant
2. Implanting the lead system
3. Evaluating lead signals using a pacing stimulus analyzer
4. Programming the nonimplanted system
5. Forming the implantation pocket and tunnel the leads
6. Connecting to the patient leads
7. Testing the system sensing and pacing efficacy
8. Implanting the system

2.5.3 PredischARGE Follow-up

During the predischARGE follow-up test, the following procedures may be performed via telemetry using the DCM:

1. Interrogating the device and obtaining bradycardia sensing and pacing data
2. Reprogramming to final pre-discharge value
3. Printing the follow-up report for the patient's chart

2.5.4 Routine Follow-up

The programming system is capable of performing the following procedures during the routine follow-up:

1. Interrogating the device
2. Checking the battery status
3. Checking the brady status
4. Performing P and R wave measurements
5. Performing pacing threshold and lead impedance tests
6. Reviewing the activity sensor history and rate histograms
7. Printing a follow-up report
8. If parameter values are changed during the follow-up visit, the new setting is verified by viewing the "Session Net Change" report
9. Clearing the Histograms

2.5.5 Ambulatory

The Pacing/Sensing functions will be available in the Ambulatory stage of the device life cycle.

2.5.6 Explant

Once the device is explanted, it is sanitized and returned to its manufacturer. A fault analysis is performed if applicable. The state of the device, when it is explanted, is a function of any fault that may have occurred and/or the state of the battery at the time of explant.

2.5.7 Disposal

For disposal, the device is sent back to its manufacturer. The device should not be destroyed by incineration because the device contains batteries that can explode when subjected to heat.

3 System Requirements

All detailed development requirements are defined in this section and the next, Bradycardia Therapy. Each requirement is defined by a sentence containing the word shall or by each item in a list of items.

For the Device Controller-Monitor (DCM) only those items that will be supported as part of the application software development for the PACEMAKER will be defined.

Programmable ranges and tolerances are provided in Appendix A.

3.1 Model Type

The PACEMAKER model type shall support single and dual chamber rate adaptive pacing.

Model	Pacemaker Designation	Functionality	Connector
DR1	DR	DDDR full function with accelerometer	Dual-in-line, 3.2 mm (IS-1)

Table 1: Model Type and Lead Port

3.2 Device Controller-Monitor (DCM)

3.2.1 DCM User Languages

The application is available in the following languages: English, Danish, Dutch, French, German, Spanish, Italian, and Swedish.

3.2.2 DCM User Interface

The user interface is capable of the following:

1. The user interface shall be capable of utilizing and managing windows for display of text and graphics.
2. The user interface shall be capable of processing user positioning and input buttons.
3. The user interface shall be capable of displaying all programmable parameters for review and modification.
4. The user interface shall be capable of visually indicating when the DCM and the device are communicating.
5. The user interface shall be capable of visually indicating when telemetry is lost due to the device being out of range.

6. The user interface shall be capable of visually indicating when telemetry is lost due to noise.
7. The user interface shall be capable of visually indicating when a different PACEMAKER device is approached than was previously interrogated.

3.2.3 DCM Utility Functions

1. The About function displays the following:
 - Application model number
 - Application software revision number currently in use
 - DCM serial number
 - Institution name
2. The Set Clock function shall set the date and time of the device.
3. The New Patient function shall allow a new device to be interrogated without exiting the software application.
4. The Quit function shall end a telemetry session.

3.2.4 Printed Reports

The following parameter and status reports are available at the user's request:

1. A Bradycardia Parameters Report shall be available.
2. A Temporary Parameters Report shall be available.
3. An Implant Data Report shall be available.
4. A Threshold Test Results Report shall be available.
5. A Measured Data Report shall be available.
6. A Marker Legend Report shall be available.
7. A Session Net Change Report shall be available.
8. A Final Report shall be available. This will consist of the Measured Data, Threshold Test, Trending, Histograms, Implant Data, and Net Change reports.

The following bradycardia diagnostic reports are available at the user's request:

1. A Rate Histogram Report shall be available.
2. A Trending Report shall be available.

Each report shall contain the following header information:

1. Institution name
2. Date and time of report printing
3. Device model and serial number
4. DCM serial number
5. Application model and version number
6. Report name

3.2.5 Strip Chart Recording Support

1. The DCM shall be capable of displaying real time and surface ECG data, which shall be accomplished using the DCM's internal monitor.
2. The system shall be capable of displaying up to three Real-Time traces (2 Telemetered, 1 Surface ECG), along with an annotation for display of event markers, in a scrollable fashion.
3. The DCM shall use the DCM's internal strip chart recorder to provide a means of printing combinations of real time data.
4. The DCM shall be capable of printing real time telemetered data and a surface ECG.
5. The printer shall be capable of simultaneously printing up to three real-time traces, along with full annotation for display of event markers.

3.2.6 DCM-PG Telemetry

The DCM shall either:

- use an inductive telemetry wand to communicate with the pulse generator, maintaining consistent communication over the range of 0 cm to 5 cm between the wand and the pulse generator; or,
- use some other medium, such as RF or ultrasound, that is safe and legal to use, for maintaining consistent telemetry with an implanted medical device.

3.3 Lead Support

1. The Atrial Bipolar Pace/Sensing lead system type shall be supported.
2. The Ventricular Bipolar Pace/Sensing lead system type shall be supported.

3. The system shall operate normally with atrial pace/sense leads between 100 and 2500 ohms.
4. The system shall operate normally with the ventricular pace/sense leads between 100 and 2500 ohms.

Note: all pacing amplitudes and pulse widths in this document are specified using a 750 ohm load.

3.4 Pacing Pulse

The device shall output pulses with programmable voltages and widths (atrial and ventricular) which provide electrical stimulation to the heart for pacing.

3.4.1 Pulse Amplitude

The atrial and ventricular pacing pulse amplitudes shall be independently programmable.

3.4.2 Pulse Width

The atrial and ventricular pacing pulse width shall be independently programmable.

3.4.3 Rate Sensing

Rate sensing shall be accomplished using bipolar electrodes and sensing circuits. All rate detection decisions shall be based on the measured cardiac cycle lengths of the sensed rhythm. Rate shall be evaluated on an interval-by-interval basis.

3.4.4 Sensitivity Adjustment

A means shall be provided for the physician to manually adjust the sensing threshold of the device for both the ventricular and atrial sense channels.

3.5 Bradycardia Operating Modes

The following bradycardia operating modes shall be programmable: Off, DDDR, VDDR, DDIR, DOOR, VOOR, AOOR, VVIR, AAIR, DDD, VDD, DDI, DOO, VOO, AOO, VVI, AAI, VVT and AAT.

OVO, OAO, ODO, and OOO shall be available in temporary operation.

Note: sometimes, “X” is used as a “don’t care” to identify a set of modes; DXXX are the dual-chamber paced modes; OXO are the sensing-only modes; XXT are the triggered modes.

	I	II	III	IV (optional)
Category	Chambers Paced	Chambers Sensed	Response To Sensing	Rate Modulation
Letters	O–None A–Atrium V–Ventricle D–Dual	O–None A–Atrium V–Ventricle D–Dual	O–None T–Triggered I–Inhibited D–Tracked	R–Rate Modulation

Table 2: Bradycardia Operating Modes**3.5.1 No Response To Sensing (O)**

Pacing without sensing is asynchronous pacing. During asynchronous pacing, paces shall be delivered without regard to senses

3.5.2 Triggered Response To Sensing (T)

During triggered pacing, a sense in a chamber shall trigger an immediate pace in that chamber.

3.5.3 Inhibited Response To Sensing (I)

During inhibited pacing, a sense in a chamber shall inhibit a pending pace in that chamber.

3.5.4 Tracked Response To Sensing (D)

During tracked pacing, an atrial sense shall cause a tracked ventricular pace after a programmed AV delay, unless a ventricular sense was detected beforehand.

3.6 Bradycardia States

The following bradycardia states shall be available: Permanent, Temporary, Pace-Now, Magnet, and Power-On Reset (POR). Operating states shall be mutually exclusive.

3.6.1 Permanent State

The permanent pacing state is the normal state of operation of the device. The normal pacing parameters programmed shall be used in the permanent brady state.

3.6.2 Temporary Bradycardia Pacing

The temporary brady pacing state is independent of other pacing functions. The temporary brady parameters programmed shall be used in the temporary

brady state. The temporary state shall be capable of being used to temporarily test various system parameters or provide patient diagnostic testing. Temporary brady pacing shall be terminated by one of the following: breaking the telemetry link, a Pace-Now pace, or a DCM command to the device to cancel temporary pacing.

3.6.3 Pace-Now State

Commanded emergency bradycardia pacing (Pace-Now) shall be available.

The Pace-Now Pace parameter values are as follows:

1. The mode Pace-Now pace parameter shall have a value of VVI.
2. The lower rate limit Pace-Now pace parameter shall have a value of 65 ppm ± 8 ms.
3. The amplitude Pace-Now pace parameter shall have a value of 5.0 V ± 0.5 V.
4. The pulse width Pace-Now pace parameter shall have a value of 1.00 ms ± 0.02 ms.
5. The ventricular refractory Pace-Now pace parameter shall have a value of 320 ms ± 8 ms.
6. The ventricular sensitivity shall have a value of 1.5 mV.
7. The first Pace-Now pacing pulse shall be issued within two cardiac cycles plus 500 ms from the time of the last user action required to activate the Pace-Now state.
8. Once initiated, Pace-Now pacing shall continue until the DCM changes the device pacing mode.

3.6.4 Magnet State

The Magnet State is used during the Magnet Test.

3.6.5 Power-On Reset (POR) State

A Power-on-reset (POR) state shall be entered when the battery voltage drops so low that PG operation is not predictable. All functions shall be disabled until the battery voltage exceeds the POR trip voltage. Above this trip voltage, the PG enters the POR state which is used to power-up the PG system to a known state and set of parameters.

The POR parameter values are as follows:

1. The mode POR pace parameter shall have a value of VVI.

2. The lower rate limit POR pace parameter shall have a value of 65 ppm ± 8 ms.
3. The amplitude POR pace parameter shall have a value of 5.0 V ± 0.5 V.
4. The pulse width POR pace parameter shall have a value of 0.5 ms ± 0.02 ms.
5. The ventricular refractory POR pace parameter shall have a value of 320 ms ± 8 ms.
6. The ventricular sensitivity shall have a value of 1.5 mV.

3.7 Magnet Test

The magnet can be used to determine the battery status of the device. A standard cardiac donut magnet shall be detected by the device at a distance of 2.5 cm between the center of the labeled surface of the device and the surface of the magnet.

When the magnet is in place, the device shall:

1. Pace asynchronously with a fixed pacing rate. The device mode shall be AOO if previous mode was AXXX, VOO if previous mode was VXXX, DOO if previous mode was DXXX, or OOO if previous mode was OXO modes.
2. At BOL the magnet rate shall be 100 ppm. At ERN the magnet rate shall decrease to 90 ppm. At ERT the magnet rate shall decrease further to 85 ppm. During post-ERT operation the rate interval may gradually decrease as the battery voltage continues to decrease.
3. When the magnet is removed the device shall automatically assume pretest operation.
4. The magnet mode shall have the capability to be programmed OFF, so that it will ignore magnet detection.

3.8 Implant Data

The device shall be capable of storing the following information in device memory:

1. The device shall be capable of storing the device model and serial number, and implant date information.
2. The device shall be capable of storing the lead implant date and polarity information.
3. The device shall be capable of storing pacing thresholds and P and R-wave amplitude information.

4. The device shall be capable of storing the pacing lead impedance information.
5. The device shall be capable of storing the patient's indications for pacing.

4 Diagnostics

The system provides the following diagnostic tools:

- Measured Data diagnostic tools shall be provided.
- Threshold Test diagnostic tools shall be provided.
- Rate Trending and Histograms diagnostic tools shall be provided.
- Real-time data diagnostic tools shall be provided.

4.1 Measured Data

Measured Data tolerances are shown in Appendix B.

4.2 P and R Wave Measurements

The device shall allow for DCM-commanded measurement of P and R waves.

4.3 Lead Impedance Measurement

Lead impedance measurements works as follows:

1. The device shall allow for manual measurement capability.
2. DCM commanded lead impedance shall be made with the device in the temporary state.
3. Lead impedance measurements shall be conducted at a default value of 5.5 V.

4.4 Battery Status

Battery status information includes the following:

1. Monitoring voltage information shall be provided.
2. Battery Status indicator information shall be provided.
3. Last interrogation date information shall be provided.
4. The battery status for the device shall be used to predict the following battery status levels:

Battery Status Level	Status	Functionality
Beginning of Life	BOL	Fully functional
Elective Replacement Near	ERN	Fully functional
Elective Replacement Time	ERT	Non-rate-adaptive single chamber modes only. Temporary programming, automatic threshold testing, measured data, electrograms, and event markers disabled.
Elective Replacement Past	ERP	Same as ERT, except pacing rate gradually decreases as battery voltage decreases.

Table 3: Battery Status and Therapy Availability

4.5 Threshold Test

Auto threshold tests work as follows:

1. An automatic pacing threshold test in AAI, VVI, and DDD modes shall be available on command of the DCM for both pulse width and amplitude measurements.
2. The test starts at a user-selectable amplitude or pulse width. After approximately every fourth cardiac cycle, the DCM automatically steps down the amplitude or pulse width one setting.
3. The user is instructed to terminate the test by removing the telemetry wand or selecting the "Stop" button when loss of capture is observed.
4. The last six test results will be displayed (each chamber) on the screen and printed report.
5. Programmable DDD/AAI back-up pacing with a programmable rate shall be available for atrial testing, and programmable DDD/VVI back-up pacing with programmable rate shall be available for ventricular testing.

4.6 Bradycardia History

To assist adjustment of pacing parameters, bradycardia history is retained in PGs for viewing with DCM.

4.6.1 Rate Histograms

The operator interface of the system shall be able to display histograms of pacing rate and intrinsic rate distributions from a histogram recording period.

Histogram data are recorded as follows:

1. Distributions shall be recorded for all paced atrial events.
2. Distributions shall be recorded for all sensed atrial events.
3. Distributions shall be recorded for all paced ventricular events.
4. Distributions shall be recorded for all sensed ventricular events.
5. The number of premature ventricular contractions (PVCs) and atrial tachycardia response episodes shall be recorded and displayed.
6. The recording period for a rate histogram shall be the time since the rate histograms were last reset to the present.
7. The rate histograms shall be resettable (clearing previously recorded data) via telemetry.
8. All rate histograms shall be cleared simultaneously.
9. The intervals associated with the histogrammed events shall begin and end at the events specified in the following table:

Histogram Event	Beginning Event	Ending Event
Sensed Atrial	Refractory or non-refractory atrial sense or atrial pace	Refractory or non-refractory atrial sense
Paced Atrial	Atrial pace or non-refractory atrial sense	Atrial pace
Sense Ventricular	Non-refractory ventricular sense or ventricular pace	Non-refractory ventricular sense
Paced Ventricular	Ventricular pace or non-refractory ventricular sense	Ventricular pace

Table 4: Intervals Associated with Histogrammed Events

4.6.2 Rate Trending

The system shall be configurable to record and display the following data items separately or concurrently over a programmable duration and storage method:

1. Pacing Rate
2. Sensor Data

4.6.3 Recording Duration and Time Stamp

The recording duration shall be programmed to one of the following options:

1. Fixed: Start recording now and stop when available storage is full (time stamped at beginning).
2. Continuous: Circular buffer keeping the latest information (time stamped at end).

The recording duration shall be time stamped as indicated above.

The system shall display only the programmable durations that are applicable for the current pacing mode.

4.6.4 Sensor Trending

The system shall provide off-line prediction analysis of sensor indicated rate with or without intrinsic rate for the synchronized data collected.

4.7 Real-time Electrograms

Real-time internal electrograms shall be made available from

1. The atrial and ventricular sense/pace leads.
2. A surface electrogram.

The real-time electrogram transmission shall be re-initiated if the telemetry link was broken during the transmission of electrograms and then reestablished.

4.7.1 Electrogram Viewing

The user shall have the option of viewing the electrograms

1. On the screen
2. Through a printed copy

The user shall have the option of selecting which electrograms are viewed and the resolution utilized.

Internal electrogram (EGM) options provided are the following:

1. An atrial internal electrogram option shall be provided.
2. A ventricular internal electrogram option shall be provided.
3. An atrial and ventricular internal electrogram option shall be provided.

For the surface electrocardiogram (ECG), the user shall have the capability to select

1. The gain utilized (0.5X, 1X, or 2X)

2. Whether high pass filtering is on

For the internal electrogram (EGM), the display gain shall

1. Be selectable (0.5X, 1X, or 2X)
2. Apply to all channels

4.8 Real-time Electrogram Event Marker Annotations

The capability shall exist to print and display annotated event marker abbreviations listed below on the real-time electrogram. These markers shall be a combination of intrinsic cardiac and device-related events.

1. Each marker shall show time-of-occurrence, accurate to 1 ms, since the most recent, non-refractory event in that chamber.
2. At most one atrial marker will occur each 1 ms.
3. At most one ventricular marker will occur each 1 ms.
4. At most one augmentation marker will occur each 1 ms.

Marker Abbreviation	Description
AS	Atrial Sensed
AP	Atrial Paced
AT	A-Tachy Sense
VS	Ventricular Sensed
VP	Ventricular paced
PVC	Premature Ventricular Contraction
TN	Noise Indication
()	During Refractory
↑	Rate Smoothing Up
↓	Rate Smoothing Down
-Sr	Motion Sensor Rate
-Hy	Hysteresis Rate Pace
ATR-Dur	ATR Onset Started
ATR-FB	ATR Fallback Started
ATR-End	ATR Fallback Ended
PVP→	PVARP Extension

Table 5: Event Marker Annotations

4.8.1 Atrial Markers: AS AP AT TN

Atrial markers are generated for events in the atrium.

AS Atrial Sense not faster than URL

AP Atrial Pace

AT Atrial Tachycardia, sense faster than URL

TN Noise indication

4.8.2 Ventricular Markers: VS VP PVC TN

Ventricular markers are generated for events in the ventricle.

VS Ventricular Sense

VP Ventricular Pace

PVC Premature Ventricular Contraction

A ventricular sense is deemed to be a premature ventricular contraction if there has been no atrial event since the previous ventricular event.

TN Noise indication

4.8.3 Marker Modifiers: () -Hy -Sr ↑ ↓

Atrial or ventricular markers may have modifiers that change their meaning.

() Sense during refractory

-Hy Hysteresis pace

-Sr Sensor-rate pace

↑ Up-rate smoothing pace

↓ Down-rate smoothing pace

4.8.4 Augmentation Markers: ATR-Dur ATR-FB ATR-End PVP→

During atrial tachycardia response (ATR) and PVARP extension, augmentation markers are generated synchronously with atrial or ventricular markers.

ATR-Dur Atrial tachycardia detected, duration started

ATR-FB ATR fallback started when tachycardia lasts for duration

ATR-End ATR ended because tachycardia ceased, or fallback period expired

PVP→ PVC caused PVARP extension

4.9 Faults and Error Handling

4.9.1 Faults

DCM malfunctions shall be indicated.

4.9.2 Errors

Errors indicated are the following:

1. Parameter interactive limit errors shall be indicated.
2. Printer errors shall be indicated.

5 Bradycardia Therapy

User programmable parameters are provided for controlling the delivery of patient-tailored, bradycardia therapy. These parameters are described in the following subsections; which parameters are meaningful with which pacing mode are listed in Table 6, Programmable Parameters for Bradycardia Therapy Modes.

Parameter	A A T	V V T	A O O	A A I	V O O	V V I	V D D	D O O	D D I	D D D	A O R	A A R	V O R	V V I	V D D	D O R	D D I	D D D
Lower Rate Limit	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Upper Rate Limit	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Maximum Sensor Rate											X	X	X	X	X	X	X	X
Fixed AV Delay							X	X	X	X					X	X	X	X
Dynamic AV Delay							X			X					X			X
Sensed AV Delay Offset										X								X
Atrial Amplitude	X		X	X				X	X	X	X	X				X	X	X
Ventricular Amplitude		X			X	X	X	X	X	X			X	X	X	X	X	X
Atrial Pulse Width	X		X	X				X	X	X	X	X				X	X	X
Ventricular Pulse Width		X			X	X	X	X	X	X			X	X	X	X	X	X
Atrial Sensitivity	X			X					X	X		X						X
Ventricular Sensitivity		X				X	X		X	X				X	X		X	X
VRP		X				X	X		X	X				X	X		X	X
ARP	X			X					X	X		X					X	X
PVARP	X			X					X	X		X					X	X
PVARP Extension							X			X					X			X
Hysteresis				X		X				X		X		X				X
Rate Smoothing				X		X	X			X		X		X	X			X
ATR Duration							X			X					X			X
ATR Fallback Mode							X			X					X			X
ATR Fallback Time							X			X					X			X
Activity Threshold											X	X	X	X	X	X	X	X
Reaction Time											X	X	X	X	X	X	X	X
Response Factor											X	X	X	X	X	X	X	X
Recovery Time											X	X	X	X	X	X	X	X

Table 6: Programmable Parameters for Bradycardia Therapy Modes

5.1 Lower Rate Limit (LRL)

The Lower Rate Limit (LRL) is the number of generator pace pulses delivered per minute (atrium or ventricle) in the absence of

- Sensed intrinsic activity.
- Sensor-controlled pacing at a higher rate.

The LRL is affected in the following ways:

1. When Rate Hysteresis is disabled, the LRL shall define the longest allowable pacing interval.
2. In DXX or VXX modes, the LRL interval starts at a ventricular sensed or paced event.
3. In AXX modes, the LRL interval starts at an atrial sensed or paced event.

5.2 Upper Rate Limit (URL)

The Upper Rate Limit (URL) is the maximum rate at which the paced ventricular rate will track sensed atrial events. The URL interval is the minimum time between a ventricular event and the next ventricular pace.

5.3 Atrial-Ventricular (AV) Delay

The AV delay shall be the programmable time period from an atrial event (either intrinsic or paced) to a ventricular pace.

In atrial tracking modes, ventricular pacing shall occur in the absence of a sensed ventricular event within the programmed AV delay when the sensed atrial rate is between the programmed LRL and URL.

AV delay shall either be

1. Fixed (absolute time)
2. Dynamic

5.3.1 Paced AV Delay

A paced AV (PAV) delay shall occur when the AV delay is initiated by an atrial pace.

5.3.2 Sensed AV Delay

A sensed AV (SAV) delay shall occur when the AV delay is initiated by an atrial sense.

5.3.3 Dynamic AV Delay

If dynamic, the AV delay shall be determined individually for each new cardiac cycle based on the duration of previous cardiac cycles. The previous cardiac cycle length is multiplied by a factor stored in device memory to create the dynamic AV delay.

The AV delay shall vary between

1. A programmable maximum paced AV delay
2. A programmable minimum paced AV delay

5.3.4 Sensed AV Delay Offset

The Sensed AV Delay Offset option shall shorten the AV delay following a tracked atrial sense.

Depending on which option is functioning, the sensed AV delay offset shall be applied to the following:

1. The fixed AV delay
2. The dynamic AV delay

5.4 Refractory Periods

To avoid false sensing, refractory periods follow events during which senses in the affected chamber are ignored. To show that a sense was ignored due to refractory, its marker is displayed in parentheses.

5.4.1 Ventricular Refractory Period (VRP)

The Ventricular Refractory Period shall be the programmed time interval following a ventricular event during which time ventricular senses shall not inhibit nor trigger pacing.

5.4.2 Atrial Refractory Period (ARP)

For single chamber atrial modes, the Atrial Refractory Period (ARP) shall be the programmed time interval following an atrial event during which time atrial events shall not inhibit nor trigger pacing.

5.4.3 Post Ventricular Atrial Refractory Period (PVARP)

The Post Ventricular Atrial Refractory Period shall be available in modes with ventricular pacing and atrial sensing. The Post Ventricular Atrial Refractory Period shall be the programmable time interval following a ventricular event when an atrial cardiac event shall not 1. Inhibit an atrial pace. 2. Trigger a ventricular pace.

5.4.4 Extended PVARP

The Extended PVARP works as follows:

1. When Extended PVARP is enabled, an occurrence of a premature ventricular contraction (PVC) shall cause the pulse generator to use the Extended PVARP value for the post-ventricular atrial refractory period following the PVC.
2. The PVARP shall always return to its normal programmed value on the subsequent cardiac cycle regardless of PVC and other events. At most one PVARP extension shall occur every two cardiac cycles.

5.4.5 Refractory During AV Interval

The PG shall also be in refractory to atrial senses during the AV interval. In this context, refractory means the pacemaker does not track or inhibit based on the sensed activity.

5.5 Noise Response

In the presence of continuous noise the device response shall be asynchronous pacing.

5.6 Atrial Tachycardia Response (ATR)

The Atrial Tachycardia Response prevents long term pacing of a patient at unacceptably high rates during atrial tachycardia. When Atrial Tachycardia Response is enabled, the pulse generator shall declare an atrial tachycardia if the intrinsic atrial rate exceeds the URL for a sufficient amount of time.

5.6.1 Atrial Tachycardia Detection

The atrial tachycardia (AT) detection algorithm determines onset and cessation of atrial tachycardia.

1. AT onset shall be detected when the intervals between atrial senses are predominately, but not exclusively, faster than URL.
2. AT cessation shall be detected when the intervals between atrial senses are mostly, but not exclusively, faster than URL.
3. The detection period shall be short enough so ATR therapy is not unnecessarily delayed nor continued.
4. The detection period shall be long enough that occasional premature atrial contractions do not cause unnecessary ATR therapy, nor cease necessary ATR therapy upon occasional slow beats.

5.6.2 ATR Duration

ATR Duration works as follows:

1. When atrial tachycardia is detected, the ATR algorithm shall enter an ATR Duration state.
2. When in ATR Duration, the PG shall delay a programmed number of cardiac cycles before entering Fallback.
3. The Duration delay shall be terminated immediately and Fallback shall be avoided if, during the Duration delay, the ATR detection algorithm determines that atrial tachycardia is over.

5.6.3 ATR Fallback

If the atrial tachycardia condition exists after the ATR Duration delay is over, the following shall occur:

1. The PG enters a Fallback state and switches to a VVIR Fallback Mode.
2. The pacing rate is dropped to the lower rate limit. The fallback time is the total time required to drop the rate to the LRL.
3. During Fallback, if the ATR detection algorithm determines that atrial tachycardia is over, the following shall occur:
 - Fallback is terminated immediately
 - The mode is switched back to normal
4. ATR-related mode switches shall always be synchronized to a ventricular paced or sensed event.

5.7 Rate-Adaptive Pacing

The device shall have the ability to adjust the cardiac cycle in response to metabolic need as measured from body motion using an accelerometer.

5.7.1 Maximum Sensor Rate (MSR)

The Maximum Sensor Rate is the maximum pacing rate allowed as a result of sensor control.

The Maximum Sensor Rate shall be

1. Required for rate adaptive modes
2. Independently programmable from the URL

5.7.2 Activity Threshold

The activity threshold is the value the accelerometer sensor output shall exceed before the pacemaker's rate is affected by activity data.

5.7.3 Response Factor

The accelerometer shall determine the pacing rate that occurs at various levels of steady state patient activity.

Based on equivalent patient activity:

1. The highest response factor setting (16) shall allow the greatest incremental change in rate.
2. The lowest response factor setting (1) shall allow a smaller change in rate.

5.7.4 Reaction Time

The accelerometer shall determine the rate of increase of the pacing rate. The reaction time is the time required for an activity to drive the rate from LRL to MSR.

5.7.5 Recovery Time

The accelerometer shall determine the rate of decrease of the pacing rate. The recovery time shall be the time required for the rate to fall from MSR to LRL when activity falls below the activity threshold.

5.8 Hysteresis Pacing

When enabled, hysteresis pacing shall result in a longer period following a sensed event before pacing. This encourages self-pacing during exercise by waiting a little longer to pace after senses, hoping that another sense will inhibit the pace.

To use hysteresis pacing:

1. Hysteresis pacing must be enabled (not Off).
2. The pacing mode must be inhibiting or tracking.
3. The current pacing rate must be faster than the Hysteresis Rate Limit (HRL), which may be slower than the Lower Rate Limit (LRL).
4. When in AAI mode, a single, non-refractory sensed atrial event shall activate hysteresis pacing.
5. When in an inhibiting or tracking mode with ventricular pacing, a single, non-refractory sensed ventricular event shall activate hysteresis pacing.

5.9 Rate Smoothing

Rate Smoothing shall limit the pacing rate change that occurs due to precipitous changes in the intrinsic rate.

Two programmable rate smoothing parameters shall be available to allow the cardiac cycle interval change to be a percentage of the previous cardiac cycle interval:

1. Rate Smoothing Up
2. Rate Smoothing Down

The increase in pacing rate shall not exceed the Rate Smoothing Up percentage.

The decrease in pacing rate shall not exceed the Rate Smoothing Down percentage.

A Programmable Parameters

Parameter	Programmable Values	Increment	Nominal	Tolerance
Mode	Off DDD VDD DDI DOO AOO AAI VOO VVI AAT VVT DDDR VDDR DDIR DOOR AOOR AAIR VOOR VVIR	—	DDD	—
Lower Rate Limit	30-50 ppm 50-90 ppm 90-175 ppm	5 ppm 1 ppm 5 ppm	60 ppm	± 8 ms
Upper Rate Limit	50-175 ppm	5 ppm	120 ppm	± 8 ms
Maximum Sensor Rate	50-175 ppm	5 ppm	120 ppm	± 4 ms
Fixed AV Delay	70-300 ms	10 ms	150 ms	± 8 ms
Dynamic AV Delay	Off, On	—	Off	—
Minimum Dynamic AV Delay	30-100 ms	10 ms	50 ms	
Sensed AV Delay Offset	Off, -10 to -100 ms	-10 ms	Off	± 1 ms
A or V Pulse Amplitude Regulated	Off, 0.5-3.2V 3.5-7.0 V	0.1V 0.5V	3.5V	$\pm 12\%$
A or V Pulse Amplitude Unregulated	Off, 1.25, 2.5, 3.75, 5.0V	—	3.75V	—
A or V Pulse Width	0.05 ms 0.1-1.9 ms	— 0.1 ms	0.4 ms	0.2 ms
A or V Sensitivity	0.25, 0.5, 0.75 1.0-10 mV	— 0.5 mV	A-0.75 mV V-2.5 mV	$\pm 20\%$
Ventricular Refractory Period	150-500 ms	10 ms	320 ms	± 8 ms
Atrial Refractory Period	150-500 ms	10 ms	250 ms	± 8 ms
PVARP	150-500 ms	10 ms	250 ms	± 8 ms
PVARP Extension	Off, 50-400 ms	50 ms	Off	± 8 ms
Hysteresis Rate Limit	Off or same choices as LRL	—	Off	± 8 ms
Rate Smoothing	Off, 3, 6, 9, 12, 15, 18, 21, 25%	—	Off	$\pm 1\%$
ATR Mode	On, Off	—	Off	—
ATR Duration	10 cardiac cycles 20-80 cc 100-2000 cc	— 20 cc 100 cc	20 cc	± 1 cc
ATR Fallback Time	1-5 min	1 min	1 min	± 1 cc
Ventricular Blanking	30-60 ms	10 ms	40 ms	—
Activity Threshold	V-Low, Low, Med-Low, Med, Med-High, High, V-High	—	Med	—
Reaction Time	10-50 sec	10 sec	30 sec	± 3 sec
Response Factor	1-16	1	8	—
Recovery Time	2-16 min	1 min	5 min	± 30 sec

Table 7: Programmable Parameters

B Measured Parameters

Parameter	Increment	Tolerance
P and R wave measurements	0.1 mV	$\pm 5\%$ referred to the connector
Lead Impedance	50 Ω	100 to 500 Ω , $\pm$ greater of 100 Ω or 30% 500 to 2000 Ω , $\pm 25\%$ 2000 to 2500 Ω , $\pm 30\%$
Battery Voltage	0.01 V	$\pm 2\%$

Table 8: Measured Parameters

A.3 What Is a Pacemaker

What is a Pacemaker?

The Cardiac Conduction System and the Artificial Pacemaker

Michael Kehinde
kehindem@mcmaster.ca

September 13, 2020

Contents

Abbreviations	ii
1 Introduction	1
2 The Cardiac Conduction System (CCS)	2
3 Cardiac Conduction System (CCS) Conduction Disorders	6
3.1 Origins of a Conduction Disorder	7
3.2 Types of Arrhythmias	7
3.3 The Role of the Cardiac Pacemaker	8
4 Parts of the Pacemaker System	9
4.1 Pulse Generator	9
4.2 Leads	9
4.3 Device Controller-Monitor (DCM)	11

Abbreviations

AV Atrio-ventricular 2, 7

BPM Beats per Minute 8

CCS Cardiac Conduction System i, 2, 4, 6–9, 14

DCM Device Controller-Monitor i, 11

ECG Electrocardiogram 3, 4

RHR Resting Heart Rate 7, 8

SA Sinoatrial 2, 7

I Introduction

The aim of this document is to give sufficient background and information about the heart and cardiac pacemakers to begin exploring pacemaker concepts and cardiac terminology. For more in-depth information, a standard pacemaker textbook, such as the one listed in the References section, may be consulted.

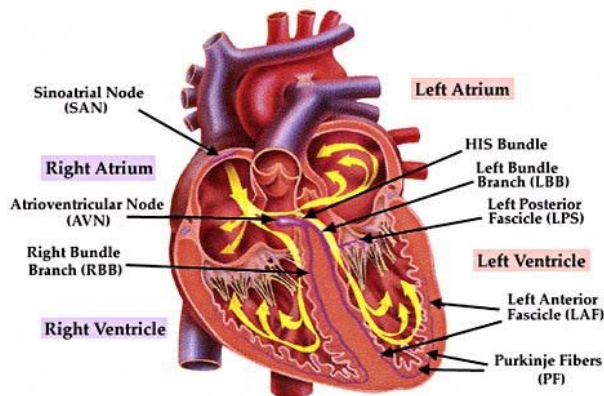
2 The Cardiac Conduction System (CCS)

The heart is a muscular organ that pumps blood throughout the body. The heart has four chambers. The two upper chambers are the left atrium and the right atrium (plural: atria). The two lower chambers are the left ventricle and the right ventricle.

Both the atria and ventricles receive blood while relaxed. The atria contract to move blood to the ventricles. The left atrium receives oxygen-rich blood from the lungs, whereas the right atrium receives oxygen-depleted blood from the rest of the body. The left ventricle contracts to pump oxygen-rich blood throughout the body. The right ventricle contracts to pump oxygen-depleted blood to the lungs for oxygenation.

The Electrical Behaviour of the Heart

The bioelectrical network that makes your heart beat is known as the CCS. The CCS generates and conducts electric stimuli to coordinate the activation and contraction of the atria and ventricles, as is shown in Figure 1b below.



(a) Anatomy of the heart and schematic of the CCS. (b) Animation — the CCS activating the mechanical function of the heart. [IPM06]

<http://www.vascularconcepts.com/img/patients/conductionssystem.gif>

Figure 1: Illustrations of the CCS.

The sequence of events that occurs during a single heart beat is known as the **cardiac cycle**. To begin a cardiac cycle, the Sinoatrial (SA) node generates an electric stimuli that is transmitted quickly to the atria, causing the atria to contract. The signal then propagates through the Atrio-ventricular (AV) node and bundle of His. The AV node

is a tissue that delays and relays the cardiac impulse to the ventricles via the Purkinje fibres. The delay typically lasts between 120-200 milliseconds. The signal spreads quickly through all the ventricular muscle via gap junctions (cell-to-cell connections) causing ventricular contraction. The muscle cells then depolarize in the opposite direction from polarization and the chambers relax to receive blood. During most of the time period that the chamber undergoes depolarization, its muscle cells cannot produce another action potential. This is known as a **refractory period**.

Nodal cells and Purkinje cells have three characteristics that allow the atria and ventricles to be synchronized: [Nek16]

1. *Automaticity*: the ability to initiate an electric impulse
2. *Excitability*: the ability to respond to an electric impulse
3. *Conductivity*: the ability to transmit an electric impulse from one cell to another

Electrocardiogram (ECG) An ECG is a graphical description of the electrical activity of the heart. Electrical activity in the heart can be depicted with the use of **surface electrodes** — conductive pads that are placed on the surface of the skin across the limbs or over the chest to enable recording of electrical currents.

An ECG lead represents the potential difference measured across two electrodes placed on different points of the body. In a 12-lead ECG system, there are three categories of leads: 3 bipolar limb leads, 3 augmented unipolar leads and 6 chest leads. Each lead allows physicians to view the heart from different angles and provides information about various parts of the heart. A plot of voltage over time may be constructed for one or more leads and assessed by the physician. The most commonly used lead is called Lead II and electrodes are placed on the left arm and left leg.

Figure 2 below illustrates an ECG waveform.

The resultant waveform is a summation of various action potentials across the heart. The letters PQRST are used to indicate the waves on the ECG recording or tracing. They are used as references when assessing a patient's ECG.

The P wave represents atrial depolarization and contraction of atria. The PR segment represents the AV nodal delay. The QRS complex represents ventricular depolarization and contraction. The T wave represents ventricular repolarization. The TP interval represents the time in which the ventricles are relaxed and filling.

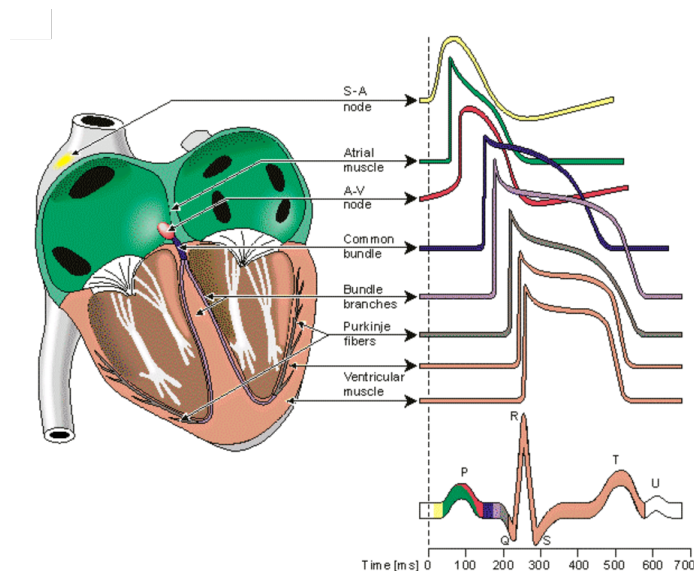


Figure 2: ECG Waveform.[Vah09]

Link between the CCS and Mechanical Events of the Cardiac Cycle

Cardiac electric events are linked with changes in mechanical properties of the heart. Wiggers diagram illustrates the pressure and volume changes in the ventricles over a cardiac cycle. The changes are presented with an ECG and a phonocardiogram, as shown in Figure 3 below.

Ventricular systole is when the ventricles contract and pump blood. Ventricular diastole is a phase of the cardiac cycle wherein the ventricles relax and the heart fills with blood. Ventricular pressure is high during systole and low during diastole.

Heart valves are cusp-like structures that control the direction of the blood flow by preventing backflow. Between the atria and ventricle chambers are the atrioventricular valves — the mitral valve and the tricuspid valve. Between the ventricles and the aorta and pulmonary artery are the aortic and pulmonary valves, respectively. The valves open or close based on the pressure differentials across the chambers.

A phonocardiogram is a graphical representation of the sounds made by the heart. Typically when one listens to the sound of a heartbeat, they are listening to the valves of the heart closing. The oscillations on the phonocardiogram in Figure 3 above represent the characteristic "lub-dub" sound one typically hears when listening to a healthy heartbeat. The first oscillation on the phonocardiogram corresponds to the reverberation of blood from the sudden closure of the mitral valve closing, whereas the second

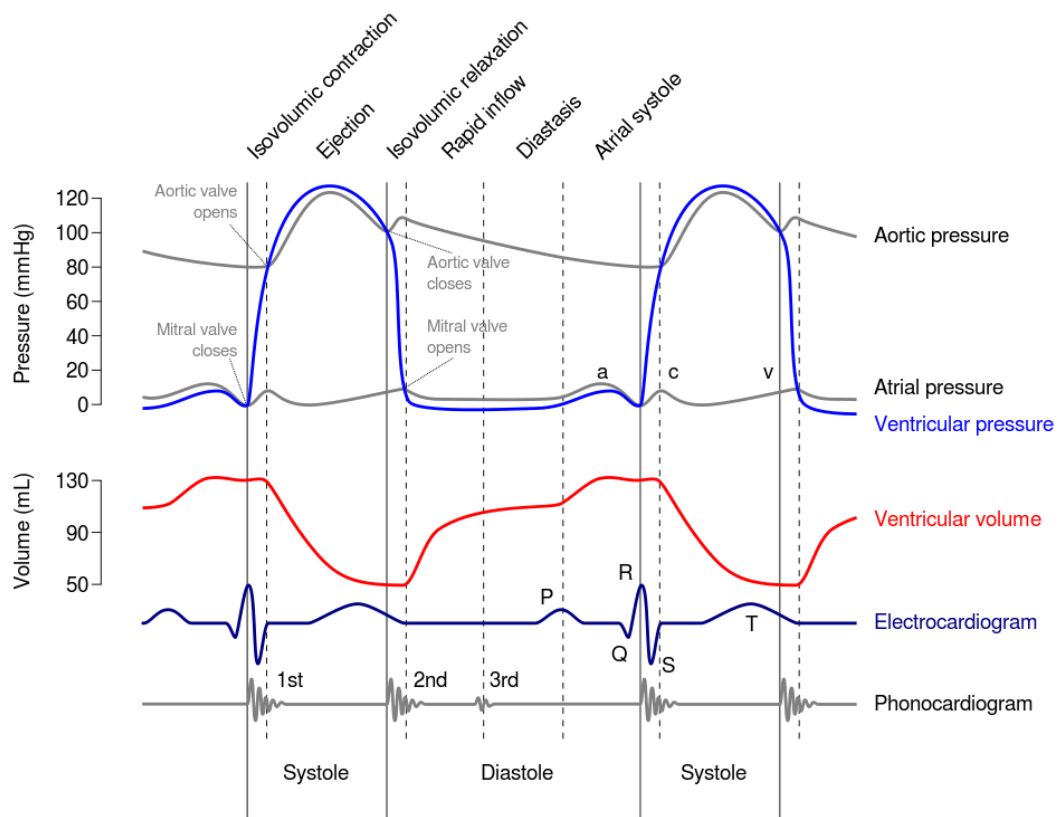
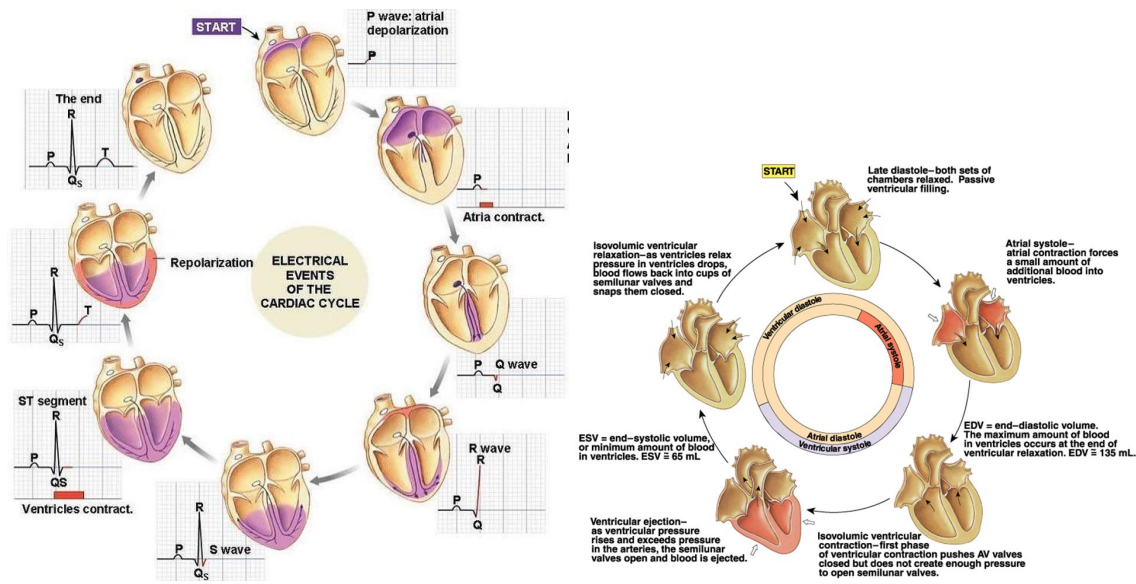


Figure 3: Wiggers diagram.[Dan12]

oscillation corresponds to the reverberation of blood from the sudden closure of aortic valve.



(a) Electrical events of the cardiac cycle. [] (b) Mechanical events of the cardiac cycle. []

Figure 4: Mechanical and electrical events of the cardiac cycle.

3 CCS Conduction Disorders

A conduction disorder is a problem with the CCS that exists when one or both of the following situations occur:

- the CCS signal is not generated properly
- the CCS signal does not travel through the heart properly

Conduction disorders may be attributed to several reasons, including:

- heart disease
- congenital heart defects
- side effects from medications
- damage from cardiac surgery

Conduction disorders are classified based on where they originate and the effect they have on the heart rate (the number of heart beats or contractions in a minute). Classifications of conduction disorders include: atrial fibrillation, atrial flutter, supraventricular tachycardia, Wolff-Parkinson-White syndrome, ventricular tachycardia, ven-

tricular fibrillation, long QT syndrome, sick sinus syndrome, conduction/heart block and premature contractions.[] []

3.1 Origins of a Conduction Disorder

SA Node Malfunction

The SA node is often called the heart's natural pacemaker because it generates the signal that begins a cardiac cycle and establishes the heart rate. Problems with the CCS arise when the SA node malfunctions, causing the heart to beat too slowly, too quickly or erratically. SA node malfunction can also be a cause of **chronotropic incompetence**, a condition where the heart rate does not change sufficiently to meet the metabolic demands of the body during exercise.

Abnormal Firing of Heart Chambers

The cells in the heart's atria or ventricles may fire abnormally and interfere with the signal coming from the SA node (fibrillation and flutter). The atria and ventricles contract out of coordination with each other and the heart rate increases.

Conduction Defects

Conduction defects occur when there is a malfunction in the electrical pathways. In some cases, muscle fibre blockages can impair or prevent the conduction of electric stimuli from the atria to the ventricles — a problem known as AV block or **heart block**. Heart block may be classified as first degree, second degree (Wenckebach) or third degree depending on the level of severity. Heart block typically disturbs the heart rhythm by delaying ventricular contraction.

In some cases, there may be an additional electrical pathway between the atria and ventricles that effectively acts as a short-circuit and causes the CCS signal to travel prematurely or to bounce back towards the chamber that the signal was traveling from (Wolff-Parkinson-White syndrome).

3.2 Types of Arrhythmias

Arrhythmia is a condition wherein the heart beats irregularly or abnormally.

Bradycardia arrhythmia is a condition wherein one has a Resting Heart Rate (RHR) that is abnormally slow. The question of "how slow is abnormally slow?" may differ from individual to individual because the heart's RHR tends to decrease with age and

among individuals with higher fitness levels. For most adults, a slow RHR is less than 50 or 60 Beats per Minute (BPM). Symptoms of bradycardia may include fatigue, dizziness, fainting, heart palpitations and breathing problems.

Tachycardia arrhythmia is a condition wherein one has a RHR that is abnormally fast. For most adults, a fast RHR is above 100 BPM.

Arrhythmia becomes a serious problem when heart does not supply the body with enough oxygen-rich blood. In some cases, conduction disorders can be fatal and also lead to heart attack.

3.3 The Role of the Cardiac Pacemaker

A physician would implant a pacemaker in a patient's body when the cardiac conduction system is not working properly. A cardiac pacemaker (also known as an artificial pacemaker) is a medical device that monitors and regulates a patient's heart rate by stimulating the chambers of the heart synthetically. Most pacemakers are implanted in the body and assist the CCS by sensing the heart's electric signals and generating electrical pulses at specified intervals to precisely determine when the atria and ventricles contract. Pacemakers are often used as a standard treatment for arrhythmia.

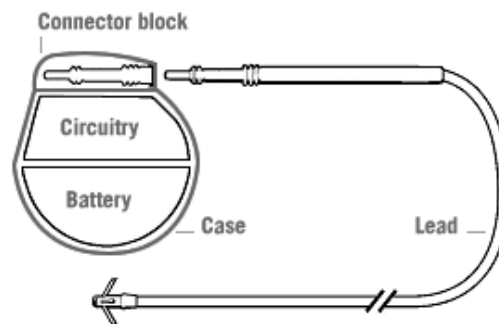
4 Parts of the Pacemaker System

A pacemaker system consists of a pacemaker device (which consists of a pulse generator and one or more pacing leads), and a device controller-monitor.

4.1 Pulse Generator



(a) Pacemaker pulse generator. []



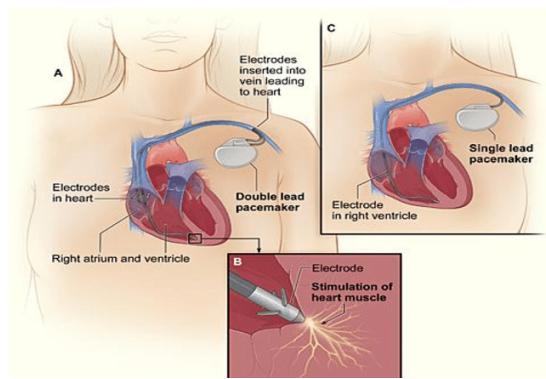
(b) Pacemaker device. []

Figure 5: Pulse generator diagrams.

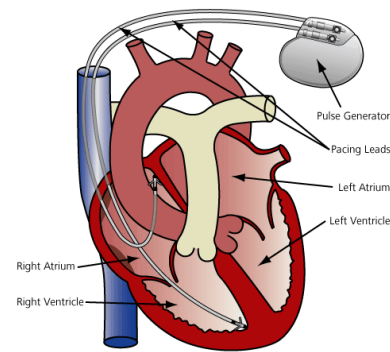
The pulse generator (also known as the “can”) contains the battery of the pacemaker and the circuits that deliver the pacing stimuli. The metal case provides electrical isolation between the internal pacemaker circuit and the CCS. The lead connector block is physical interface for the can and the pacemaker leads. Separate channels are assigned to each heart chamber being paced. Pacemakers usually have a telemetry antenna or magnet built in that allows pacemaker to be re-programmed wirelessly. During a surgical implant, the pulse generator is placed in a pouch underneath the skin below the collarbone.

4.2 Leads

Leads are insulated wires that carry electrical stimuli from the pulse generator to the heart, and carry information about the heart’s natural activity back to the pulse generator. They are the first component of the pacemaker system that is implanted in the



(a) Pacemaker placed in body. []



(b) Pacemaker leads. []

Figure 6: Pacemaker leads placement.

body during surgery and are usually inserted through the subclavian vein. The electrodes on the end of the lead are then attached to the heart. The leads may be poked into heart tissue (if the leads are passive fixated leads) or hooked or screwed into heart tissue (if the leads are active fixated leads). Note that the tip electrode on the active fixation lead in Figure 7 is a small screw-like mechanism designed to screw into the heart tissue at the electrode-myocardium interface. Lead connections are typically tested by taking electrical measurements of the heart, before the pulse generator is connected and inserted under the skin.

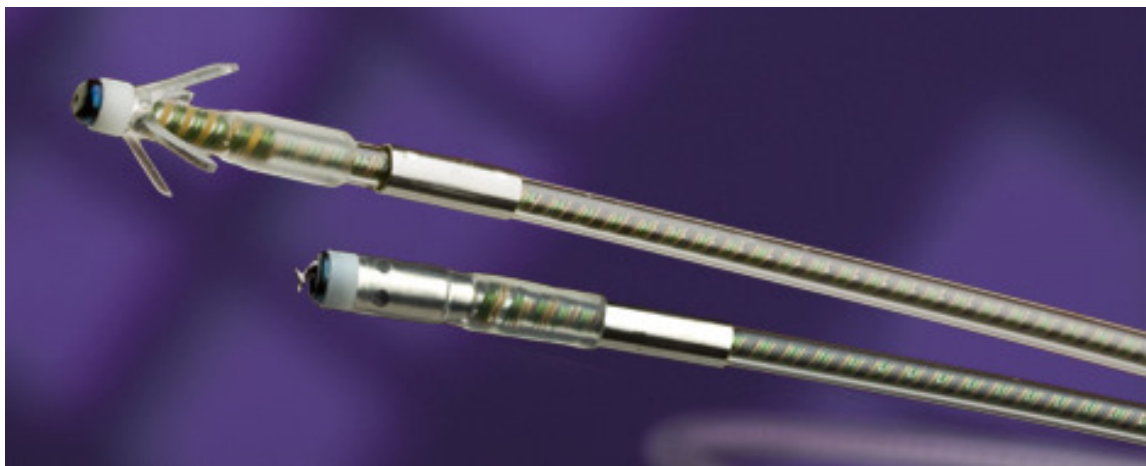


Figure 7: Passive (top) and Active (bottom) Fixation Pacemaker Bipolar Leads.[]

Leads may be bipolar or unipolar. With bipolar leads, the electric stimuli is transmitted

across two electrodes: the ring electrode and the tip electrode. Both electrodes are embedded inside the heart. With unipolar leads, one electrode is embedded inside the heart and the other is the pacemaker can.

Lead Construction Unipolar leads consist of a single helical conductor coil connecting the lead connector to the tip electrode. Bipolar leads have an additional coaxial outer coil connecting the lead connector to the ring electrode and separated from the inner coil with insulation. [CB08]

Pacemakers can have anywhere between 1-3 leads depending which chamber of the heart requires pacing stimuli, and the degree of synchronization that is required between the chambers. Most pacemakers are dual chambered pacing systems, having one lead connected to the right atria and another connected to the right ventricle.

4.3 Device Controller-Monitor (DCM)

The Device Controller-Monitor (DCM) is a device used by the physician to test, interrogate and alter the behaviour of the pacemaker device. The DCM provides a means of remotely transmitting instructions to the pacemaker (**programming**) and receiving information from the pacemaker (**telemetry**). The DCM is used to configure the pacemaker immediately after the pacemaker implant procedure and every 3-6 months during scheduled follow-up appointments with the patient.

The following figure illustrates the concepts of programmability and telemetry.

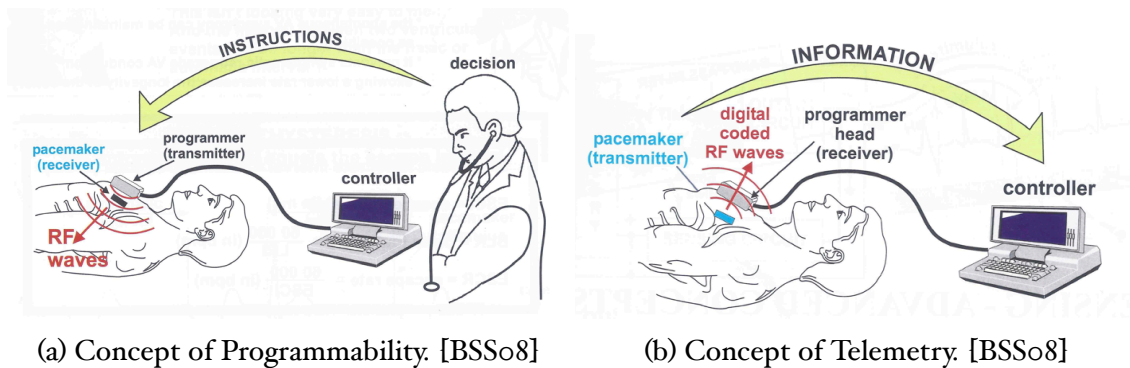


Figure 8: Concepts of Programmability and Telemetry.

Programmability means the system is capable of accepting a new set of instructions that alter its behaviour. Instructions are sent to the pacemaker to modify any of its

programmable parameters, which in turn alter the behavior of the pacemaker. Programmable parameters include, and are not limited to:

- pulse rate (the frequency of the pacing pulse)
- pulse width (the duration of the pacing pulse)
- pulse amplitude (the strength of the pulse)
- sensitivity of the sensing circuit
- mode of pacing
- refractory period
- hysteresis on/off

Telemetry is the process of measuring the readings from the pacemaker remotely. Data obtainable by telemetry include:

- Administrative Data (model, serial number, patient's name, date of implantation, indication for implantation)
- Programmed Data (mode, rate, refractory period, hysteresis on/off, pulse amplitude & width, sensitivity)
- Measured Data (rate, pulse amplitude, pulse current, pulse energy, pulse charge, lead impedance, battery impedance, battery voltage, battery current drain)
- Stored Data (Holter function, rhythm histogram)
- Marker Signals (for ECG interpretation)
- Intracardiac Electrogram (potential difference between the two electrodes used for pacing)

Pacemaker telemetry and programming are both performed without removing the pacemaker, as shown in Figure 8. Physicians try to limit the number of times subsequent surgical operations are required to maintain the pacemaker device. Additional surgical operations may increase the patient's risk of further heart complications or infections. However, sometimes a surgical operation cannot be avoided. It is necessary to extract or replace the leads in the event of complications such as lead fractures, formation of scar tissue at the electrode-myocardium interface expediting battery depletion, onset of infections at the site of the pulse generator or leads, and blockage in the subclavian vein. Lead extraction can be especially challenging when leads become surrounded by fibrous tissue and tethered to the heart and veins. Moreover, when the

pacemaker battery is low, a surgical operation is required to replace the pacemaker device. The lithium iodine battery is currently the gold standard for pacemaker batteries due to its long life. However, improving power efficiency and the battery life for pacemakers continues to be an advancing area of research with modern pacemakers.

List of Figures

1	Illustrations of the CCS	2
2	ECG Waveform.	4
3	Wiggers diagram.	5
4	Mechanical and electrical events of the cardiac cycle.	6
5	Pulse generator diagrams.	9
6	Pacemaker leads placement.	10
7	Active and Passive Fixation Bipolar Pacemaker Leads.	10
8	Concepts of Programmability and Telemetry.	11

References

- [IPMo6] Blagoy Iliev, Gheorghe Pop, and Gerard Meijer. “In-vivo Blood Characterization System”. In: May 2006, pp. 1781–1785. DOI: 10.1109/IMTC.2006.328249.
- [BSSo8] S.S. Barold, R.X. Stroobandt, and A.F. Sinnaeve. *Cardiac Pacemakers Step-by-Step: An Illustrated Guide*. Wiley, 2008. ISBN: 978-1-4051-4065-2. URL: <https://books.google.ca/books?id=1Cs0HpKsby4C>.
- [CBo8] A.W.C. Chow and A.E. Buxton. *Implantable Cardiac Pacemakers and Defibrillators: All You Wanted to Know*. Wiley, 2008. ISBN: 9780470790427. URL: <https://books.google.ca/books?id=jECAnTGBoUC>.
- [Vah09] Anwar Vahed. “3-Lead Wireless ECG”. In: 2009.
- [Nek16] Paula Nekic. *Pacemaker learning package*. 2016. URL: https://www.aci.health.nsw.gov.au/__data/assets/pdf_file/0003/306588/Pacemaker_Learning_Package.pdf.
- [] *Conduction Disorders*. [Online; accessed September 7, 2020]. URL: <https://www.heart.org/en/health-topics/arrhythmia/about-arrhythmia/conduction-disorders>.
- [] *Heart arrhythmia*. [Online; accessed September 7, 2020]. URL: <https://www.mayoclinic.org/diseases-conditions/heart-arrhythmia/symptoms-causes/syc-20350668>.

Images

- [Dan12] DanielChangMD revised original work of DestinyQx; Redrawn as SVG by xavax / CC BY-SA (<https://creativecommons.org/licenses/by-sa/2.5>). *Wiggers Diagram*. [Online; accessed September 6, 2020]. 2012. URL: https://commons.wikimedia.org/wiki/File:Wiggers_Diagram.svg.
- [] *Boston Scientific Essentio MRI Pacemaker*. [Online; accessed September 8, 2020]. URL: https://www.bostonscientific.com/content/dam/bostonscientific/Rhythm%5C%20Management/portfolio-group/Pacemakers/ESSENTIO/Essentio-MRI_960x960.png.
- [] *Cardiac Cycle Diagram*. [Online; accessed September 7, 2020]. URL: <http://3.bp.blogspot.com/-k6Y7KP050mc/TZSLwLmUY2I/AAAAAAAAACg/US6WL80aqWA/s1600/cardiac%5C%2Bcycle%5C%2Bdiagram.JPG>.
- [] *Electrical Events of the Cardiac Cycle*. [Online; accessed September 7, 2020]. URL: <https://i.pinimg.com/originals/e0/02/1b/e0021b%5C%20f51ff8da59cc04e80fd027ef19.jpg>.

- *Pacemaker Leads Placement*. [Online; accessed September 8, 2020]. URL: http://3.bp.blogspot.com/_ZWqgYBR0GHw/TR5GMQLw7sI/AAAAAAAAACFo/IbDLiixIUNg/s1600/Pacemaker+Leads+Placement.gif.
- *Pacemaker Placed In Body*. [Online; accessed September 8, 2020]. URL: <https://www.researchgate.net/publication/317153404/figure/fig1/AS:761433004466176@1558551183953/Pacemaker-placed-in-body.png>.
- *Photoheader Implantable*. [Online; accessed September 8, 2020]. URL: https://3.bp.blogspot.com/_jQ-v-9IkLys/TUJ3G0_6ihI/AAAAAAAAAWU/JiHFh0EzV-0/s1600/photoheader-implantable.jpg.
- *Tekening Pacemaker*. [Online; accessed September 8, 2020]. URL: http://www.vitatron.com/images/tekening_pacemaker.gif.

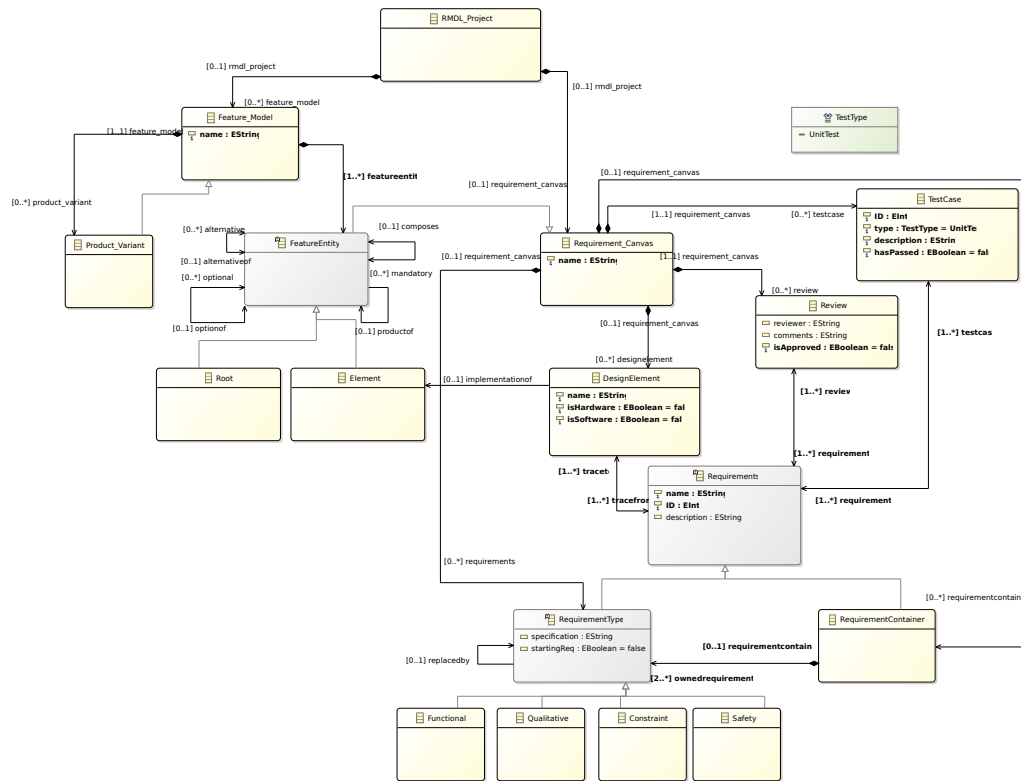


Figure A.1: CyclicL metamodel. Shows the specification for both the requirement canvas and feature modelling portions of CyclicL.

A.4 CyclicL Complete Metamodel

A.5 Simple Gherkin-based Requirement Specification Implementation

```
grammar org.xtext.mcmaster.requirements.gherkin.rgh.Rgh with
    org.eclipse.xtext.common.Terminals
```

generate rgh

"http://www.xtext.org/mcmaster/requirements/gherkin/rgh/Rgh"


```
import "http://www.eclipse.org/emf/2002/Ecore" as ecore
```

Model:

```
(requirement+=Steps)*;
```

Steps:

```
Context | Given | When | Then;
```

Context:

```
{Context} 'Check' '{' (feature+=Feature)* '}' 'exists'  
;
```

QualifiedName:

```
ID('.' ID)*  
;
```

Feature:

```
'Feature' name=Description  
;
```

Given:

```
{Given} 'Given' '{' (preconditions+=Precondition)* '}';
```

When:

```
{When} 'When' '{' (events+=Event)* '}';
```

Then:

```
{Then} 'Then' '{' (postconditions+=Postcondition)* '}';
```

```
Precondition:
```

```
'Precond' name = ID ':' description = Description  
;
```

```
Event:
```

```
'Event' name=ID ':' description = Description  
;
```

```
Postcondition:
```

```
'Postcond' name=ID ':' description=Description  
;
```

```
Description returns Description:
```

```
value = STRING;
```

```
EString returns ecore::EString:
```

```
STRING;
```

Bibliography

- [1] Steve R Palmer and Mac Felsing. *A practical guide to feature-driven development*. Pearson Education, 2001 (cit. on pp. [1](#), [29](#)).
- [2] Kyo Kang et al. “Feature-oriented domain analysis (FODA) feasibility study. Software Engineering Institute”. In: *Universitas Carnegie Mellon, Pittsburgh, Pennsylvania* (1990) (cit. on pp. [3](#), [15](#), [29](#), [53](#), [62](#)).
- [3] Kyo C Kang et al. “FORM: A feature-; oriented reuse method with domain-; specific reference architectures”. In: *Annals of software engineering* 5.1 (1998), pp. 143–168 (cit. on pp. [3](#), [53](#)).
- [4] A van Lamsweerde. *Requirements engineering: from system goals to UML models to software specifications*. John Wiley & Sons, Ltd, 2009 (cit. on pp. [3](#), [8](#), [18](#), [25](#), [34](#)).
- [5] Christian Kastner et al. “FeatureIDE: A tool framework for feature-oriented software development”. In: *2009 ieee 31st international conference on software engineering*. IEEE. 2009, pp. 611–614 (cit. on pp. [5](#), [15](#), [54](#), [63](#)).
- [6] Thomas Thüm et al. “FeatureIDE: An extensible framework for feature-oriented software development”. In: *Science of Computer Programming* 79 (2014), pp. 70–85 (cit. on pp. [5](#), [15](#), [30](#), [54](#), [63](#)).

- [7] Charles W. Krueger. “BigLever software gears and the 3-tiered SPL methodology”. In: *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion. OOPSLA '07*. Montreal, Quebec, Canada: Association for Computing Machinery, 2007, 844–845. ISBN: 9781595938657. URL: <https://doi-org.libaccess.lib.mcmaster.ca/10.1145/1297846.1297918> (cit. on pp. 5, 15, 30, 63).
- [8] Peter Höfner, Ridha Khedri, and Bernhard Möller. “Feature algebra”. In: *International Symposium on Formal Methods*. Springer. 2006, pp. 300–315 (cit. on pp. 5, 16, 54).
- [9] Peter Höfner, Ridha Khedri, and Bernhard Möller. “An algebra of product families”. In: *Software & Systems Modeling* 10 (2011), pp. 161–182 (cit. on pp. 5, 16, 54).
- [10] Government of Canada. *Canadian Motor Vehicle Traffic Collision Statistics: 2022*. <https://tc.canada.ca/en/road-transportation/statistics-data/canadian-motor-vehicle-traffic-collision-statistics-2022>. [Online; accessed 2024-11-27]. 2024 (cit. on p. 7).
- [11] OMG Available Specification. “Omg systems modeling language (omg sysml™), v1. 6”. In: *Object Management Group* (2019) (cit. on pp. 8, 20, 41, 47, 70, 108).
- [12] Bertrand Meyer. *Handbook of Requirements and Business Analysis*. Springer, 2022 (cit. on pp. 9, 18, 26, 33, 45).
- [13] Kyo C Kang, Jaejoon Lee, and Patrick Donohoe. “Feature-oriented product line engineering”. In: *IEEE software* 19.4 (2002), pp. 58–65 (cit. on pp. 15, 53).

- [14] Krzysztof Czarnecki, Simon Helsen, and Ulrich Eisenecker. “Staged configuration using feature models”. In: *Software Product Lines: Third International Conference, SPLC 2004, Boston, MA, USA, August 30-September 2, 2004. Proceedings 3*. Springer. 2004, pp. 266–283 (cit. on p. 16).
- [15] Mariem Mefteh, Nadia Bouassida, and Hanène Ben-Abdallah. “Implementation and evaluation of an approach for extracting feature models from documented uml use case diagrams”. In: *Proceedings of the 30th annual acm symposium on applied computing*. 2015, pp. 1602–1609 (cit. on p. 16).
- [16] Jose-Miguel Horcas, Mónica Pinto, and Lidia Fuentes. “Software product line engineering: a practical experience”. In: *Proceedings of the 23rd International Systems and Software Product Line Conference-Volume A*. 2019, pp. 164–176 (cit. on p. 17).
- [17] Martin Becker, Rick Rabiser, and Goetz Botterweck. “Not Quite There Yet: Remaining Challenges in Systems and Software Product Line Engineering as Perceived by Industry Practitioners”. In: *Proceedings of the 28th ACM International Systems and Software Product Line Conference*. 2024, pp. 179–190 (cit. on p. 17).
- [18] Damir Bilic et al. “An integrated model-based tool chain for managing variability in complex system design”. In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE. 2019, pp. 288–293 (cit. on p. 17).
- [19] Robbert Jongeling et al. “Co-evolution of simulink models in a model-based product line”. In: *Proceedings of the 23rd ACM/IEEE interna-*

- tional conference on model driven engineering languages and systems*. 2020, pp. 263–273 (cit. on p. 17).
- [20] Len Wozniak and Paul Clements. “How automotive engineering is taking product line engineering to the extreme”. In: *Proceedings of the 19th International Conference on Software Product Line*. SPLC ’15. Nashville, Tennessee: Association for Computing Machinery, 2015, 327–336. ISBN: 9781450336130. URL: <https://doi.org/10.1145/2791060.2791071> (cit. on p. 17).
- [21] Suzanne Robertson and James Robertson. *Mastering the requirements process: Getting requirements right*. Addison-wesley, 2012 (cit. on p. 19).
- [22] James Robertson and Suzanne Robertson. “Volere”. In: *Requirements Specification Templates* (2000) (cit. on pp. 19, 49, 71).
- [23] Ian Sommerville and Pete Sawyer. *Requirements engineering: a good practice guide*. John Wiley & Sons, Inc., 1997 (cit. on p. 19).
- [24] Ian Sommerville and Pete Sawyer. “Viewpoints: principles, problems and a practical approach to requirements engineering”. In: *Annals of software engineering* 3.1 (1997), pp. 101–130 (cit. on p. 19).
- [25] Martin Fowler. *UML distilled: a brief guide to the standard object modeling language*. Addison-Wesley Professional, 2018 (cit. on pp. 20, 25).
- [26] Keng Siau and Lihyunn Lee. “Are use case and class diagrams complementary in requirements analysis? An experimental study on use case and class diagrams in UML”. In: *Requirements engineering* 9 (2004), pp. 229–237 (cit. on p. 20).

- [27] Thomas von der Maßen and Horst Lichter. “Modeling variability by UML use case diagrams”. In: *Proceedings of the International Workshop on Requirements Engineering for product lines*. 2002, pp. 19–25 (cit. on p. 20).
- [28] Alain Wegmann and Guy Genilloud. “The role of “Roles” in use case diagrams”. In: *International Conference on the Unified Modeling Language*. Springer. 2000, pp. 210–224 (cit. on p. 20).
- [29] Sandy Beidu and Joanne M. Atlee. “Detecting Feature Interactions in FORML Models”. In: *From Software Engineering to Formal Methods and Tools, and Back: Essays Dedicated to Stefania Gnesi on the Occasion of Her 65th Birthday*. Ed. by Maurice H. ter Beek, Alessandro Fantechi, and Laura Semini. Cham: Springer International Publishing, 2019, pp. 220–235. ISBN: 978-3-030-30985-5. URL: https://doi.org/10.1007/978-3-030-30985-5_14 (cit. on p. 20).
- [30] Pourya Shaker, Joanne M. Atlee, and Shige Wang. “A feature-oriented requirements modelling language”. In: *2012 20th IEEE International Requirements Engineering Conference (RE)*. 2012, pp. 151–160 (cit. on p. 20).
- [31] Milena Rota Sena Marques, Eliane Siegert, and Lisane Brisolara. “Integrating UML, MARTE and sysml to improve requirements specification and traceability in the embedded domain”. In: *2014 12th IEEE International Conference on Industrial Informatics (INDIN)*. 2014, pp. 176–181 (cit. on p. 21).
- [32] Jane Cleland-Huang et al. “Best Practices for Automated Traceability”. In: *Computer* 40.6 (2007), pp. 27–35 (cit. on p. 21).

- [33] Ankit Agrawal and Jane Cleland-Huang. “Leveraging Traceability to Integrate Safety Analysis Artifacts into the Software Development Process”. In: *31st IEEE International Requirements Engineering Conference, RE 2023 - Workshops, Hannover, Germany, September 4-5, 2023*. Ed. by Kurt Schneider, Fabiano Dalpiaz, and Jennifer Horkoff. IEEE, 2023, pp. 475–478 (cit. on p. 21).
- [34] Jane Cleland-Huang et al. “Requirements-driven configuration of emergency response missions with small aerial vehicles”. In: *SPLC ’20: 24th ACM International Systems and Software Product Line Conference, Montreal, Quebec, Canada, October 19-23, 2020, Volume A*. 2020, 26:1–26:12. URL: <https://doi.org/10.1145/3382025.3414950> (cit. on p. 21).
- [35] Mehdi Mirakhorli and Jane Cleland-Huang. “Tracing architectural concerns in high assurance systems (NIER track)”. In: *Proceedings of the 33rd International Conference on Software Engineering*. 2011, pp. 908–911 (cit. on p. 21).
- [36] Philipp Heisig et al. “A generic traceability metamodel for enabling unified end-to-end traceability in software product lines”. In: *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*. 2019, pp. 2344–2353 (cit. on p. 21).
- [37] Ryosuke Tsuchiya et al. “Recovering traceability links between requirements and source code in the same series of software products”. In: *Proceedings of the 17th International Software Product Line Conference*. 2013, pp. 121–130 (cit. on p. 22).

- [38] Justin Kelleher. “A reusable traceability framework using patterns”. In: *Proceedings of the 3rd international workshop on Traceability in emerging forms of software engineering*. 2005, pp. 50–55 (cit. on p. 22).
- [39] Jonathan I Maletic, Michael L Collard, and Bonita Simoes. “An XML based approach to support the evolution of model-to-model traceability links”. In: *Proceedings of the 3rd international workshop on Traceability in emerging forms of software engineering*. 2005, pp. 67–72 (cit. on p. 22).
- [40] Hazeline U Asuncion, Arthur U Asuncion, and Richard N Taylor. “Software traceability with topic modeling”. In: *Proceedings of the 32nd ACM/IEEE international conference on Software Engineering-Volume 1*. 2010, pp. 95–104 (cit. on p. 22).
- [41] Nicolas Anquetil and Vasco Amaral. “A Model-Driven Traceability Framework to Software Product Line Development”. In: *ECMDA Traceability... (2008)* (cit. on p. 22).
- [42] André Sousa et al. “A model-driven traceability framework to software product line development”. In: *ECMDA Traceability Workshop (ECMDA-TW)*. 2008, pp. 97–109 (cit. on p. 23).
- [43] Thomas Chiang et al. “A Tool For Feature-Requirement Traceability Using Requirement Canvas and Encapsulation”. In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. MODELS Companion '24*. Linz, Austria: Association for Computing Machinery, 2024, 725–734. ISBN: 9798400706226. URL: <https://doi.org/10.1145/3652620.3687816> (cit. on p. 24).
- [44] Thomas Chiang et al. “Mapping Requirements to Features to Create Traceability in Product Line Models”. In: *Proceedings of the ACM/IEEE*

- 27th International Conference on Model Driven Engineering Languages and Systems. MODELS Companion '24*. Linz, Austria: Association for Computing Machinery, 2024, 1180–1187. ISBN: 9798400706226. URL: <https://doi.org/10.1145/3652620.3688349> (cit. on pp. 24, 64).
- [45] Government of Canada. *Measured standing height, by age and sex, household population, Canada, 2009 to 2011*. <https://www150.statcan.gc.ca/n1/pub/82-626-x/2013001/t023-eng.htm>. [Online; accessed 2024-11-27]. 2015 (cit. on p. 33).
- [46] Government of Canada. *Measured weight, by age and sex, household population, Canada, 2009 to 2011*. <https://www150.statcan.gc.ca/n1/pub/82-626-x/2013001/t024-eng.htm>. [Online; accessed 2024-11-27]. 2015 (cit. on p. 33).
- [47] Yves Wautelet et al. “Building a rationale diagram for evaluating user story sets”. In: *2016 IEEE Tenth International Conference on Research Challenges in Information Science (RCIS)*. IEEE. 2016, pp. 1–12 (cit. on p. 34).
- [48] Lidia López, Xavier Franch, and Jordi Marco. “Specialization in i* strategic rationale diagrams”. In: *Conceptual Modeling: 31st International Conference ER 2012, Florence, Italy, October 15-18, 2012. Proceedings 31*. Springer. 2012, pp. 267–281 (cit. on p. 34).
- [49] “Gherkin Reference”. In: (2024). URL: <https://cucumber.io/docs/gherkin/reference/> (cit. on pp. 48, 57, 76).
- [50] Vladimir Viyović, Mirjam Maksimović, and Branko Perisić. “Sirius: A rapid development of DSM graphical editor”. In: *IEEE 18th Interna-*

- tional Conference on Intelligent Engineering Systems INES 2014*. IEEE. 2014, pp. 233–238 (cit. on pp. 60, 111).
- [51] Moritz Eysholdt and Heiko Behrens. “Xtext: implement your language faster than the quick and dirty way”. In: *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*. 2010, pp. 307–309 (cit. on pp. 60, 76).
- [52] Mark Richters and Martin Gogolla. “OCL: Syntax, semantics, and tools”. In: *Object Modeling with the OCL: The Rationale behind the Object Constraint Language*. Springer, 2002, pp. 42–68 (cit. on p. 68).
- [53] Jordi Cabot and Martin Gogolla. “Object constraint language (OCL): a definitive guide”. In: *International school on formal methods for the design of computer, communication and software systems*. Springer, 2012, pp. 58–90 (cit. on p. 68).
- [54] Daniel Moody. “The “Physics” of Notations: Toward a Scientific Basis for Constructing Visual Notations in Software Engineering”. In: *IEEE Transactions on Software Engineering* 35.6 (2009), pp. 756–779 (cit. on p. 75).
- [55] Kamil Ncieja. *Writing Great Specifications: Using Specification by Example and Gherkin*. Simon and Schuster, 2017 (cit. on p. 76).
- [56] Government of Canada. “GUIDANCE DOCUMENT Guidance on the Risk-based Classification System for NonIn Vitro Diagnostic Devices (non-IVDDs)”. In: (2015). URL: <https://www.canada.ca/en/health-canada/services/drugs-health-products/medical-devices/application-information/guidance-documents/guidance-document-guidance->

[risk-based-classification-system-non-vitro-diagnostic.html](#)

(cit. on pp. 84, 85).

- [57] Government of Canada. “GUIDANCE FOR INDUSTRY Keyword Index to Assist Manufacturers in Verifying the Class of Medical Devices”. In: (2006). URL: <https://www.canada.ca/en/health-canada/services/drugs-health-products/medical-devices/application-information/guidance-documents/guidance-industry-keyword-assist-manufacturers-class-medical-devices.html> (cit. on pp. 84, 85).
- [58] Boston Scientific. “Pacemaker system specification”. In: *Boston Scientific* (2007) (cit. on pp. 87, 96, 127).
- [59] Obeo. *Acceleo Query Language Documentation*. 2015. URL: <https://eclipse.dev/acceleo/documentation/> (cit. on p. 120).
- [60] Vivien You WANG. “Identification and Documentation of Environmental Assumptions for the PACEMAKER System”. PhD thesis. 2012 (cit. on p. 127).